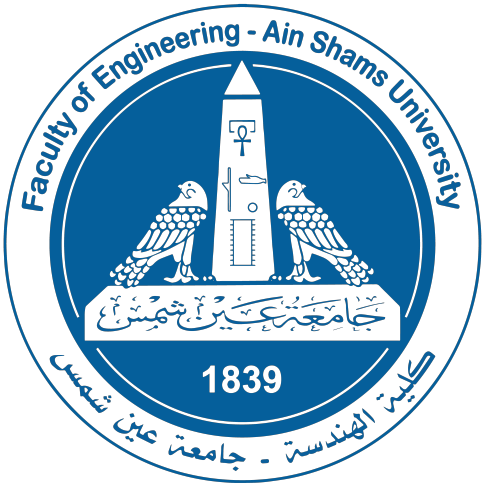

Graduation Project

-

SmartShield Intrusion Prevention System

Team Members

Name	ID
Mostafa Hamada Hassan Ahmed	2100587
Zeyad Magdy Roushdy	2100393
Ahmed Ashraf Ahmed	2100476
Karim Walid Moussa Shehata	2100295

Supervised by Dr. Islam El-Maddah

Table of Contents

List of Tables

List of Figures

Appendix

1

Problem Statement

1.1 Operational Technology (OT): Systems, Security, and Industrial Significance

1.1.1 Definition and Core Characteristics

Operational Technology (OT) encompasses specialized hardware and software systems that **monitor, control, and automate physical processes** in industrial and infrastructure environments. These systems interact directly with the physical world, converting digital commands into physical actions and translating sensor measurements into digital data.

Key Characteristics:

- **Real-Time Operation:** Requires deterministic, near-instantaneous response times to ensure reliable control of industrial machinery.
- **Safety & Availability Focus:** Prioritizes operational continuity and safety; downtime can lead to physical hazards, environmental damage, or significant financial loss.
- **Long Lifecycles:** Assets often remain in service for decades, leading to reliance on legacy software, hardware, and proprietary protocols.
- **Predictable Behavior:** Designed for stability and repeatability, with low tolerance for unplanned changes or disruptions.

Common Industries: Manufacturing, Energy, Utilities, Transportation, Water Treatment, and other Critical Infrastructure sectors.

1.1.2 Key Components and Examples

OT systems integrate various specialized devices and software to automate industrial processes.

Core Components:

- **Programmable Logic Controllers (PLCs):** Industrial computers that automate electromechanical processes.
- **Supervisory Control and Data Acquisition (SCADA):** Systems for centralized monitoring and control of dispersed assets.
- **Distributed Control Systems (DCS):** Integrated control systems for complex, process-oriented plants.
- **Human-Machine Interfaces (HMIs):** Dashboards allowing operators to interact with the OT system.
- **Sensors & Actuators:** Devices that measure physical parameters (e.g., temperature, pressure) and execute physical actions (e.g., opening a valve).

Additional Devices: Remote Terminal Units (RTUs), Industrial Internet of Things (IIoT) devices, industrial robots, and smart grid systems.

1.1.3 OT vs. Information Technology (IT)

OT and IT serve fundamentally different purposes and are managed under different principles.

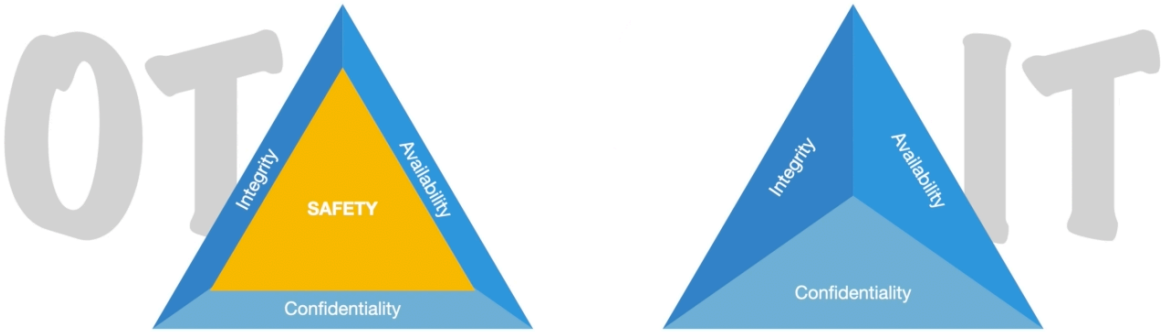

Figure 1.1: OT vs IT triad

Aspect	Operational Technology (OT)	Information Technology (IT)
Primary Goal	Control physical processes; ensure safety, reliability, and availability.	Manage data and business information; ensure confidentiality, integrity, and availability (CIA).
Performance Need	Deterministic, real-time response is critical.	High throughput is valued; latency is more tolerable.
Update Cycles	Infrequent (years), due to stability requirements and long lifecycles.	Frequent patches and updates (weeks/ months).
Risk Priority	**Availability > Integrity > Confidentiality.**	**Confidentiality > Integrity > Availability.**
Impact of Failure	Physical damage, environmental harm, safety threats, operational shutdown.	Data loss, financial cost, reputational damage.

Table 1.1: OT vs IT

IT-OT Convergence: The increasing interconnection of IT and OT networks enables improved data analytics, predictive maintenance, and operational efficiency. However, this convergence significantly expands the cybersecurity attack surface.

1.2 The Security Gap: Why OT Needs IT Protection

While Information Technology (IT) prioritizes data confidentiality and integrity, OT systems prioritize Availability and Safety above all else. This fundamental difference creates a significant security gap:

- **Legacy Vulnerability:** OT assets often remain in service for decades and rely on legacy software or proprietary protocols that lack modern security features.
- **Patching Constraints:** Critical systems often cannot be taken offline for updates, leaving them unpatched and vulnerable to known exploits.
- **The Convergence Risk:** As OT networks increasingly connect to IT networks to improve efficiency (IT-OT Convergence), these traditionally isolated (air-gapped) systems are now exposed to the expanded attack surface of the internet.
- **Flat Networks:** Traditionally unsegmented architectures allow threats to move laterally with ease.
- **Physical Safety Implications:** Cyber attacks can lead directly to equipment damage, operational disruption, or threats to human safety.

Historical Attacks: Notable incidents demonstrating these risks include Stuxnet [1] (targeting Iranian nuclear facilities), the Ukrainian power grid cyberattacks [2], and the Colonial Pipeline ransomware incident [3].

1.2.1 Case Study: Stuxnet (2010)

Often cited as the first "digital weapon," Stuxnet targeted Iranian uranium enrichment centrifuges at the Natanz facility.

- **Target:** Uranium enrichment infrastructure.
- **Attack Vector:** Malicious USB drives were used to bridge the air-gap isolating the facility's network.
- **Mechanism:** The worm exploited multiple zero-day Windows vulnerabilities to propagate through the IT network before specifically targeting Siemens Step7 software. It reprogrammed the Programmable Logic Controllers (PLCs) to alter the frequency of the centrifuges, causing them to spin at destructive speeds while displaying normal operating data to monitoring systems.
- **Outcome:** The attack resulted in the physical destruction of approximately 1,000 centrifuges, significantly delaying Iran's nuclear program.

1.2.2 Case Study: Ukrainian Power Grid (2015)

This incident marked the first confirmed cyberattack to successfully take down a power grid.

- **Target:** Three energy distribution companies in Ukraine.
- **Attack Vector:** Spear-phishing emails containing BlackEnergy malware were sent to IT staff.
- **Mechanism:** Attackers gained entry via the corporate IT network, harvested credentials, and pivoted to the OT network. They remotely accessed SCADA workstations to open breakers and cut power, while simultaneously deploying KillDisk malware to wipe systems.
- **Outcome:** Approximately 225,000 customers lost power for several hours, demonstrating that IT vectors can be leveraged to cause physical critical infrastructure failure.

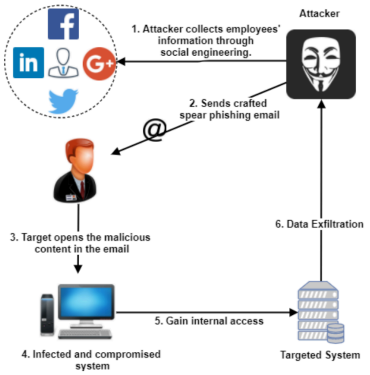

Figure 1.2: Spear Phishing Example

1.2.3 Case Study: Colonial Pipeline (2021)

A prime example of how IT dependencies can paralyze OT operations, even without direct OT compromise.

- **Target:** The largest fuel pipeline in the United States.
- **Attack Vector:** A compromised VPN password for a legacy account on the IT network.
- **Mechanism:** The DarkSide ransomware group encrypted the company's IT billing and administrative systems.
- **Outcome:** While the OT control systems were not directly infected, the pipeline was shut down proactively due to the inability to track billing and concerns about the potential spread of the malware. This led to widespread fuel shortages and panic buying across the U.S. East Coast.

1.2.4 Best Practices for Securing OT

Securing OT requires tailored strategies that respect operational constraints.

- **Network Segmentation & Microsegmentation:** Isolate critical OT infrastructure from IT networks and within the OT environment to limit attack spread.
- **Continuous OT Visibility and Monitoring:** Implement passive monitoring and active querying to maintain an accurate asset inventory and detect anomalies.
- **Zero Trust Architectures:** Apply strict access controls, least-privilege principles, and continuous verification for all users and devices.

- **Defense-in-Depth:** Employ layered security controls, including next-generation firewalls, intrusion detection/prevention systems (IDS/IPS), and specialized OT anomaly detection tools.
- **OT-Specific Training:** Educate both engineering and security staff on the unique risks and protocols in OT environments.
- **Incident Response Planning:** Develop and regularly test response plans designed for OT-specific impacts and operational continuity requirements.

1.3 SmartShield as the Protection Line

Because OT systems are often too fragile or outdated to host their own security agents, the defense strategy must shift to the network level. SmartShield is implemented as a security solution that acts as the "shield" for both IT and OT environments.

By deploying SmartShield at the IT/OT boundary or within the network, it provides the necessary Defense-in-Depth. It passively monitors traffic without disrupting the real-time requirements of the industrial processes, identifying reconnaissance (like Modbus scanning) and threats before they can impact physical operations. SmartShield effectively serves as the modern protection line for an infrastructure that cannot protect itself.

1.3.1 Importance in Industry and Infrastructure

OT is the foundational layer for automation and control in modern industry and critical infrastructure, enabling:

- **Enhanced Operational Efficiency** through automated, precise control of physical processes.
- **Improved Safety and Reliability** by continuously monitoring and managing industrial systems.
- **Predictive Maintenance** to reduce unplanned downtime and associated costs.
- **Data-Driven Insights** via integration with IT systems, supporting better business and operational decision-making.

1.3.2 Conclusion

Operational Technology is essential for the automation and control of the physical world. Its distinct characteristics, long lifecycles, real-time demands, and safety-critical nature; combined with increasing IT convergence, create complex operational and cybersecurity challenges. A deep understanding of OT's unique ecosystem, vulnerabilities, and tailored

security practices is crucial for protecting industrial operations and maintaining the resilience of critical infrastructure.

2

SmartShield

2.1 Introduction

SmartShield is a Python/Zeek-based intrusion prevention system that uses machine learning to detect malicious behaviors in the network traffic. Smart Shield was designed to focus on targeted attacks, to detect of command and control channels, and to provide good visualisation for the analyst. Smart Shield is able to analyze real live traffic from the device and the large network captures in the type of a `pcap` files, Suricata, Zeek and Argus flows. As a result, Smart Shield highlights suspicious behaviour and connections that needs to be more deeply analyzed.

The system sits passively on a Linux server at the IT/OT network boundary, receiving a mirrored copy of all traffic via a SPAN port or passive TAP Device (See Figure 2.1). It detects 20 categories of OT/ICS attacks, generates structured evidence, and optionally blocks attackers through `iptables`, with a critical OT-safe mechanism that prevents any PLC, HMI, or SCADA server from ever being auto-blocked.

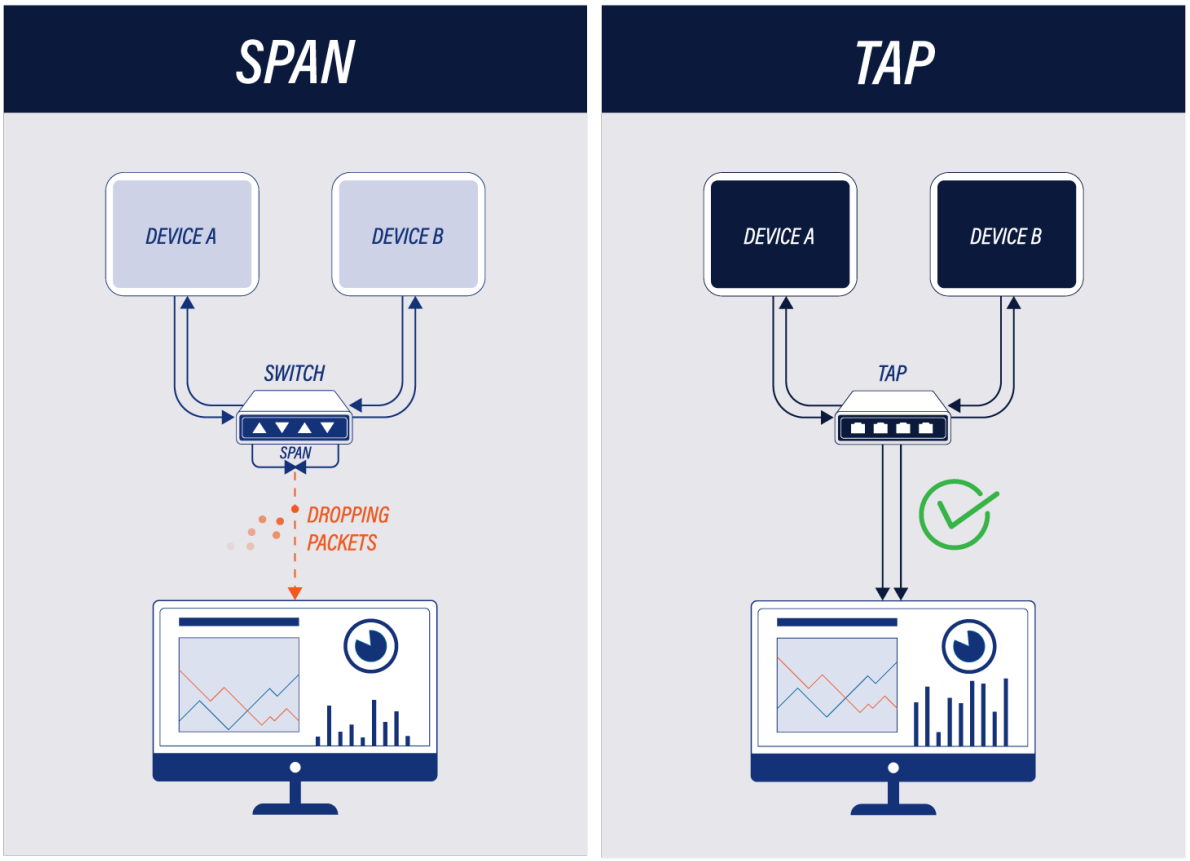

Figure 2.1: SPAN Port and TAP Device

2.2 Use-Case Diagram

The Use Case Diagram illustrates the interactions between SmartShield and its key actors: Security Analysts, Developers, and ML Engineers. It highlights the system's core functionalities, such as network traffic analysis, threat detection, alert management, and the machine learning lifecycle, showing how users leverage SmartShield to secure the network.

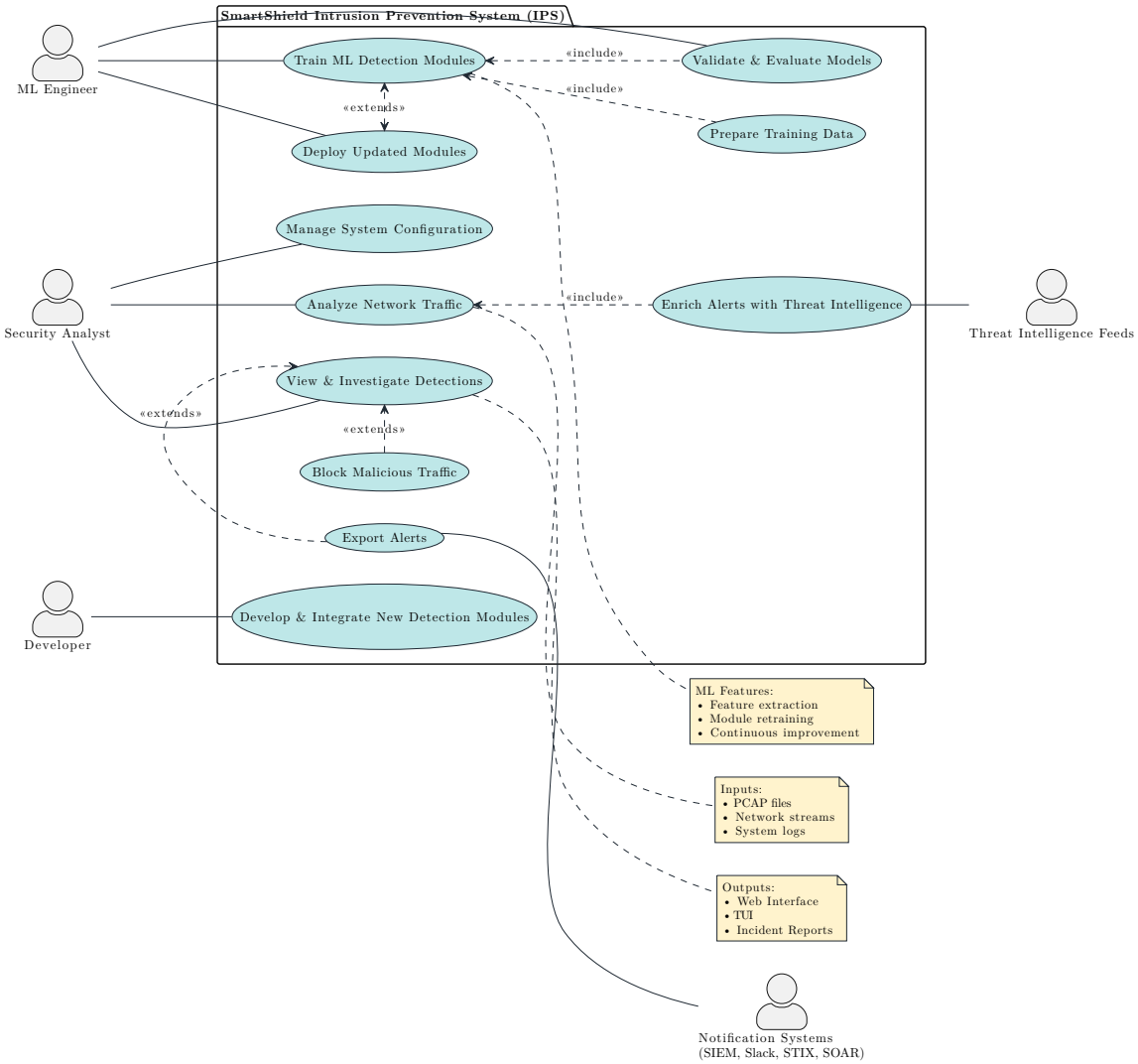

Figure 2.2: Use Case Diagram

2.3 User Stories

User stories describe features from the user's perspective, focusing on the "Who", "What", and "Why". SmartShield follows the standard Agile format: **As a [role], I want [action], So that [benefit]**. Each story includes Acceptance Criteria to confirm successful implementation.

2.3.1 EPIC: Deployment & Configuration

Goal: Deploy and configure SmartShield reliably in different environments.

User Story – Centralized Configuration Management
As a Security Analyst, **I want to** configure SmartShield using a single, well-documented YAML configuration file, **So that** I can manage interfaces, block/allow lists, detection modules, and outputs consistently.
Acceptance Criteria: ☑ A default configuration file is generated during installation ☑ Configuration changes are applied on restart without errors ☑ The file supports core features: interfaces, detection modules, threat intelligence, alerting outputs

User Story – Containerized Deployment
As a DevOps Engineer, **I want to** deploy SmartShield as a Docker container, **So that** it can be consistently run in cloud and on-prem environments.
Acceptance Criteria: ☑ A Dockerfile and docker-compose configuration are provided ☑ The container starts successfully and analyzes live or sample traffic ☑ Environment variables can override key configuration options ☑ Logs are written to stdout/stderr for orchestration platforms

2.3.2 EPIC: Monitoring & Alerting

Goal: Detect, visualize, and respond to security incidents effectively.

User Story – Real-Time Alert Dashboard
As a Security Analyst, **I want to** view real-time alerts ranked by severity and confidence, **So that** I can prioritize and investigate critical threats quickly.
Acceptance Criteria: ☑ Running SmartShield launches a web-based dashboard ☑ Alerts display timestamps, confidence scores, categories, and IP metadata ☑ Analysts can drill down into flow-level evidence ☑ The dashboard updates automatically without manual refresh

User Story – Alert Response Actions
As a Security Analyst, **I want to** block or export detected threats directly from the alert view, **So that** I can respond immediately or forward incidents to external systems.
Acceptance Criteria: ☑ Alerts can trigger traffic blocking based on policy ☑ Alerts can be exported to SIEM, SOAR, or messaging systems ☑ Exported alerts include contextual and enrichment data ☑ Actions taken are logged for audit purposes

User Story – Whitelist Management
As a Security Analyst, **I want to** whitelist trusted IPs or networks, **So that** false positives are reduced and alert fatigue is minimized.
Acceptance Criteria: ☑ Whitelisted IPs suppress alert generation ☑ CIDR ranges and IP lists are supported ☑ Whitelist configuration persists across restarts ☑ Changes take effect immediately or on command reload

2.3.3 EPIC: Threat Hunting & Investigation

Goal: Perform deep investigation and proactive threat analysis.

User Story – Offline Traffic Analysis
As a Security Analyst, **I want to** analyze offline PCAP files, **So that** I can investigate historical incidents and suspicious activity.
Acceptance Criteria: ☑ SmartShield processes PCAP files without errors ☑ A summary report and interactive timeline are generated ☑ Results can be exported in JSON and CSV formats ☑ Behavioral, signature-based, and ML detections are applied

User Story – IP Context Investigation
As a Security Analyst, **I want to** investigate all activity related to a specific IP address, **So that** I can assess scope and impact.
Acceptance Criteria: ☑ The dashboard supports IP-based search ☑ Associated connections, protocols, and timestamps are displayed ☑ Analysts can pivot to related IPs and sessions ☑ Historical data is retained for investigation

User Story – Custom Detection Module Development
As a Developer, **I want to** develop and integrate custom detection modules, **So that** SmartShield can detect organization-specific threats.
Acceptance Criteria: ☑ A detection module template is provided ☑ Custom modules are auto-loaded at startup ☑ Modules can analyze traffic and raise alerts ☑ Alerts appear in the dashboard with proper attribution

2.3.4 EPIC: Machine Learning Lifecycle

Goal: Continuously improve detection accuracy using machine learning.

User Story – Training Data Preparation
As a ML Engineer, **I want to** prepare labeled training data from traffic and alerts, **So that** machine learning models can be trained effectively.
Acceptance Criteria: ☑ Traffic features are extracted and normalized ☑ Labels include benign, malicious, and false-positive cases ☑ Datasets can be exported for offline experimentation

User Story – ML Model Training & Validation
As a ML Engineer, **I want to** train and validate detection models, **So that** their accuracy and reliability are measurable before deployment.
Acceptance Criteria: ☑ Models can be trained using prepared datasets ☑ Validation metrics (accuracy, precision, recall) are reported ☑ Models failing validation are rejected

User Story – Model Deployment
As a ML Engineer, **I want to** deploy validated models into SmartShield, **So that** improved detections are used in live traffic analysis.
Acceptance Criteria: ☑ Only validated models can be deployed ☑ Model updates do not disrupt running analysis ☑ Active model version is visible in system status

2.3.5 EPIC: Maintenance & System Health

Goal: Keep SmartShield stable, performant, and up to date.

User Story – Threat Intelligence Updates
As a Security Analyst, **I want to** update threat intelligence feeds regularly, **So that** SmartShield detects newly identified threats.
Acceptance Criteria: ☑ Updates fetch the latest threat indicators ☑ Update logs include sources and changes ☑ New indicators are applied to subsequent traffic ☑ Updates can be scheduled automatically

User Story – System Health Monitoring

As a Security Analyst,

I want to monitor system health and performance,

So that SmartShield operates reliably under load.

Acceptance Criteria:

- ☑ CPU, memory, and throughput metrics are visible
- ☑ Packet drops and queue backlogs are detected
- ☑ Metrics are exportable to monitoring platforms

User Story – Detection Tuning & Feedback

As a Security Analyst,

I want to tune detection sensitivity and provide feedback on alerts,

So that detection accuracy improves over time.

Acceptance Criteria:

- ☑ Thresholds can be adjusted dynamically
- ☑ Alerts can be marked as false positives
- ☑ Feedback is logged and usable for ML retraining

2.4 SmartShield Architecture

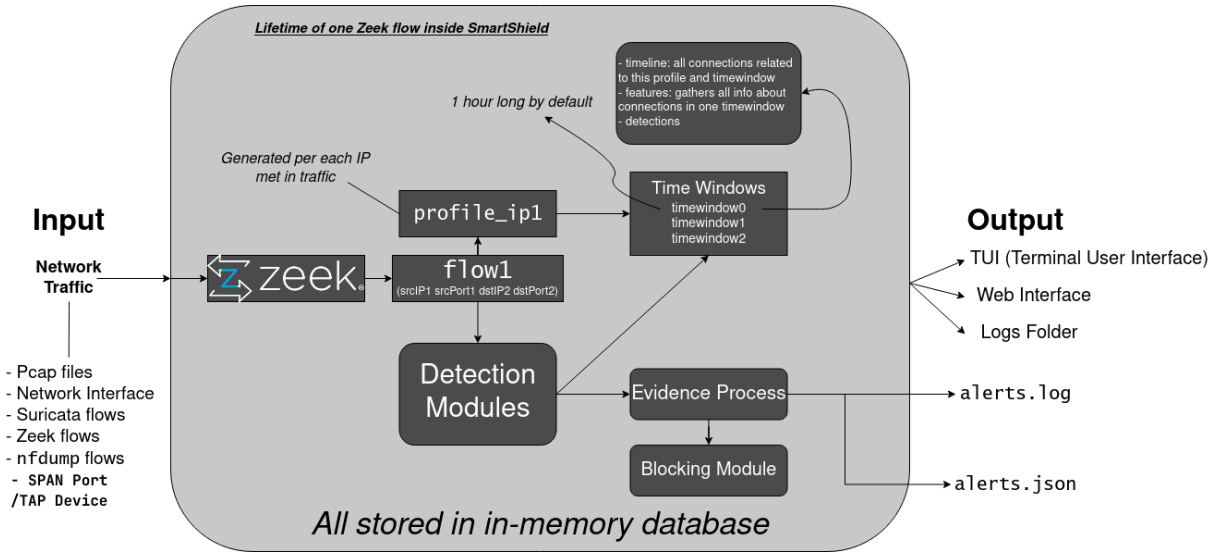

Figure 2.3: SmartShield Architecture

The architecture of Smartshield is:

- To receive some data as input
- To process it to a common format
- To enrich it (gather all possible info about the IPs/MAC/User-Agents etc.)
- To apply detection modules
- To output results

Smartshield is heavily based on the open source *Zeek* network security monitoring tool as input tool for packets from the interface and pcap file, due to its excellent recognition of protocols and easiness to identify the content of the traffic.

Figure 2.3 shows how the data is analyzed by Smartshield. As we can see, Smartshield internally uses Zeek, Smartshield divides flows into profiles and each profile into timewindows, timewindows are numbered from 1 to infinity. SmartShield runs detection modules on each flow and stores all evidence, alerts and features in an appropriate profile structure. All profile info, performed detections, profiles and timewindows' data, is stored inside a Redis database. All flows are read, interpreted by Smartshield, labeled, and stored in the SQLite database in the `output/` dir of each run The output of Smartshield is a folder with logs (`output/` directory) that has `alert.json`, `alerts.log`, `errors.log`.

2.4.1 Network Architecture

SmartShield is deployed at the IT/OT boundary following the ISA-95 / Purdue Model (Table 2.1 and Figure 2.4) for industrial network segmentation. It monitors all traffic passively; OT devices never know it exists.

Zone	Name	Contains
Levels 5-4	IT / Enterprise	Business systems, ERP systems, and internet-connected services. Standard IT security applies above this line.
IT/OT Boundary	DMZ	SmartShield server sits here. A SPAN/TAP receives all traffic crossing the boundary.
Level 3	Operations	SCADA servers, historians (OSIsoft PI), OPC-UA servers, and MES systems.
Level 2	Control	HMIs, engineering workstations, and DCS systems. Run TIA Portal and SCADA client software.
Level 0–1	Field devices	PLCs, RTUs, VFDs, sensors, actuators. These communicate via Modbus, S7Comm, DNP3, PROFINET, and CIP.

Table 2.1: Purdue Model Network Zones

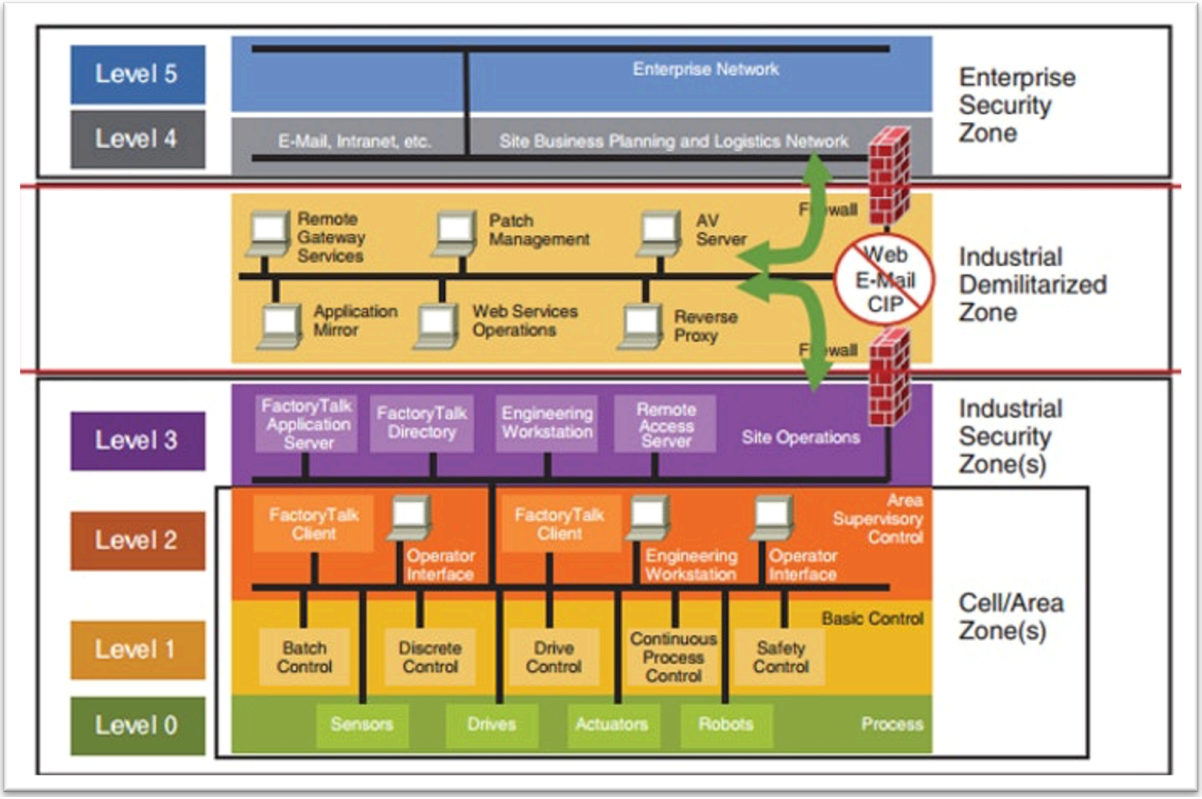

Figure 2.4: Purdue Model

2.4.2 Internal representation of data

SmartShield works at a flow level, instead of a packet level, gaining a high level view of behaviors. Smartshield creates traffic profiles for each IP that appears in the traffic. A

profile contains the complete behavior of an IP address. Each profile is divided into time windows. Each time window is 1 hour long by default and contains dozens of features computed for all connections that start in that time window. Detections are done in each time window, allowing the profile to be marked as uninfected in the next time window. This is what SmartShield stores for each IP/Profile it creates:

- Ipv4 - ipv4 of this profile
- IPv6 - list of ipv6 used by this profile
- Threat_level - the threat level of this profile, updated every TW.
- Confidence - how confident SmartShield is that the threat level is correct
- Past threat levels - history of past threat levels
- Used software - list of software used by this profile, for example SSH, Browser, etc.
- MAC and MAC Vendor - MAC of the IP and the name of the vendor
- Host-name - the name of the IP
- first User-agent - First UA seen use by this profile.
- OS Type - Type of OS used by this profile as extracted from the user agent
- OS Name - Name of OS used by this profile as extracted from the user agent
- Browser - Name of the browser used by this profile as extracted from the user agent
- User-agents history - history of the all user agents used by this profile
- DHCP - if the IP is a dhcp or not
- Starttime - epoch formatted timestamp of when the profile first appeared
- Duration - the standard duration of every TW in this profile
- Modules labels - the labels assigned to this profile by each module
- Gateway - if the IP is the gateway (router) of the network
- Timewindow count - Amount of timewindows in this profile
- ASN - autonomous service number of the IP
- Asnorg - name of the org that own the ASN of this IP
- ASN Number
- SNI - Server name indicator
- Reverse DNS - name of the IP in reverse dns
- Threat Intelligence - If the IP appeared in any of Smartshield blacklist
- Description - Description of this IP as taken from the blacklist
- Blacklist Threat level - threat level of the blacklisted that has this IP
- Passive DNS - All the domains that resolved into this IP
- Certificates - All the certificates that were used by this IP
- Geocountry - Country of this IP

2.4.3 Data Flow Through SmartShield (Summary)

Stage	What Happens
SPAN/TAP port	Switch mirrors traffic to eth1. SmartShield sees a copy of every packet without disrupting OT communications.
Zeek	Parses raw packets into structured logs: conn.log, modbus.log, dnp3.log, notice.log, etc.
Redis pub/sub	Zeek writes events to Redis channels. All SmartShield modules read from Redis, not directly from Zeek.
OT Detection module	Subscribes to new_flow, new_modbus, new_s7comm, new_dnp3, new_notice. Runs detection logic.
Evidence pipeline	Evidence objects flow through evidence_handler.py → alert_handler.py → alerts.json
Blocking module	Subscribes to new_blocking channel. Checks OT-safe list before adding iptables rules.
Export module	Subscribes to export_evidence. Sends to Slack, STIX, Syslog, or HTTP webhook.
Web interface	Flask app reads Redis directly. Shows profiles, alerts, evidence, blocked IPs in real time.

2.4.3.1 Usage of Zeek

SmartShield uses Zeek to generate files for most input types, and this data is used to create the profiles. For example, SmartShield uses this data to create a visual timeline of activities for each time window. This timeline consists of Zeek generated flows and additional interpretation from other logs like `dns.log`, and `http.log`.

2.4.3.2 Usage of Redis database

All the data inside SmartShield is stored in Redis, an in-memory data structure. Redis allows all the modules in SmartShield to access the data in parallel. Apart from read and write operations, SmartShield takes advantage of the Redis messaging system called Redis PUB/SUB. Processes may publish data into the channels, while others subscribe to these channels and process the new data when it is published

2.4.3.3 Usage of SQLite database

SmartShield uses SQLite database to store all flows in SmartShield interpreted format. The SQLite database is stored in the `output/` dir and each flow is labeled to either “malicious” or “benign” based on SmartShield detections. all the labeled flows in the SQLite database can be exported to `tsv` or `json` format. SmartShield does this as saving Redis database is not possible in Docker.

2.4.3.4 Threat Levels

SmartShield has 5 threat levels, more details on how they are utilized are in (2.4.5): SmartShield Evidence & Alert Processing Pipeline

Threat Level	Normalized Value	Description	Example
Info	0	Information, Do nothing	SSH login
Low	0.2	Interesting activity to consider	DNS without connection
Medium	0.5	Suspicious activity that shouldn't happen	PING Sweep
High	0.8	Malicious activity	Password guessing
Critical	1.0	Critical for your security, results in a direct block	Malicious downloaded Files

Table 2.2: SmartShield Threat Levels

2.4.4 Modules

SmartShield is a modular software that can be extended. When SmartShield is run, it spawns several child processes to manage the I/O, to profile attackers and to run the detection modules. Modules are Python-based files that allow any developer to extend the functionality of smartshield. They process and analyze data, perform additional detections and store data in Redis for other modules to consume. Currently, smartshield has the following modules:

Module	Description
ARP	Finds ARP scans and MITM with ARP in the local network.
Exporting Alerts	Syslog (UDP/TCP, RFC-5424) and HTTP webhook export for SIEM integration
IP Info	Finds Geolocation country, and ASN for an IP address.
Update Manager	Takes care of downloading each of the files used by SmartShield, but only if there is a need to update them. It stores and checks the ETags of remote files to know if they changed. It can be configured to update each file with a different frequency. Most importantly it updates all the Threat Intelligence feeds.
Threat Intelligence	Checks if any domain or IP is included in Threat Intelligence feeds. Domains include DNS requests, DNS replies, HTTP hostnames, and TLS SNI. IPs include source and destination IPs, both IPv4 and IPv6.
Port Scan Detector	Detects Horizontal and Vertical port scans.
RNN C&C Detection	Detects command and control channels using recurrent neural network and the behavioral letters.
Flow Alerts	Finds malicious behaviours by analyzing only one flow. Now detects: self-signed certificates, TLS certificates which validation failed, vertical port scans detected by Zeek (contrary to detected by SmartShield), horizontal port scans detected by Zeek (contrary to detected by SmartShield), password guessing in SSH as detected by Zeek, long connection, successful SSH.
Leak Detector	Module to detect leaks of data in the traffic using YARA rules.
HTTP Analyzer	Module to analyze HTTP traffic.
Blocking	Blocks the alerted IPs in the Linux iptables Firewall.
OT Detection Module	Main OT module; 20 attack types, 6 OT protocols, sliding-window flood counters, OT-safe blocking, deduplication

Table 2.3: SmartShield Modules

2.4.4.1 Threat Intelligence Module

The Threat Intelligence (TI) Module is SmartShield's primary mechanism for detecting known malicious indicators in network traffic. This module functions as a rule-based detection engine that compares observed network elements (IPs, domains, hashes) against extensive databases of known threats. It operates by checking both locally stored

threat feeds and querying external intelligence services. In the SmartShield system, this module provides the critical first line of defense by identifying connections to known malicious entities, significantly reducing the attack surface before more sophisticated behavioral analysis is required.

2.4.4.1.1 Input Data

The TI module accepts data from SmartShield's core processing pipeline:

- **Primary Input Type**: Processed network flows and metadata from Zeek logs (`conn.log`, `dns.log`, `http.log`, `ssl.log`, `files.log`).
- **Data Sources**:
  - PCAP files converted to Zeek logs
  - Live network traffic processed by Zeek
  - NetFlow data converted to Zeek format
- **Features Used**:
  - Source and destination IP addresses
  - Domain names from DNS queries/responses and HTTP hosts
  - JA3/JA3S fingerprints from TLS handshakes
  - SSL certificate SHA1 hashes
  - MD5 hashes of downloaded files
  - URLs from HTTP traffic
  - Autonomous System Numbers (ASNs)
- **Traffic Origin**: The module works with both real OT traffic (from mirrored OT networks) and simulated/captured traffic for testing.

2.4.4.1.2 Data Used for Training

This module does not require training. It operates entirely on rule-based matching using predefined threat intelligence feeds. The module:

- Uses static threat intelligence feeds containing known-bad indicators (IPs, domains, hashes)
- Relies on manually curated and community-maintained threat lists
- Operates without machine learning models or statistical analysis
- Employs deterministic matching algorithms: an indicator either matches or doesn't

2.4.4.1.3 Configuration Requirements

Configuration is managed through several files and parameters:

- **Primary Configuration File**: `config/smartshield.yaml`
  - `ti_files`: Path to TI feeds CSV file (default: `config/TI_feeds.csv`)

  - `download_path_for_local_threat_intelligence`: Local storage for downloaded feeds
  - `TI_files_update_period`: How often to update feeds (default: 24 hours)
- **TI Feeds File**: `config/TI_feeds.csv`
  - Contains URLs of remote threat feeds with format: `url,threat_level,tag`
  - Example: `https://lists.blocklist.de/lists/bruteforcelogin.txt,medium,['honeypot']`
- **Local TI Files**:
  - `config/local_data_files/own_malicious_iocs.csv`: Custom IPs, domains, ranges, ASNs
  - `config/local_data_files/own_malicious_JA3.csv`: Custom JA3 fingerprints
  - `config/local_data_files/known_fp_md5_hashes.csv`: Known false-positive file hashes
- **External Service Credentials**:
  - `config/vt_api_key`: VirusTotal API key (optional)
  - `config/RiskIQ_credentials`: RiskIQ email and API key (optional)
- **Sensitivity Parameters**:
  - Increasing update frequency improves detection of new threats but increases network/processing load
  - Adding more feeds increases coverage but may increase false positives
  - Lower threat level thresholds make the system more sensitive

2.4.4.1.4 Detection Logic

The module operates on a simple but comprehensive matching logic:

1. **Normal Behavior**: Any connection that doesn't match known malicious indicators
2. **Detection Trigger**: Exact or partial match with threat intelligence databases
3. **Matching Hierarchy**:
   - First check exact matches in local databases
   - If no exact match, check IP ranges and domain substrings
   - For files, check against known false positives first to reduce false alerts
   - Finally query external services (if configured and rate limits allow)

2.4.4.1.5 Detection Capabilities

The module detects connections involving:

- **Known Malicious IP Addresses**: Direct connections to/from blacklisted IPs
- **IP Range Associations**: Connections to IPs within known malicious ranges
- **Blacklisted Domains**: DNS queries/responses for malicious domains

- **Malicious Subdomains**: Subdomains of known bad domains (with reduced confidence)
- **Suspicious TLS Fingerprints**: JA3/JA3S hashes associated with malware
- **Compromised SSL Certificates**: SHA1 hashes of malicious certificates
- **Malicious File Downloads**: MD5 hashes of known malware files
- **Suspicious URLs**: HTTP connections to known malicious URLs
- **ASN-based Threats**: Connections to IPs belonging to malicious ASNs
- **DNS Chain Threats**: CNAME records pointing to malicious domains

2.4.4.1.6 Evidence Handling

The module generates detailed forensic evidence for each detection:

- **Evidence Storage**:
  - Stored in Redis database with complete context
  - Includes timestamps, source/destination IPs, UID references
  - Contains threat level (info, low, medium, high, critical) and confidence score
- **Forensic Value**:
  - **Timestamps**: Precise incident timing for timeline reconstruction
  - **Source/Destination Information**: Complete connection context
  - **Threat Intelligence Metadata**: Feed source, description, tags for investigation
  - **Correlation IDs**: Evidence relationships for multi-step attack analysis
- **Investigation Use**:
  - Timeline reconstruction of malicious activities
  - Attribution to known threat actors (via feed descriptions)
  - Identification of compromised assets
  - Basis for blocking decisions and incident response

2.4.4.1.7 Module Interaction (Integration with SmartShield)

The Threat Intelligence Module is tightly integrated with other SmartShield components:

- **Data Sharing**: Provides malicious IP/domain information to other modules via Redis
- **Profile Enrichment**: Marks profiles as “malicious” in their metadata when threats are detected
- **Risk Score Contribution**: Directly increases host risk scores when threats are identified
- **Trigger for Behavioral Modules**: Findings can trigger deeper behavioral analysis

- **Blocking Module Integration**: Provides immediate blocking candidates for the Blocking Module
- **Evidence Correlation**: Evidence IDs are shared between related alerts for correlation
- **Dependencies**:
  1. **Update Manager**: Receives updated threat feeds from this module
  2. **IP Info Module**: Provides ASN information for ASN-based detection
  3. **Flow Processing Core**: Receives flow data for indicator extraction
- **System Impact**: When this module identifies a threat, it:
  1. Increases the associated profile's threat level
  2. Creates correlated evidence for the alerting system
  3. Provides data for the Blocking Module to take action
  4. Contributes to the overall risk assessment of the network

The module's design allows it to work both as a standalone detector and as an enrichment source for other detection mechanisms, making it a foundational component of the SmartShield defense-in-depth strategy.

strategy.

2.4.4.2 PortScan Module

The PortScan Module is responsible for detecting various types of network reconnaissance activities, which are critical early indicators of potential attacks. This module specifically targets scanning behaviors where attackers probe networks to discover active hosts, open ports, and services. In the SmartShield system, port scan detection serves as a primary alert mechanism for identifying reconnaissance activities that typically precede more serious attacks, making it essential for early threat detection in both IT and OT environments.

2.4.4.2.1 Input Data

The PortScan module processes data from SmartShield's profiling system:

- **Primary Input Type**: Processed network flow data stored in time windows from Zeek/Bro logs
- **Data Sources**:
  - Flow metadata from Zeek `conn.log` (TCP/UDP connections)
  - Notice logs from Zeek for ICMP sweep detection
  - DHCP logs for DHCP scan detection
- **Features Used**:

- Source and destination IP addresses
- Source and destination ports
- Protocol information (TCP, UDP, ICMP, DHCP)
- Connection states (Established, Not Established)
- Packet counts and timestamps
- Flow UIDs for evidence correlation

- **Traffic Origin**: The module analyzes both real OT traffic (typically more predictable) and general network traffic, identifying abnormal scanning patterns in either context.

2.4.4.2.2 Data Used for Training

This module does not require training. It operates on threshold-based detection with configurable parameters. The module:

- Uses statistical analysis of connection patterns
- Relies on predefined thresholds for what constitutes scanning behavior
- Implements deterministic algorithms based on connection counts and distributions
- Does not use machine learning or require labeled datasets

2.4.4.2.3 Configuration Requirements

Configuration is managed through initialization parameters in the module:

- **Scan Thresholds**:
  - `port_scan_minimum_dports`: Minimum destination ports for vertical scan (default: 5)
  - `pingscan_minimum_flows`: Minimum ICMP flows for ping scan detection (default: 5)
  - `pingscan_minimum_scanned_ips`: Minimum scanned IPs for ICMP sweep (default: 5)
  - `minimum_requested_addrs`: Minimum DHCP requests for DHCP scan (default: 4)
- **Timing Parameters**:
  - `time_to_wait_before_generating_new_alert`: Delay between alerts for same scan (default: 25 seconds)
- **Detection Sensitivity**:
  - Increasing threshold values reduces false positives but may miss slower/scaled-down scans
  - Decreasing thresholds increases sensitivity but may generate more false alerts
  - The module implements progressive thresholding - subsequent detections require incrementally more activity
- **Protocol Specifics**:
  - TCP/UDP scans detected via “Not Established” connections only
  - ICMP scans use Zeek’s built-in detection with threshold of 25 flows

- DHCP scans monitor DHCPREQUEST messages for multiple address requests

2.4.4.2.4 Detection Logic

The module implements several distinct scanning detection algorithms:

Normal Behavior:

- Clients connecting to typical services with established connections
- Normal DHCP address requests (single or repeated same address)
- Occasional ICMP traffic for network diagnostics

Detection Triggers:

```python
# Simplified detection logic from code analysis
def detect_scans(profileid, twid, flows):
    # Horizontal Port Scan: Same port, multiple IPs
    for port in scanned_ports:
        dst_ips = get_destination_ips_for_port(port)
        if len(dst_ips) ≥ threshold and
        increases_since_last_alert(dst_ips):
            trigger_evidence("HorizontalPortScan")
    # Vertical Port Scan: Same IP, multiple ports
    for dst_ip in destination_ips:
        dst_ports = get_ports_for_ip(dst_ip)
        if len(dst_ports) ≥ threshold and
        increases_since_last_alert(dst_ports):
            trigger_evidence("VerticalPortScan")
    # ICMP Scan: Multiple ICMP flows to different hosts
    icmp_flows = get_icmp_flows()
    if is_icmp_sweep(icmp_flows):
        trigger_evidence("ICMPScan")
    # DHCP Scan: Multiple different address requests
    dhcp_requests = get_dhcp_requests()
    if len(unique_addresses(dhcp_requests)) ≥ threshold:
        trigger_evidence("DHCPScan")
```

Key Detection Principles:

1. **State-based filtering**: Only "Not Established" connections considered for TCP/UDP scans
2. **Progressive thresholds**: Each new alert requires significantly more activity than previous
3. **Time window isolation**: Scans are evaluated within single time windows (default 1 hour)
4. **Protocol-specific logic**: Different algorithms for TCP/UDP vs ICMP vs DHCP

2.4.4.2.5 Detection Capabilities

The module detects four distinct types of scanning activities:

- **Horizontal Port Scans**: Scanning the same port across multiple destination IPs
  - **Example**: Scanning port 22 (SSH) across all IPs in 192.168.1.0/24 subnet
  - **OT Relevance**: Scanning Modbus (502) or other OT protocols across multiple PLCs
- **Vertical Port Scans**: Scanning multiple ports on a single destination IP
  - **Example**: Scanning ports 1-1000 on a single server
  - **OT Relevance**: Scanning all possible ports on a critical HMI or SCADA server
- **ICMP Scans**: Network discovery using ICMP packets
  - **Types detected**: Address scans, timestamp scans, address mask scans
  - **Example**: ICMP echo requests to discover live hosts in a network
  - **OT Relevance**: Mapping OT network topology before targeted attacks
- **DHCP Scans**: Discovery through DHCP address requests
  - **Example**: Client requesting multiple different IP addresses to map address space
  - **OT Relevance**: Less common in OT but could indicate rogue device discovery

2.4.4.2.6 Evidence Handling

The module generates detailed forensic evidence for each detection:

- **Evidence Types Created**:
  - `HORIZONTAL_PORT_SCAN`: Evidence for horizontal scanning patterns
  - `VERTICAL_PORT_SCAN`: Evidence for vertical scanning patterns
  - `ICMP_TIMESTAMP_SCAN`, `ICMP_ADDRESS_SCAN`, `ICMP_ADDRESS_MASK_SCAN`: ICMP scan variants
  - `DHCP_SCAN`: Evidence for DHCP discovery scans
- **Evidence Storage**:
  - Stored in Redis with complete flow context
  - Includes all relevant UIDs for flow correlation

  - Contains scan statistics: packet counts, scanned IPs/ports, confidence scores
- **Forensic Value**:
  - **Complete UID lists**: All flows contributing to the scan detection
  - **Timestamps**: Precise timing of scanning activity
  - **Scan metrics**: Number of targets, packets sent, ports scanned
  - **Confidence scoring**: Based on scan intensity and protocol
- **Investigation Use**:
  - Identify reconnaissance phases in attack kill chains
  - Correlate scanning with later exploitation attempts
  - Map attacker methodology and tools
  - Support incident response with detailed scan evidence

2.4.4.2.7 Module Interaction (Integration with SmartShield)

The PortScan Module has strategic integration with other SmartShield components:

- **Core Dependency**: Relies on SmartShield's profiling system for time-windowed flow data
- **Data Flow**:
  - Receives `tw_modified` events when time windows change
  - Processes `new_notice` events for Zeek-based ICMP detection
  - Handles `new_dhcp` events for DHCP scan detection
- **Evidence Contribution**:
  - Increases risk scores significantly (HIGH threat level)
  - Provides early warning to other behavioral modules
  - Feeds blocking module with clear malicious IPs
- **System Synergies**:
  - **Threat Intelligence Module**: Scanning IPs checked against known malicious entities
  - **Behavioral Modules**: Scan patterns can trigger deeper behavioral analysis
  - **Blocking Module**: Immediate blocking candidates from clear scan attacks
  - **Timeline Module**: Scan events provide critical timeline markers
- **OT Environment Special Considerations**:
  - **Protocol-specific detection**: OT protocols have different normal connection patterns
  - **Threshold adjustments**: OT networks may require different scanning thresholds
  - **False positive management**: Industrial devices may have legitimate scanning behaviors

The module's design as both a standalone detector and integrated component makes it particularly valuable for OT security, where reconnaissance often precedes targeted attacks on critical infrastructure. Its ability to detect both broad network scans and targeted protocol scans provides comprehensive coverage for the early stages of attack campaigns. campaigns.

2.4.4.3 RNN C&C Detection Module

2.4.4.3.1 Module Purpose

This module is responsible for detecting command and control (C&C) channels in network traffic by analyzing behavioral patterns represented as Behavioral Letters. It targets advanced persistent threats (APTs) and malware that establish covert communication channels with command servers using sophisticated evasion techniques. In the SmartShield system, this module is critical for identifying stealthy C&C communications that bypass traditional signature-based detection methods by focusing on behavioral characteristics rather than content analysis.

2.4.4.3.2 Input Data

- **Type of input**:
  - Network flows (real-time)
  - Behavioral Letters generated from TCP flows
- **Features used**:
  - Source IP addresses
  - Destination IP addresses
  - Destination ports
  - Transport protocols (TCP only)
  - Flow timing characteristics
  - Behavioral patterns encoded as letters
- **Data characteristics**:
  - Real operational traffic analyzed in real-time
  - Behavioral models built from established TCP connections

2.4.4.3.3 Data Used for Training

This module is Machine-learning based with the following training characteristics:

- **Dataset sources**:
  - Verified malware C&C connections from executed malware samples
  - Normal connections (benign traffic) to reduce false positives
  - Mix of both malicious and legitimate traffic patterns

- **Type of traffic**:
  - **Normal behavior**: Legitimate TCP communications
  - **Malicious behavior**: Verified C&C channels from malware families
- **Training approach**:
  - Supervised learning with labeled data
  - Offline training using historical datasets
  - Model versioning with multiple iterations (v1.0 to v1.5)
- **Training datasets**:
  - **File**: `modules/rnn_cc_detection/datasets/all_datasets_raw.tsv`
  - **Format**: TSV with columns: `[ID] | Label | 4-tuple | letters`
  - **Size**: 7,631 outtuples (4-tuples) in current dataset

```
Example sample of 3 rows
[492] | From-Normal-TCP-HTTPS-Facebook-9                                  |
[36m147.32.84.134-66.220.151.91-443-tcp[0m       |
66.w.W.W,i,Y.F.W+W.Z+Z.z.W.W.h.Z.Z.Z.W.Z.Z+Z.Z.I.Z.Z.W.w.W,Z+Z.Z.Z.Z.z.Y
↳ ,Y.W,W.F.W.z.W+W.Z.Z+Z.z.z.Z.W,Z*Z.I.z.W.W.W.F.Y.H.W+z*Z.i.I.z.F.W.w.Z,
[493] | From-Normal-TCP-HTTP-Facebook-24                                  |
[36m147.32.84.134-66.220.153.23-80-tcp[0m        | 66.
[494] | From-Normal-TCP-HTTP-Facebook-25                                  |
[36m147.32.84.134-66.220.158.18-80-tcp[0m        | 9
```

- **Model Architecture**:

```
                          Model Architecture (v1.2)
Input: (500 timesteps, 1 feature)
↓
Embedding (50 → 64)
↓
Bidirectional GRU (32 units)
↓
Dense (32, ReLU)
↓
Dropout
↓
Dense (1, Sigmoid)
```

```
v1.5: Embedding(94) → Reshape → Bidirectional GRU(16) → Dense(24) →
Dropout → Output(1)
```

- **Training Performance**: The RNN model training shows effective learning with minimal overfitting, as demonstrated by the following metrics:

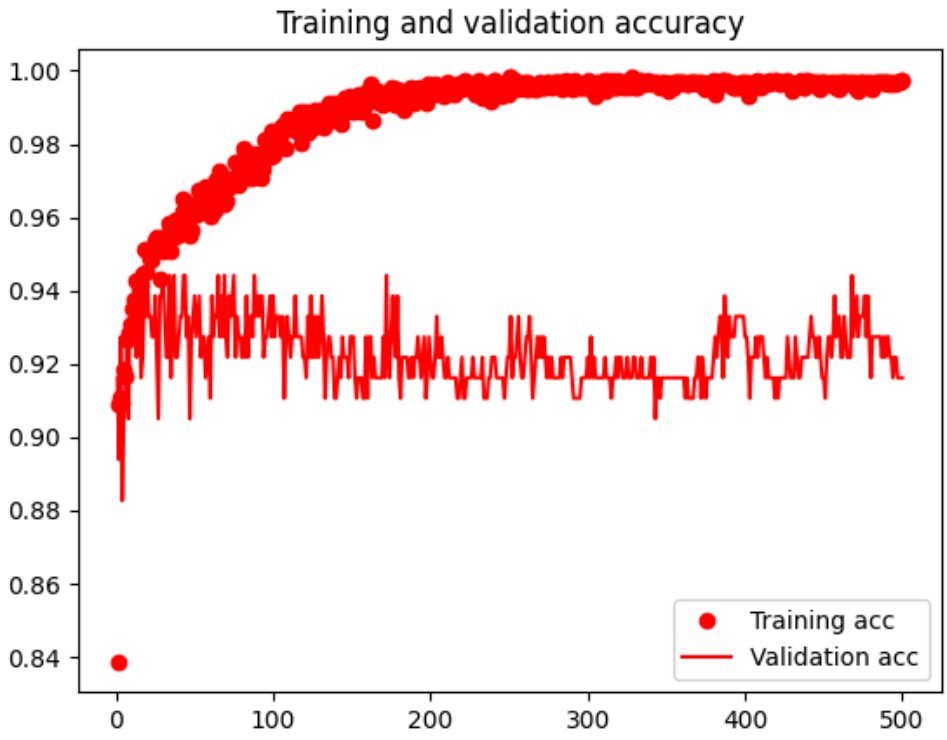

Figure 2.5: Training and validation accuracy across 500 epochs showing consistent learning with minimal divergence between training and validation performance

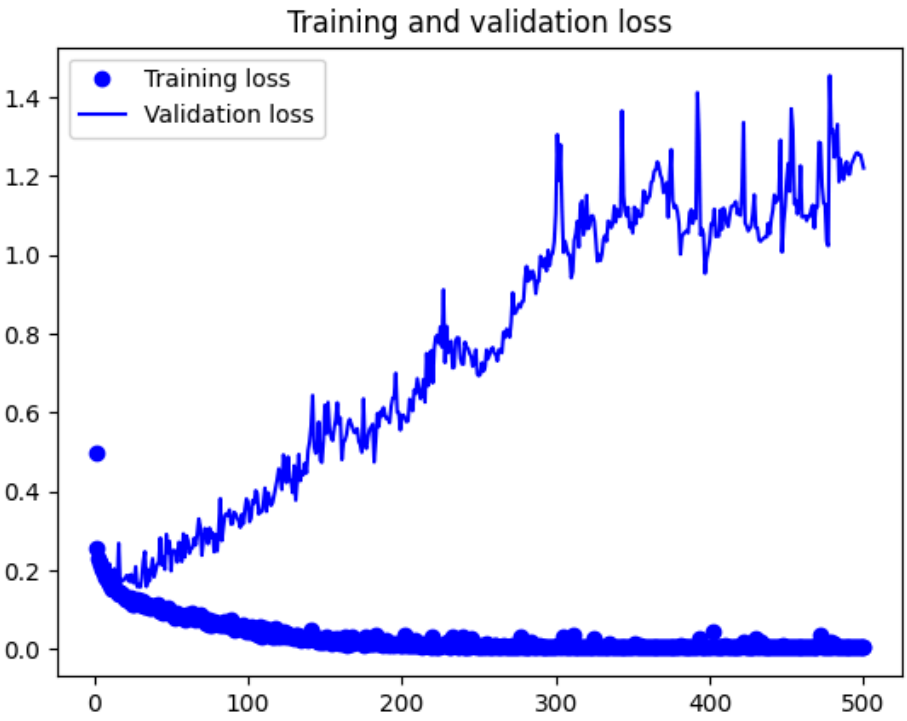

Figure 2.6: Training and validation loss across 500 epochs demonstrating stable convergence with minimal overfitting

- **Training Results**:
  - v1.2:

```python
# Evaluate model on the new dataset
loss, accuracy = model.evaluate(X_test, y_test)

# Print evaluation results
print("Test Generatilization Results:")
print("Test Loss:", loss)
print("Test Accuracy:", accuracy)
```

```
14/14 [==============================] - 1s 66ms/step - loss: 1.2258 - accuracy: 0.9305
Test Generatilization Results:
Test Loss: 1.2258477210998535
Test Accuracy: 0.9304932951927185
```

  - v1.5: Test Accuracy: 93.0%, Test Loss: 0.476 (Best production model)

2.4.4.3.4 Configuration Requirements

- **Configuration files**: Uses default model `rnn_model.h5`
- **Thresholds**:
  - **Detection threshold**: 0.99 (minimum similarity score to trigger alert)
  - **Confidence threshold**: 100 letters (minimum behavioral model length for full confidence)
- **Enable/disable**: Module can be enabled/disabled in SmartShield configuration

- **Parameters affecting sensitivity**:
  - Increasing threshold reduces false positives but may miss subtle C&C
  - Decreasing threshold increases sensitivity but raises false positives
  - Confidence scaling based on behavioral model length (20 letters = 0.2 confidence, 100+ letters = 1.0 confidence)

2.4.4.3.5 Detection Logic (How it works)

- **Normal behavior**: Legitimate TCP flows with varied timing, size, and duration patterns that don't match known C&C behavioral signatures
- **Detection trigger**: When a behavioral model (string of Letters) receives a prediction score > 0.99 from the RNN model
- **Detection method**:
  1. Collect Letters for each 4-tuple (srcIP, dstIP, dstPort, proto)
  2. Convert letters to numerical encoding using vocabulary mapping
  3. Pad sequence to fixed length (500 characters)
  4. Feed to pre-trained RNN model for prediction
  5. If score > threshold, generate C&C alert with confidence based on sequence length

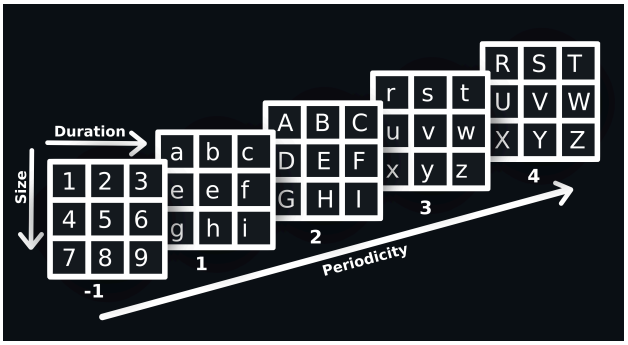

Figure 2.8: Numerical encoding

- **Logic flow**:

```
New TCP flow → Generate Letters → Build behavioral model →
Convert to numerical format → RNN prediction →
Score > 0.99? → Yes: Generate C&C evidence
```

2.4.4.3.6 Detection Capabilities

- **Types of attacks detected**:
  - Command and control channels for malware/APT communication
  - Covert TCP channels using periodic beaconing
  - Evasive C&C using varying timing patterns

- Stealthy communication channels disguised as normal traffic
- **Specific detection examples**:
  - Periodic beaconing to C&C servers
  - Size/duration patterns consistent with C&C data exfiltration
  - Behavioral patterns matching known malware families in training data

2.4.4.3.7 Evidence Handling

- **Evidence generated**:
  - C&C channel detection alerts with confidence scores
  - Behavioral model strings that triggered detection
  - RNN prediction scores (0.0 to 1.0)
  - Flow metadata (IPs, ports, timestamps)
- **Storage format**:
  - JSON evidence objects stored in SmartShield database
  - Linked evidence IDs for source and destination IPs
  - Timestamped with flow start time
- **Forensic value**:
  - Complete behavioral model available for analysis
  - Confidence score indicates detection certainty
  - Can be correlated with other module findings
  - Useful for incident response and threat hunting
- **Timestamping**: Uses flow start time converted to standardized format
- **Source/Destination**: Both endpoints included in evidence with directional context

2.4.4.3.8 Module Interaction (Integration with SmartShield)

- **Independent operation**: Can detect C&C channels without input from other modules
- **Results sharing**:
  - Triggers JARM hash checking when C&C is detected
  - Shares evidence with correlation engine
  - Both source and destination IPs receive alerts
- **Risk scoring**: Contributes to host risk scoring in SmartShield system
- **Correlation capabilities**:
  - Evidence can be correlated with port scanning detection
  - Can be combined with DGA detection for comprehensive C&C identification
  - Works alongside behavioral analysis modules for layered defense

- **System integration**:
  - Subscribes to `new_letters` channel for behavioral updates
  - Listens to `tw_closed` events for time window management
  - Publishes to alert channels for system-wide notification
  - Integrates with evidence database for persistent storage
- **Integration Flow**:

```
Letters Export → RNN C&C Detection → Evidence Generation →
Correlation Engine → Alert Dashboard → JARM Hash Check (optional)
```

This module represents a sophisticated AI-driven approach to C&C detection that complements traditional rule-based methods, providing defense against evolving, behavior-based threat actor techniques.

2.4.4.4 Flow Alerts Module

2.4.4.4.1 Module Purpose

The Flow Alerts module provides comprehensive behavioral analysis of individual network flows to detect a wide range of suspicious activities and attacks. It operates by analyzing each flow in isolation using multiple detection techniques, complementing the correlation-based approaches of other modules. This module is essential for identifying immediate threats that manifest within single flows or connections, providing real-time detection without requiring complex pattern correlation.

2.4.4.4.2 Input Data

- **Type of input**:
  - Network flows (real-time or from files)
  - Zeek logs (`conn.log`, `dns.log`, `ssl.log`, `ssh.log`, etc.)
  - Protocol-specific logs (HTTP, SMTP, SSL, DNS, SSH)
- **Features used**:
  - Flow metadata (timestamps, duration, byte counts)
  - Protocol-specific attributes (ports, services, certificates)
  - Behavioral patterns (connection attempts, DNS behavior)
  - Security indicators (JA3 hashes, SSL certificates, user agents)
- **Data characteristics**:
  - Real-time flow analysis with immediate alerting
  - Protocol-aware inspection across multiple layers
  - Integration with Zeek for deep packet inspection

2.4.4.4.3 Detection Techniques (How it works)

The module employs 25+ detection techniques across multiple protocol layers:

Connection Behavior Analysis:

- **Long Connections**: Detects connections exceeding configurable duration (default: 1500 seconds/25 minutes)
- **Multiple Reconnection Attempts**: 5+ failed connection attempts to same IP/port
- **Connection to Unknown Ports**: Ports not in known services list or organization-specific ports
- **Connection to Port 0**: Malicious connections to port 0 (excluding IGMP/ICMP)
- **Connection to Multiple Ports**: Scanning behavior detection

DNS Analysis:

- **Connections without DNS Resolution**: Suspicious direct IP connections (with whitelist exceptions)
- **DNS Resolutions without Connection**: Unused DNS lookups (with domain exceptions)
- **DGA Detection**: 10+ NXDOMAIN responses from source IP
- **DNS ARPA Scans**: 10+ ARPA queries within 2 seconds
- **Invalid DNS Answers**: Detection of blocking responses (0.0.0.0, 127.0.0.1)
- **High Entropy DNS TXT**: Encrypted/encoded strings in TXT records (entropy ≥ 5)

Protocol-Specific Detection:

- **Successful SSH Connections**: Detected via Zeek logs or byte count (>4290 bytes)
- **SSH Version Changing**: Version inconsistency detection
- **SMTP Login Bruteforce**: 3+ failed logins within 10 seconds
- **Non-SSL on Port 443**: Established connections on 443 without SSL
- **Weird HTTP Methods**: Detection of unusual HTTP methods
- **Pastebin Downloads**: Files >700 bytes from pastebin

Security Validation:

- **Malicious JA3/JA3s Hashes**: C&C and client fingerprint detection
- **Malicious SSL Certificates**: SHA1 hash-based detection
- **Incompatible CN**: Certificate CN mismatch with organization
- **CN URL Mismatch**: Server name mismatch with certificate CN

- **Young Domains**: Domains < 60 days old (supported TLDs)

Network Anomalies:

- **Data Exfiltration**: Uploads exceeding 100MB (configurable)
- **Connection to Private IPs**: Internal network communications
- **Private IPs Outside Local Network**: Cross-subnet private communications
- **Devices Changing IPs**: MAC address IP reassignment
- **GRE Tunnels & Scans**: Tunnel detection and scanning behavior

Zeek Integration Alerts:

- Self-signed certificates
- Invalid certificates
- Port scans and address scans
- Password guessing
- Bad SMTP logins

2.4.4.4.4 Configuration Requirements

- **Configuration files**: Main configuration via `smartshield.yaml`
- **Key thresholds**:
  - `long_connection_threshold`: 1500 seconds (default)
  - `data_exfiltration_threshold`: 100 MB
  - `pastebin_download_threshold`: 700 bytes
  - `entropy_threshold`: 5.0
  - `DGA threshold`: 10 NXDOMAINs
- **Whitelists & Exceptions**:
  - Private/local/reserved IP ranges
  - Well-known organizations (Google, Apple, Microsoft, etc.)
  - Tranco whitelist domains
  - Specific domain exceptions (.local, .arpa, etc.)
- **Time-based filters**:
  - 3-minute grace period for DNS resolution detection on interfaces
  - 30-minute wait for connection verification

2.4.4.4.5 Detection Logic

- **Normal behavior**: Legitimate flows matching expected patterns for each protocol
- **Detection triggers**: Threshold violations, pattern mismatches, security indicator matches
- **Analysis flow**:

```
New Flow/Log → Protocol Identification → Specific Analyzer
→
Pattern Matching → Threshold Checking → Whitelist Verification →
Alert Generation → Evidence Storage
```

- **Multi-protocol support**: Separate analyzers for DNS, SSH, SSL, SMTP, HTTP, etc.
- **Asynchronous processing**: Coroutine-based analysis for performance

2.4.4.4.6 Detection Capabilities

- **Types of attacks detected**:
  - Network scanning and reconnaissance
  - Data exfiltration attempts
  - C&C communication (via JA3/SSL)
  - Credential attacks (SSH/SMTP bruteforce)
  - DNS-based attacks (DGA, tunneling)
  - Protocol anomalies and evasion
  - Certificate and encryption anomalies
  - Internal network policy violations
- **Specific examples**:
  - Port scanning via multiple connection attempts
  - Data theft via large uploads
  - Malware communication via malicious JA3 hashes
  - Phishing via young domains
  - Internal reconnaissance via ARPA scans

2.4.4.4.7 Evidence Handling

- **Evidence types**: Protocol-specific alerts with detailed context
- **Information included**:
  - Flow metadata (IPs, ports, timestamps)
  - Protocol-specific details (certificates, hashes, domains)
  - Behavioral metrics (duration, byte counts, attempt counts)
  - Threat intelligence matches
- **Confidence scoring**: Based on threshold exceedance and pattern strength
- **Correlation support**: Evidence can be correlated with other module findings
- **Forensic value**: Complete flow context with all relevant protocol details

2.4.4.4.8 Module Architecture

- **Core Components**:
  - `Conn`: Connection flow analysis
  - `DNS`: DNS behavior analysis
  - `SSL`: SSL/TLS certificate and encryption analysis
  - `SSH`: SSH connection and authentication analysis
  - `SMTP`: Email protocol analysis
  - `Software`: Software/version detection
  - `Notice`: Zeek notice integration
  - `DownloadedFile`: File download analysis
  - `Tunnel`: Network tunneling detection
- **Channel Subscriptions**:
  - `new_flow`: New connection events
  - `new_dns`: DNS query events
  - `new_ssh`: SSH connection events
  - `new_ssl`: SSL/TLS events
  - `new_smtp`: SMTP events
  - `new_software`: Software detection events
  - `new_notice`: Zeek notice events
  - `new_downloaded_file`: File download events
  - `new_tunnel`: Tunnel detection events
  - `tw_closed`: Time window closure events
- **Processing Model**:
  - Asynchronous, event-driven architecture
  - Protocol-specific analyzers run in parallel
  - Graceful shutdown with task completion
  - Resource-efficient processing

2.4.4.4.9 Integration & Performance

- **System integration**:
  - Receives events from Zeek parsers
  - Publishes alerts to correlation engine
  - Integrates with whitelisting system
  - Shares evidence with risk scoring
- **Performance considerations**:
  - Asynchronous processing prevents blocking

- Protocol-specific analyzers optimize resource usage
- Time-based filters reduce false positives
- Whitelist checking minimizes unnecessary processing

- **Scalability**: Designed for high-throughput environments with configurable thresholds

This module serves as the first line of behavioral defense in the SmartShield system, providing immediate detection of suspicious activities while feeding higher-level correlation engines with valuable behavioral evidence.

2.4.4.5 ARP Module

2.4.4.5.1 Module Purpose

This module is responsible for detecting ARP-based attacks and anomalies in network traffic. ARP (Address Resolution Protocol) attacks are critical to identify as they can enable man-in-the-middle (MITM) attacks, network reconnaissance, and traffic redirection. By analyzing ARP traffic patterns, this module provides essential protection against layer 2 network attacks that can bypass higher-layer security measures. It complements other detection modules by focusing on the data link layer, which is often overlooked in traditional network security monitoring.

2.4.4.5.2 Input Data

- **Type of input**:
  - ARP flows from Zeek `arp.log`
  - Real-time ARP traffic from network interfaces
  - ARP requests and replies
- **Features used**:
  - Source and destination MAC addresses
  - Source and destination IP addresses
  - ARP operation type (request/reply)
  - Timestamps and flow metadata
  - Network interface information
- **Data characteristics**:
  - Requires custom Zeek scripts for ARP logging (included with SmartShield)
  - Layer 2 protocol analysis
  - Real-time detection capability
  - Interface-specific context awareness

2.4.4.5.3 Detection Techniques (How it works)

The module employs four primary detection techniques for comprehensive ARP attack identification:

1. **ARP Scans**:
   - **Detection**: 5+ non-gratuitous ARP requests to different IPs within 30 seconds
   - **Logic**: Groups ARP requests by source IP, tracks destination IP diversity and timing
   - **Exclusions**: Gratuitous ARP, gateway traffic, and broadcast addresses
   - **Confidence**: 0.8 (High)

2. **ARP to Destination IP Outside Local Network**:
   - **Detection**: ARP requests sent to IPs outside configured local network ranges
   - **Logic**: Validates destination IP against home network definitions
   - **Exclusions**: Multicast, link-local, and self-addressed ARP packets
   - **Confidence**: 0.6 (Medium)

3. **Unsolicited ARP**:
   - **Detection**: Broadcast ARP replies without preceding requests
   - **Logic**: Identifies ARP replies sent to broadcast MAC (`ff:ff:ff:ff:ff:ff`)
   - **Threat Level**: LOW (informational)
   - **Purpose**: Detection of ARP cache poisoning attempts or legitimate updates

4. **MITM ARP Attack**:
   - **Detection**: MAC address claiming ownership of multiple IP addresses
   - **Logic**: Tracks MAC-to-IP mappings, detects when MAC claims new IP ownership
   - **Context**: Identifies potential ARP spoofing for traffic interception
   - **Confidence**: 0.2 (Low but critical)

2.4.4.5.4 Configuration Requirements

- **Configuration files**:
  - Main configuration via `smartshield.yaml`
  - Home network definitions for local network detection
  - P2P settings for peer identification
- **Detection thresholds**:
  - ARP scan threshold: 5 different destination IPs
  - ARP scan time window: 30 seconds
  - Evidence accumulation window: 10 seconds

- **Network definitions**:
  - Home network ranges for local/remote distinction
  - Gateway identification for traffic filtering
  - Private IP ranges for context awareness
- **System integration**:
  - Requires ARP logging enabled in Zeek
  - Supports blocking mode integration
  - P2P trust model for peer identification

2.4.4.5.5 Detection Logic

- **Normal behavior**: Legitimate ARP requests/responses within local network
- **Detection flow**:

```text
ARP Flow → Operation Analysis → Context Validation →
Pattern Matching → Threshold Checking → Evidence Generation
```

- **Gratuitous ARP handling**: Special processing for broadcast ARP replies
- **Time window management**: Evidence accumulation over 10-second windows
- **Interface awareness**: Gateway and local network context consideration
- **Detailed Processing**:
  - **Operation Classification**: Request vs. reply analysis
  - **Address Validation**: MAC/IP consistency checks
  - **Network Context**: Local vs. remote network determination
  - **Pattern Analysis**: Scan detection and timing analysis
  - **Evidence Filtering**: SmartShield peer and self-defense filtering

2.4.4.5.6 Detection Capabilities

- **Types of attacks detected**:
  - ARP scanning: Network reconnaissance and host discovery
  - ARP spoofing: MAC address impersonation attacks
  - MITM attacks: Traffic interception through ARP manipulation
  - Network policy violations: ARP to external networks
  - Unsolicited ARP: ARP cache poisoning attempts
- **Specific scenarios**:
  - Internal network scanning via ARP requests
  - MAC address conflicts indicating spoofing
  - Broadcast ARP replies for cache manipulation

- Cross-subnet ARP traffic (potential misconfiguration)

2.4.4.5.7 Evidence Handling

- **Evidence types**:
  - `ARP_SCAN`: ARP scanning behavior
  - `ARP_OUTSIDE_LOCALNET`: External ARP traffic
  - `UNSOLICITED_ARP`: Unsolicited ARP replies
  - `MITM_ARP_ATTACK`: Man-in-the-middle ARP attacks
- **Information included**:
  - Source and destination IP/MAC addresses
  - ARP operation details
  - Timing and frequency data
  - Network context (local/remote, gateway)
  - Confidence scores based on detection logic
- **Filtering mechanisms**:
  - SmartShield peer identification (P2P trust ≥ 0.3)
  - Self-defense traffic filtering
  - Gateway traffic exclusion

2.4.4.5.8 Module Architecture

- **Core Components**:
  - `Main ARP Analyzer`: Primary detection logic and flow processing
  - `Evidence Filter`: Filters SmartShield-related ARP activities
  - `Cache Management`: Tracks ARP requests and MAC/IP mappings
  - `Timer Thread`: Evidence accumulation and time-based analysis
- **Data Structures**:
  - `cache_arp_requests`: Groups ARP requests by profile/time window
  - `pending_arp_scan_evidence`: Queue for evidence accumulation
  - `home_network`: Configurable local network ranges
- **Thread Management**:
  - Timer thread: Waits 10 seconds for additional ARP scan evidence
  - Asynchronous processing: Evidence accumulation without blocking
  - Graceful shutdown: Thread-safe termination
- **Channel Subscriptions**:
  - `new_arp`: New ARP flow events
  - `tw_closed`: Time window closure events

2.4.4.5.9 Integration & Special Features

- **Zeek Integration**:
  - Custom Zeek scripts for ARP logging
  - Real-time ARP flow processing
  - Log file management (optional deletion)
- **SmartShield Ecosystem**:
  - P2P trust model integration
  - Self-defense traffic recognition
  - Blocking mode compatibility
- **Network Awareness**:
  - Gateway identification and filtering
  - Local network context awareness
  - Interface-specific analysis
- **Performance Features**:
  - Evidence accumulation to reduce alert volume
  - Time-based cache cleanup
  - Resource-efficient thread management

2.4.4.5.10 Strengths

- **Best Practices**:
  - Enable Zeek ARP logging for comprehensive coverage
  - Configure accurate home network ranges
  - Enable P2P for SmartShield peer filtering
  - Monitor gateway ARP traffic for baseline
  - Review unsolicited ARP alerts for policy violations

This module provides essential layer 2 security monitoring, detecting ARP-based attacks that can compromise network integrity and enable higher-level attacks. By focusing on the data link layer, it complements other SmartShield modules to provide comprehensive network security coverage from physical layer to application layer.

2.4.4.6 HTTP Analyzer Module

2.4.4.6.1 Module Purpose

The HTTP Analyzer module provides comprehensive HTTP traffic analysis to detect suspicious web activities and protocol anomalies. HTTP remains one of the most commonly exploited protocols for malware distribution, C&C communication, and data exfiltration. This module focuses on application-layer detection by analyzing HTTP

headers, user agents, content types, and connection patterns to identify malicious web traffic that may bypass lower-layer security controls. It complements other detection modules by providing protocol-specific intelligence for web-based attacks.

2.4.4.6.2 Input Data

- **Type of input**:
  - HTTP flows from Zeek `http.log`
  - Weird logs from Zeek `weird.log`
  - Connection flows from Zeek `conn.log`
  - Real-time HTTP traffic monitoring
- **Features used**:
  - HTTP headers (User-Agent, Host, URI, Method)
  - Content types and MIME types
  - Request/response body lengths
  - Flow metadata (timestamps, IPs, ports)
  - Protocol-specific attributes
- **Data characteristics**:
  - Application-layer protocol analysis
  - Real-time HTTP traffic inspection
  - Integration with Zeek for deep HTTP parsing
  - Multi-source data correlation

2.4.4.6.3 Detection Techniques (How it works)

The module employs eight primary detection techniques for comprehensive HTTP traffic analysis:

1. **Multiple Empty Connections**:
   - **Detection**: 4+ empty HTTP connections to popular domains (Google, Bing, Yahoo, etc.)
   - **Logic**: Tracks URI="/" requests to known domains with zero request body length
   - **Purpose**: Identifies malware internet connectivity checks
   - **Confidence**: 1.0 (High)
   - **Exclusions**: Whitelisted domains and non-empty connections

2. **Suspicious User Agents**:
   - **Detection**: Known malicious/suspicious user agent strings
   - **Suspicious UAs**: `httpsend`, `chm_msdn`, `pb`, `jndi`, `tesseract`
   - **Logic**: String matching against known malicious patterns

- **Threat Level**: HIGH

3. **Incompatible User Agents**:
   - **Detection**: User agent mismatch with device MAC vendor
   - **Logic**: Compares UA OS information with MAC OUI vendor
   - **OS Keyword Mapping**:
     - Apple: `macos`, `ios`, `apple`, `os x`, `mac`, `macintosh`, `darwin`
     - Microsoft: `microsoft`, `windows`, `nt`
     - Android/Google: `android`, `google`
   - **Threat Level**: HIGH
   - **Data Sources**: MAC vendor database + online UA analysis
4. **Multiple User Agents**:
   - **Detection**: Same IP using multiple different user agents
   - **Logic**: Tracks UA changes and OS inconsistencies per IP
   - **Example**: IP using macOS UA then Android UA
   - **Threat Level**: INFO (informational)
   - **Purpose**: Potential device/user impersonation
5. **Pastebin Downloads**:
   - **Detection**: File downloads from pastebin.com exceeding threshold
   - **Threshold**: ≥700 bytes (configurable via `pastebin_download_threshold`)
   - **Logic**: Host identification + GET method + response size check
   - **Threat Level**: INFO
   - **Purpose**: Malware payload delivery detection
6. **Unencrypted HTTP Traffic**:
   - **Detection**: Plaintext HTTP connections
   - **Logic**: All HTTP flows trigger informational alerts
   - **Threat Level**: INFO
   - **Purpose**: Security policy compliance monitoring
7. **Non-HTTP Connections on Port 80**:
   - **Detection**: Established TCP connections on port 80 without HTTP protocol
   - **Logic**: 5-minute sliding window analysis to avoid false positives
   - **Validation**: Checks for matching HTTP flows within ±5 minutes
   - **Threat Level**: LOW/MEDIUM (depending on direction)
   - **Purpose**: Protocol evasion detection

8. **Executable MIME Types**:
   - **Detection**: Download of executable files via HTTP
   - **Executable MIME Types**:

```text
application/x-msdownload
application/x-ms-dos-executable
application/x-ms-exe
application/x-exe
application/x-winexe
application/x-winhlp
application/x-winhelp
application/octet-stream
application/x-dosexec
```

   - **Threat Level**: LOW
   - **Purpose**: Malware download detection

2.4.4.6.4 Configuration Requirements

- **Configuration files**:
  - Main configuration via `smartshield.yaml`
  - Whitelist configuration for domain exceptions
- **Detection thresholds**:
  - Empty connections: 4 requests to same domain
  - Pastebin downloads: 700 bytes (default)
  - Non-HTTP detection: 5-minute time window
- **External services**:
  - User agent analysis: `http://useragentstring.com`
  - MAC vendor database for device identification
- **Integration settings**:
  - Zeek HTTP logging enabled
  - Whitelist support for legitimate domains
  - Time window management for flow correlation

2.4.4.6.5 Detection Logic

- **Normal behavior**: Standard HTTP traffic with consistent user agents
- **Detection flow**:

```
HTTP Flow → Header Analysis → Content Inspection →
Pattern Matching → Context Validation → Evidence Generation
```
text

- **Multi-stage analysis**:
  - **Header Inspection**: User agent, host, method analysis
  - **Content Analysis**: MIME types, body lengths, file types
  - **Context Validation**: MAC vendor, historical patterns
  - **Temporal Analysis**: Time-based correlation and sequencing
- **Advanced Processing**:
  - **User Agent Intelligence**: Online lookup for OS/browser details
  - **MAC-Vendor Mapping**: Device type validation via OUI database
  - **Time Window Correlation**: 5-minute sliding windows for protocol detection
  - **Evidence Accumulation**: Multiple detections combined into single alerts

2.4.4.6.6 Detection Capabilities

- **Types of attacks detected**:
  - Malware communication: Suspicious UAs and empty connectivity checks
  - Data exfiltration: Large downloads from pastebin
  - Malware delivery: Executable file downloads
  - Protocol evasion: Non-HTTP traffic on standard ports
  - Impersonation attacks: Inconsistent user agents
  - Reconnaissance: Multiple empty connections to search engines
- **Specific scenarios**:
  - Malware checking internet connectivity via empty requests
  - Suspicious tools using identifiable user agents
  - Device impersonation via UA spoofing
  - Malicious file downloads disguised as normal web traffic
  - Protocol tunneling attempts on port 80

2.4.4.6.7 Evidence Handling

- **Evidence types**:
  - `EMPTY_CONNECTIONS`: Multiple empty HTTP requests
  - `SUSPICIOUS_USER_AGENT`: Known malicious user agents
  - `INCOMPATIBLE_USER_AGENT`: UA/MAC vendor mismatch
  - `MULTIPLE_USER_AGENT`: Multiple different UAs from same IP
  - `PASTEBIN_DOWNLOAD`: File downloads from pastebin
  - `HTTP_TRAFFIC`: Unencrypted HTTP connections

  - `NON_HTTP_PORT_80_CONNECTION`: Non-HTTP on port 80
  - `EXECUTABLE_MIME_TYPE`: Executable file downloads
  - `WEIRD_HTTP_METHOD`: Unusual HTTP methods
- **Information included**:
  - Full HTTP header details
  - User agent strings and parsed information
  - MAC vendor and device context
  - File sizes and MIME types
  - Temporal patterns and sequencing
- **Correlation features**:
  - Bidirectional evidence for source and destination IPs
  - Time window contextualization
  - Historical pattern tracking

2.4.4.6.8 Module Architecture

- **Core Components**:
  - `Main HTTP Analyzer`: Primary detection logic and flow processing
  - `SetEvidence Helper`: Evidence generation and formatting
  - `Flow Classifier`: Protocol flow object conversion
  - `UA Intelligence`: Online user agent analysis
- **Data Structures**:
  - `connections_counter`: Tracks empty connections per domain
  - `http_recognized_flows`: Sliding window of legitimate HTTP flows
  - `executable_mime_types`: List of executable MIME patterns
  - `empty_connection_hosts`: Known malware check domains
- **Asynchronous Features**:
  - Time-based waiting: 5-minute delays for flow correlation
  - Non-blocking analysis: Async processing for performance
  - Task management: Graceful shutdown with task completion
- **Channel Subscriptions**:
  - `new_http`: HTTP flow events
  - `new_weird`: Zeek weird log events
  - `new_flow`: General flow events

2.4.4.6.9 Integration & Special Features

- **Zeek Integration**:
  - Comprehensive HTTP log parsing

  - Weird log analysis for protocol anomalies
  - Flow metadata correlation
- **External Intelligence**:
  - Online user agent database queries
  - MAC vendor database integration
  - Domain reputation checking
- **Time Management**:
  - 5-minute sliding windows for protocol detection
  - Real-time vs. historical analysis
  - Evidence accumulation and correlation
- **Performance Optimization**:
  - Lock-based thread safety for shared data
  - Bisect-based timestamp searching for efficiency
  - Periodic cleanup of historical data

2.4.4.6.10 Strengths

- **Strengths**:
  - Comprehensive HTTP protocol analysis
  - Application-layer threat detection
  - User agent intelligence integration
  - Low false positive rate through correlation
  - Real-time and historical analysis
  - Configurable sensitivity thresholds
- **Best Practices**:
  - Enable Zeek HTTP logging for full visibility
  - Configure appropriate whitelists for legitimate domains
  - Monitor pastebin download alerts for potential malware
  - Review incompatible UA alerts for device impersonation
  - Use in conjunction with SSL analyzer for complete web traffic analysis

The HTTP Analyzer module is particularly valuable for web security monitoring, malware detection, and policy compliance enforcement. By focusing on the application layer, it provides critical visibility into web-based attacks that may bypass network-layer security controls, making it essential for comprehensive threat detection in modern network environments.

2.4.4.7 Leak Detector Module

2.4.4.7.1 Module Purpose

The Leak Detector module specializes in sensitive data leakage detection by applying YARA rules to network packet captures (PCAPs). It identifies accidental or malicious disclosure of sensitive information such as GPS coordinates, credentials, API keys, and other confidential data in network traffic. This module is particularly valuable for forensic analysis, compliance auditing, and data loss prevention scenarios where post-capture analysis of network traffic is required to identify information leaks.

2.4.4.7.2 Input Data

- **Type of input**:
  - Network packet captures (PCAP files)
  - Pre-captured network traffic for offline analysis
  - Packet payloads and network packet contents
- **Features analyzed**:
  - Raw packet payloads and application data
  - Protocol-specific data structures
  - Pattern matching in packet content
  - Metadata from packet headers
- **Data characteristics**:
  - PCAP-only operation: Requires pre-captured traffic files
  - Deep packet inspection: Analyzes packet payload content
  - Offline analysis: Post-capture processing mode
  - Pattern-based detection: YARA rule matching

2.4.4.7.3 Detection Methodology (How it works)

The module employs YARA-based pattern matching for sensitive data detection:

- **Processing Pipeline**:
  - **Rule Compilation**: Automatically compiles YARA rules from source files
  - **PCAP Analysis**: Applies compiled rules to PCAP file contents
  - **Pattern Matching**: Scans packet payloads for sensitive data patterns
  - **Evidence Generation**: Creates alerts for detected leaks with packet context
- **Key Components**:
  - **YARA Engine**: Pattern matching engine for sensitive data detection
  - **Rule Management**: Automatic compilation and loading of detection rules
  - **Packet Context Extraction**: Identifies source/destination of leaked data

- **Evidence Correlation**: Links detected leaks to specific packets and flows

2.4.4.7.4 Detection Capabilities

- **Types of leaks detected**:
  - **GPS/Location Data**: Latitude/longitude coordinates in network traffic
  - **Credentials**: Plaintext passwords and authentication tokens
  - **API Keys**: Secret keys in API communications
  - **Personal Information**: PII data in cleartext transmissions
  - **Configuration Secrets**: Database credentials, encryption keys
  - **Sensitive Documents**: Confidential documents in transit
- **Example Detection (GPS Leaks)**:

```yara
rule NETWORK_gps_location_leaked {
    strings:
        $rgx_gps_lat = /(lat|latitude){1}(\":|=){1}(-){0,1}\d{1,3}(.){1}\d{2,16}/i
        $rgx_gps_lon = /(lon|lng|long|longitude){1}(\":|=){1}(-){0,1}\d{1,3}(.){1}\d{2,16}/i
        $rgx_gps_loc = /(locations|ll|q|latlon|path){1}(\":|=){1}(-){0,1}\d{1,3}(.){1}\d{2,16}(,){1}(-){0,1}\d{1,3}(.){1}\d{2,16}/i
    condition:
        ($rgx_gps_lat and $rgx_gps_lon) or $rgx_gps_loc
}
```

- **Pattern Matching Logic**:
  - **Regular Expressions**: Complex pattern matching for structured data
  - **Case Insensitivity**: Broad detection coverage
  - **Multi-pattern Conditions**: Combined rule conditions for accuracy
  - **Context Awareness**: Meta-information for rule validation

2.4.4.7.5 Configuration Requirements

- **System Dependencies**:
  - **YARA**: Pattern matching engine (`sudo apt install yara`)
  - **yara-python**: Python bindings for YARA
  - **tshark**: Wireshark CLI for packet analysis (`sudo apt install wireshark`)
  - **PCAP Input**: Network capture files for analysis

- **Directory Structure**:

```text
modules/leak_detector/yara_rules/
├── rules/           # Source YARA rules (.yara files)
└── compiled/        # Compiled YARA rules (auto-generated)
```

- **Operation Mode**:
  - **PCAP Analysis Only**: Requires PCAP file input
  - **Automatic Rule Compilation**: On-module startup
  - **Packet Context Extraction**: Uses tshark for packet information

2.4.4.7.6 Detection Logic

- **Normal behavior**: Network traffic without sensitive data patterns
- **Detection flow**:

```text
PCAP File → Packet Extraction → YARA Rule Application →
Pattern Matching → Packet Context Analysis → Evidence Generation
```

- **Rule application**:
  - **Rule Loading**: Compiles and loads all YARA rules from rules directory
  - **Packet Scanning**: Applies rules to each packet payload
  - **Match Validation**: Verifies pattern matches in packet context
  - **Evidence Creation**: Generates detailed alerts with packet information
- **Advanced Features**:
  - **Automatic Rule Discovery**: Scans rules directory for new YARA files
  - **Rule Compilation Cache**: Stores compiled rules for performance
  - **Packet Metadata Extraction**: Captures source/destination IPs and ports
  - **Match Context Preservation**: Stores surrounding packet data for analysis

2.4.4.7.7 Evidence Handling

- **Evidence types**:
  - `SENSITIVE_DATA_LEAK`: General sensitive data detection
  - `GPS_LOCATION_LEAK`: Geographic coordinate leaks
  - `CREDENTIAL_LEAK`: Authentication data exposure
  - `API_KEY_LEAK`: Secret key disclosure
  - `PII_LEAK`: Personal identifiable information exposure
- **Information included**:
  - **Matched Pattern**: Specific YARA rule and strings matched

- **Packet Metadata**: Source/destination IPs, ports, timestamps
- **Match Context**: Surrounding data from the packet
- **Rule Metadata**: Author, description, reference information
- **Confidence Score**: Based on pattern specificity and context

- **Forensic Value**:
  - Complete packet context for investigation
  - Rule-based classification of leak type
  - Timestamp correlation with other events
  - Source/destination attribution for incident response

2.4.4.7.8 Module Architecture

- **Core Components**:
  - **YARA Rule Manager**: Handles rule compilation and loading
  - **PCAP Analyzer**: Processes packet capture files
  - **Pattern Matcher**: Applies YARA rules to packet content
  - **Evidence Generator**: Creates structured evidence from matches
  - **Packet Context Extractor**: Uses tshark for packet metadata
- **File Structure**:
  - **Source Rules**: `modules/leak_detector/yara_rules/rules/*.yara`
  - **Compiled Rules**: `modules/leak_detector/yara_rules/compiled/`
  - **Main Module**: `modules/leak_detector/leak_detector.py`
- **Processing Model**:
  - **Offline Analysis**: Post-capture processing of PCAP files
  - **Batch Processing**: Complete PCAP file analysis
  - **Rule-based Detection**: YARA pattern matching engine
  - **Metadata Extraction**: Packet context for evidence

2.4.4.7.9 Integration & Extensibility

- **Rule Extension**: Add custom YARA rules to detection repertoire
- **Integration Points**:
  - PCAP file processing pipeline
  - Forensic analysis workflows
  - Compliance auditing systems
  - Data loss prevention frameworks
- **Customization Options**:
  - **New Rule Creation**: Add domain-specific detection patterns

- **Rule Optimization**: Tune existing rules for accuracy/performance
- **Context Enrichment**: Enhance evidence with additional metadata
- **Output Formats**: Custom evidence reporting formats

- **Extending the Module**:
  - Add new YARA rule files to `modules/leak_detector/yara_rules/rules/`
  - Rules are automatically compiled and loaded on module startup
  - Custom patterns can detect organization-specific sensitive data
  - Rule metadata supports categorization and prioritization

2.4.4.7.10 Strengths & Limitations

- **Strengths**:
  - **Deep Packet Inspection**: Analyzes packet payload content
  - **Flexible Detection**: Customizable YARA rules for any pattern
  - **Forensic Capability**: Complete packet context for investigations
  - **Low False Positives**: Rule-based matching with contextual validation
  - **Extensible Architecture**: Easy addition of new detection patterns
- **Limitations**:
  - **PCAP-Only**: Requires pre-captured traffic files
  - **Offline Analysis**: Not suitable for real-time detection
  - **Encryption Blindness**: Cannot analyze encrypted payloads
  - **Performance Intensive**: Deep packet inspection is resource-heavy
  - **Rule Maintenance**: Requires ongoing rule updates and tuning
- **Best Practices**:
  - Use for post-incident analysis and forensic investigations
  - Apply to suspicious traffic captures for sensitive data scanning
  - Maintain updated YARA rules for current threat patterns
  - Combine with real-time modules for comprehensive coverage
  - Review match contexts to validate detection accuracy

The Leak Detector module is particularly valuable for security audits, compliance verification, incident response, and forensic investigations where identifying sensitive data exposure in network traffic is critical. While limited to offline PCAP analysis, it provides deep inspection capabilities that complement real-time detection modules for comprehensive security monitoring.

2.4.4.8 Update Manager Module

2.4.4.8.1 Module Purpose

The Update Manager module is responsible for automatically maintaining and updating Threat Intelligence (TI) feeds, configuration files, and databases used by SmartShield. It ensures that the detection system remains current with the latest IoCs (Indicators of Compromise), signatures, and intelligence data from multiple sources. By automating the update process, this module eliminates manual maintenance overhead and ensures that SmartShield's detection capabilities evolve with the threat landscape. It provides version control, integrity verification, and scheduled updates for all external data sources.

2.4.4.8.2 Input Data

- **Type of input**:
  - Remote TI feed URLs (IPs, domains, JA3, SSL certificates)
  - Local configuration files (organizations, ports, services)
  - MAC address databases
  - Online whitelists (e.g., Tranco)
  - RiskIQ threat intelligence API
- **Update sources**:
  - **Remote TI Feeds**: 45+ external threat intelligence sources
  - **Local Files**: Organization info, port mappings, service definitions
  - **Specialized Feeds**: JA3 fingerprints, SSL certificate hashes
  - **External APIs**: RiskIQ domain intelligence
  - **Online Resources**: MAC address databases, domain whitelists
- **Update mechanisms**:
  - HTTP/HTTPS downloads with ETag validation
  - API-based intelligence gathering
  - Local file hash verification
  - Scheduled periodic updates

2.4.4.8.3 Update Management (How it works)

The module employs multiple update strategies based on data source type:

1. **Remote TI Feed Updates**:
   - **Frequency**: Configurable (default: 24 hours via `TI_files_update_period`)
   - **Validation**: ETag/Last-Modified headers for change detection
   - **Processing**: Automatic parsing and database integration

- **Types**: IP addresses, domains, JA3 fingerprints, SSL hashes

2. **Local File Updates**:
   - **Detection**: SHA256 hash comparison for file changes
   - **Scope**: Organization info, port mappings, service definitions
   - **Automatic**: Updates when local files are modified
   - **Caching**: Optimized to avoid redundant processing
3. **MAC Database Updates**:
   - **Frequency**: Configurable via `mac_db_update_period`
   - **Source**: External MAC address vendor database
   - **Purpose**: Device type identification and vendor mapping
4. **Online Whitelist Updates**:
   - **Frequency**: Configurable via `online_whitelist_update_period`
   - **Content**: Top domain whitelists (e.g., Tranco list)
   - **Purpose**: Reduce false positives for legitimate domains
5. **RiskIQ Intelligence**:
   - **API-based**: Requires credentials configuration
   - **Frequency**: Weekly domain threat intelligence
   - **Content**: Malicious domains detected by RiskIQ

2.4.4.8.4 Configuration Requirements

- **Configuration files**:
  - Main configuration via `smartshield.yaml`
  - TI feed definitions in configuration files
  - RiskIQ credentials (optional)
  - Update period settings
- **Key parameters**:
  - `TI_files_update_period`: Default 86400 seconds (24 hours)
  - `wait_for_TI_to_finish`: Block startup until updates complete
  - `mac_db_update_period`: MAC database update frequency
  - `online_whitelist_update_period`: Whitelist refresh interval
- **External dependencies**:
  - Internet connectivity for remote updates
  - API credentials for premium services (RiskIQ)
  - File system write permissions for cached data
- **Locking mechanism**:
  - Exclusive lock to prevent multiple instances updating simultaneously

- Prevents race conditions and data corruption

2.4.4.8.5 Update Logic

- **Normal operation**: Scheduled updates with integrity verification
- **Update flow**:

```text
Check Update Schedule → Verify Data Freshness →
Download/Retrieve Data → Validate Integrity →
Parse and Process → Update Database → Cleanup
```

- **Validation strategies**:
  - **ETag/Last-Modified**: For HTTP resources with change detection
  - **Hash Comparison**: For local file modifications
  - **Time-based**: For scheduled periodic updates
  - **API-based**: For external intelligence services
- **Advanced Features**:
  - **Concurrent Updates**: Parallel processing of multiple feeds
  - **Error Resilience**: Retry mechanisms and graceful failure handling
  - **Memory Management**: Streaming processing for large files
  - **Version Tracking**: ETag and hash-based version control
  - **Cleanup Operations**: Removal of outdated or unused cached data

2.4.4.8.6 Update Capabilities

- **Data types managed**:
  - **IP Addresses**: Malicious IP ranges and individual addresses
  - **Domains**: Suspicious and malicious domain names
  - **JA3 Fingerprints**: TLS client fingerprint intelligence
  - **SSL Certificates**: Malicious SSL certificate hashes
  - **MAC Addresses**: Vendor database for device identification
  - **Organization Data**: Company-specific IP ranges and domains
  - **Port Mappings**: Service-to-port mappings and organization-specific ports
  - **Whitelists**: Legitimate domain and service exemptions
- **Intelligence sources**:
  - **Public Feeds**: Abuse.ch, CERT-PL, and other open-source TI
  - **Specialized Lists**: JA3 and SSL certificate blacklists
  - **Commercial Intelligence**: RiskIQ domain threat data
  - **Community Resources**: MAC address vendor databases

- **Local Intelligence**: Organization-specific configurations

2.4.4.8.7 Database Integration

- **Update targets**:
  - **IoC Database**: IPs, domains, ranges with threat metadata
  - **JA3 Database**: Malicious TLS fingerprint signatures
  - **SSL Database**: Compromised certificate hashes
  - **MAC Database**: Vendor-to-address mappings
  - **Organization Database**: Company-specific intelligence
  - **Whitelist Database**: Legitimate domain exemptions
- **Metadata storage**:
  - **Source attribution**: Feed origins and timestamps
  - **Threat levels**: Severity classification per IoC
  - **Tags**: Categorization and classification labels
  - **Descriptions**: Contextual information and intelligence
- **Performance optimization**:
  - **Bloom filters**: Efficient IoC existence checking
  - **Hash-based indexing**: Fast lookup and deduplication
  - **Incremental updates**: Change-based processing
  - **Memory-efficient structures**: Streamlined data storage

2.4.4.8.8 Module Architecture

- **Core Components**:
  - **Update Manager**: Main orchestration and scheduling
  - **Timer Manager**: Infinite timer for periodic updates
  - **Feed Parser**: Specialized parsers for different data formats
  - **Database Interface**: Efficient data storage and retrieval
  - **HTTP Client**: Download management with validation
- **Timer System**:
  - **Multiple Timers**: Separate schedules for different data types
  - **Infinite Timer**: Continuous operation with configurable intervals
  - **Graceful Shutdown**: Proper termination and cleanup
  - **Concurrency Safety**: Thread-safe timer management
- **Processing Pipelines**:
  - **TI Feed Pipeline**: Download -> Parse -> Validate -> Store
  - **Local File Pipeline**: Hash check -> Parse -> Update

- **API Pipeline**: Authenticate -> Query -> Process -> Integrate
- **Database Pipeline**: Deduplicate -> Enrich -> Index -> Store

- **Locking Mechanism**:
  - **Exclusive Process Lock**: Prevents concurrent updates
  - **File-based Locking**: Cross-process coordination
  - **Graceful Failure**: Clean handling of lock contention
  - **Timeout Management**: Configurable wait periods

2.4.4.8.9 Integration & Special Features

- **SmartShield Ecosystem Integration**:
  - **Startup Coordination**: Optional blocking until updates complete
  - **Module Dependency**: All detection modules use updated intelligence
  - **Configuration Syncing**: Local file changes propagate automatically
  - **Performance Impact**: Background operation minimizes interference
- **Error Handling**:
  - **Retry Logic**: Multiple attempts for failed downloads
  - **Partial Updates**: Continue despite individual feed failures
  - **Error Logging**: Detailed diagnostics for troubleshooting
  - **Graceful Degradation**: Fallback to cached data when updates fail
- **Performance Optimization**:
  - **Parallel Processing**: Concurrent feed downloads and parsing
  - **Memory Efficiency**: Streaming processing for large files
  - **Cache Management**: Intelligent caching and cleanup
  - **Database Optimization**: Batch operations and indexing

2.4.4.8.10 Strengths & Limitations

- **Best Practices**:
  - **Configure Appropriate Intervals**: Balance freshness vs. resource usage
  - **Enable Startup Blocking**: For critical environments requiring current intelligence
  - **Monitor Update Logs**: Regular review of update success/failure
  - **Maintain Credentials**: Keep API credentials current for premium services
  - **Allocate Resources**: Ensure sufficient disk space and memory for updates
  - **Review Feed Sources**: Periodically evaluate TI feed effectiveness

The Update Manager module is critical for maintaining SmartShield's detection effectiveness over time. By ensuring that all intelligence sources remain current and

accurate, it provides the foundation for up-to-date threat detection while minimizing administrative overhead. The module's automatic operation, comprehensive coverage, and robust error handling make it an essential component for production deployments where detection accuracy and minimal maintenance are paramount.

2.4.4.9 IP Info Module

2.4.4.9.1 Module Purpose

The IP Info module serves as the intelligence enrichment engine for SmartShield, gathering comprehensive metadata about network entities (IP addresses, MAC addresses, domains) from multiple sources. It enriches detection capabilities by providing contextual information such as geolocation, autonomous system numbers (ASN), device vendors, and domain registration details. This enrichment transforms raw network data into actionable intelligence, enabling more sophisticated threat detection and reducing false positives through better understanding of network entities' characteristics and legitimacy.

2.4.4.9.2 Input Data

- **Type of input**:
  - IP addresses from network flows
  - MAC addresses from ARP and DHCP logs
  - Domain names from DNS queries
  - Network interface information
- **Data sources**:
  - **Offline databases**: GeoLite2 for ASN and geolocation, MAC address vendor database
  - **Online services**: ip-api.com for ASN, macvendors.com for vendor lookup
  - **Network protocols**: Reverse DNS lookups, WHOIS queries
  - **System utilities**: ARP cache, network interface configuration
- **Enrichment targets**:
  - IP addresses -> ASN, geolocation, reverse DNS
  - MAC addresses -> Vendor/organization
  - Domains -> Registration age, organization
  - Network interfaces -> Gateway identification

2.4.4.9.3 Intelligence Gathering Capabilities (How it works)

The module employs multiple intelligence gathering techniques for comprehensive entity profiling:

1. **ASN (Autonomous System Number) Intelligence**:
   - **Offline lookup**: GeoLite2 ASN database (`databases/GeoLite2-ASN.mmdb`)
   - **Online fallback**: ip-api.com service for missing ASNs
   - **Range caching**: WHOIS-based range detection and caching via cymru.com TXT DNS queries
   - **Update strategy**: Monthly refresh with intelligent caching
2. **Geolocation Intelligence**:
   - **Offline database**: GeoLite2 Country database (`databases/GeoLite2-Country.mmdb`)
   - **Classification**: Country identification with "Private" designation for internal IPs
   - **Real-time processing**: Immediate enrichment for new IP discoveries
3. **MAC Address Vendor Intelligence**:
   - **Offline database**: MAC vendor JSON database (`databases/macaddress-db.json`)
   - **Online fallback**: macvendors.com API for unknown vendors
   - **Automatic updates**: Bi-weekly database updates via Update Manager
   - **Caching**: LRU caching (700 entries) for performance optimization
4. **Reverse DNS Intelligence**:
   - **Standard protocol**: In-addr.arpa DNS queries
   - **Validation**: IP address verification to filter out self-referential records
   - **Dual-stack support**: IPv4 and IPv6 reverse lookups
5. **Domain Intelligence**:
   - **WHOIS queries**: Domain age and registration organization
   - **TLD validation**: Filtering of unsupported and special domains (.arpa, .local)
   - **Hierarchical lookup**: Parent domain analysis for subdomain context
6. **Gateway Intelligence**:
   - **Automatic detection**: Gateway IP and MAC identification for interface monitoring
   - **Multi-interface support**: Handling of complex network topologies
   - **Access Point mode**: Special handling for wireless access point deployments
7. **JARM Fingerprinting**:
   - **Active fingerprinting**: TLS server fingerprinting for threat intelligence
   - **Multi-probe approach**: 10 different TLS handshake variations
   - **Hash generation**: Fuzzy hashing algorithm for fingerprint matching
   - **Integration**: Malicious JARM hash detection with evidence generation

2.4.4.9.4 Configuration Requirements

- **Database files**:
  - `databases/GeoLite2-ASN.mmdb`: MaxMind ASN database
  - `databases/GeoLite2-Country.mmdb`: MaxMind geolocation database
  - `databases/macaddress-db.json`: MAC vendor database
- **External services**:
  - **Optional**: Internet connectivity for online lookups
  - **Rate limiting**: Built-in caching to minimize external requests
  - **Fallback mechanisms**: Graceful degradation when services unavailable
- **Network permissions**:
  - DNS resolution capabilities
  - Socket operations for reverse DNS and JARM fingerprinting
  - Network interface query permissions
- **Update dependencies**:
  - Integration with Update Manager for database refreshes
  - MAC database update period configurable via `mac_db_update`

2.4.4.9.5 Intelligence Processing Logic

- **Processing flow**:

```text
Entity Discovery → Source Prioritization → Data Retrieval →
Validation & Enrichment → Database Storage → Context Integration
```

- **Priority hierarchy**:
  - **Local cache**: Previously gathered intelligence
  - **Offline databases**: Built-in GeoLite2 and MAC databases
  - **Online services**: External APIs with rate limiting
  - **Protocol queries**: DNS, WHOIS, and active probing
- **Advanced Processing Features**:
  - **Intelligent caching**: ASN range caching to minimize WHOIS queries
  - **Parallel processing**: Asynchronous database operations
  - **Error resilience**: Graceful handling of service failures
  - **Memory optimization**: LRU caching and efficient data structures
  - **Update coordination**: Synchronization with Update Manager

2.4.4.9.6 Intelligence Coverage

- **Entity types enriched**:

- **IP Addresses**: ASN, geolocation, reverse DNS, threat context
- **MAC Addresses**: Vendor/organization, device type inference
- **Domains**: Registration age, organization, hierarchical context
- **Network Gateways**: Identification and profiling
- **SSL/TLS Servers**: JARM fingerprints for threat detection

- **Metadata dimensions**:
  - **Geographic**: Country-level geolocation
  - **Organizational**: ASN and company associations
  - **Technical**: Network ranges and protocol characteristics
  - **Temporal**: Domain registration ages
  - **Device**: Manufacturer and vendor information
  - **Behavioral**: TLS fingerprint patterns

2.4.4.9.7 Database Integration

- **Storage locations**:
  - **IPsInfo**: ASN, geolocation, reverse DNS for IP addresses
  - **MACinfo**: Vendor associations for MAC addresses
  - **DomainsInfo**: Age and organization for domains
  - **Profiles**: MAC vendor associations per network profile
  - **Gateway info**: Default gateway identification per interface
- **Data structures**:
  - **Nested dictionaries**: Hierarchical intelligence storage
  - **Timestamp tracking**: Update timing for refresh decisions
  - **Relationship mapping**: Entity-to-intelligence associations
  - **Cache management**: Intelligent expiration and refresh
- **Performance features**:
  - **Bloom filters**: Efficient existence checking
  - **Range-based caching**: ASN range optimizations
  - **LRU caches**: Frequently accessed vendor lookups
  - **Batch operations**: Efficient database updates

2.4.4.9.8 Module Architecture

- **Core Components**:
  - **Main IP Info Engine**: Primary intelligence gathering and coordination
  - **ASN Manager**: Specialized ASN lookup with range caching
  - **JARM Fingerprinter**: Active TLS fingerprinting implementation

- **Gateway Detector**: Network gateway identification and profiling
- **Database Interface**: Efficient intelligence storage and retrieval

- **Sub-modules**:
  - `asn_info.py`: ASN-specific intelligence gathering
  - `jarm.py`: JARM fingerprinting implementation
  - **Integrated with SmartShield's whitelisting system**
- **Processing Model**:
  - **Event-driven**: Intelligence gathering triggered by entity discovery
  - **Asynchronous**: Non-blocking operations for performance
  - **Layered**: Multiple fallback sources for reliability
  - **Cached**: Intelligent caching to minimize redundant operations
- **Channel Subscriptions**:
  - `new_ip`: New IP address discoveries
  - `new_MAC`: New MAC address detections
  - `new_dns`: New domain queries for intelligence
  - `check_jarm_hash`: JARM fingerprinting requests

2.4.4.9.9 Integration & Special Features

- **SmartShield Ecosystem Integration**:
  - **Threat Intelligence**: Enrichment for IoC detection
  - **Whitelisting**: Organization-based exemption support
  - **Profiling**: Enhanced entity profiles for behavioral analysis
  - **Gateway awareness**: Network topology understanding
- **JARM Fingerprinting Integration**:
  - **Active detection**: On-demand TLS server fingerprinting
  - **Threat matching**: Malicious JARM hash database checking
  - **Evidence generation**: Alert creation for malicious fingerprints
  - **Performance optimized**: Efficient fingerprinting algorithms
- **Network Context Awareness**:
  - **Gateway identification**: Automatic detection of network gateways
  - **Interface profiling**: Multi-interface environment support
  - **AP mode handling**: Special processing for access point deployments
  - **Topology mapping**: Basic network structure understanding
- **Performance Optimizations**:
  - **Intelligent caching**: Minimizes external API calls
  - **Parallel processing**: Asynchronous database operations

- **Memory efficiency**: LRU caches and optimized data structures
- **Update coordination**: Synchronized with database refresh cycles

The IP Info module is the foundation of contextual awareness in SmartShield, transforming raw network data into enriched intelligence that powers more accurate threat detection, reduces false positives, and provides valuable context for security analysis and incident response.

2.4.4.10 Blocking Module

2.4.4.10.1 Module Purpose

The Blocking module serves as SmartShield's active response mechanism, providing real-time network-level threat mitigation by automatically blocking malicious IP addresses. This module translates detection intelligence into concrete security actions, preventing ongoing attacks and containing threats at the network perimeter. By integrating directly with the system's firewall (iptables), it enables automated incident response that stops attacks in progress, reduces attack surface, and prevents further compromise. The blocking functionality represents the final layer of defense in the SmartShield security stack, converting threat intelligence into actionable protection.

2.4.4.10.2 Operational Overview

- **Core Function**: Real-time IP blocking via iptables firewall rules
- **Trigger Condition**: Automatic blocking upon malicious IP detection
- **Blocking Scope**: All network traffic to/from malicious IPs
- **Persistence**: Permanent blocking until manual intervention
- **Platform Support**: Linux systems with iptables (interface mode only)
- **Activation**: Enabled via command-line flag (`-p` or `--blocking`)

2.4.4.10.3 Architecture & Components

- **Core Module (`blocking.py`)**:
  - **Main controller**: Orchestrates all blocking operations
  - **Firewall management**: iptables chain initialization and rule management
  - **Logging system**: Persistent logging of all blocking actions
  - **Interface handling**: Multi-interface and Access Point mode support
- **Unblocker Subsystem (`unblocker.py`)**:
  - **Scheduled unblocking**: Time-based blocking expiration
  - **Request management**: Queue for pending unblock operations
  - **Thread management**: Background processing of unblock requests
  - **Time window tracking**: Time-based blocking duration management

- **Command Executor** (`exec_iptables_cmd.py`):
  - **Firewall interaction**: Safe iptables command execution
  - **Rule construction**: Dynamic firewall rule generation
  - **Error handling**: Graceful failure recovery
  - **Command validation**: Secure command parameter handling
- **Chain Manager** (`smartshield_chain_manager.py`):
  - **Firewall chain management**: Dedicated iptables chain operations
  - **Cleanup operations**: Graceful shutdown and resource release
  - **Chain validation**: Existence checking and initialization
  - **Integration cleanup**: Proper removal of firewall rules

2.4.4.10.4 Blocking Mechanism (How it works)

1. **Initialization Phase**:

```text
System Startup → Firewall Detection → Chain Creation →
Rule Integration
```

   - **Firewall detection**: Identifies iptables availability
   - **Chain creation**: Establishes dedicated `smartshieldBlocking` chain
   - **Rule integration**: Inserts jumps from INPUT/OUTPUT/FORWARD chains
   - **Platform validation**: Confirms Linux/iptables compatibility

2. **Triggering & Decision Making**:

```text
Detection Alert → IP Evaluation → Blocking Decision → Rule
Application
```

   - **Alert reception**: Receives `new_blocking` channel messages
   - **IP validation**: Verifies IP format and current blocking status
   - **Duplicate prevention**: Checks existing iptables rules
   - **Scope determination**: Private vs. public IP handling

3. **Rule Application**:

```text
Rule Construction → iptables Execution → Result
Verification → Logging
```

   - **Bidirectional blocking**: Default block both incoming/outgoing traffic
   - **Granular control**: Protocol, port, and interface-specific rules

- **Private IP handling**: Interface-specific blocking for internal addresses
- **Public IP handling**: Global blocking across all interfaces

4. **Management & Monitoring**:

```text
Blocking Queue → Time Tracking → Unblock Scheduling → Log Management
```

- **Request queuing**: Tracks all active blocking requests
- **Time window tracking**: Monitors blocking duration
- **Scheduled unblocking**: Prepares future unblock operations
- **Comprehensive logging**: Detailed audit trail of all actions

2.4.4.10.5 Key Technical Features

- **Firewall Integration**:
  - **Dedicated chain**: Isolated `smartshieldBlocking` iptables chain
  - **Rule precedence**: High-priority insertion at chain beginnings
  - **Comment tagging**: All rules tagged with "smartshield rule" for identification
  - **Granular control**: Protocol, port, and direction-specific blocking
- **Intelligent Blocking Logic**:
  - **Private IP handling**: Interface-specific rules for internal addresses
  - **Public IP blocking**: Global rules for external threats
  - **Duplicate prevention**: Prevents redundant rule creation
  - **Validation checks**: Comprehensive IP and rule validation
- **Time Management**:
  - **Time window tracking**: Blocking duration measured in time windows
  - **Scheduled unblocking**: Automatic expiration after specified periods
  - **Duration calculation**: Precise blocking time measurement
  - **Extension support**: Blocking period extension for persistent threats
- **Error Resilience**:
  - **Command validation**: Safe iptables command construction
  - **Error handling**: Graceful failure recovery
  - **State verification**: Continuous chain and rule validation
  - **Logging coverage**: Comprehensive error and operation logging
- **Logging & Audit**:
  - **Dedicated log file**: `blocking.log` with timestamps and details
  - **Operation tracking**: All block/unblock operations logged

- **Duration recording**: Precise blocking time measurements
- **Thread-safe logging**: Concurrent access protection

2.4.4.10.6 Data Flow & Processing

- **Blocking Request Processing**:

```python
# Example blocking request structure
blocking_data = {
    "ip": "192.168.1.100",      # IP to block
    "tw": 1,                    # Current time window
    "block": True,              # Block (True) or unblock (False)
    "from": True,               # Block traffic from IP
    "to": True,                 # Block traffic to IP
    "dport": 80,                # Optional: Destination port
    "sport": None,              # Optional: Source port
    "protocol": "tcp",          # Optional: Protocol
    "interface": "eth0"         # Optional: Network interface
}
```

- **Processing Pipeline**:
  - **Message Reception**: JSON parsing from `new_blocking` channel
  - **Validation**: IP format, duplicate rule, and parameter validation
  - **Rule Construction**: Dynamic iptables command generation
  - **Execution**: Safe iptables command execution with sudo
  - **Result Verification**: Command success/failure checking
  - **State Update**: Database and internal state updates
  - **Logging**: Comprehensive operation logging
  - **Unblock Scheduling**: Future unblock request registration
- **Firewall Rule Examples**:
  - **Default blocking**: `iptables -I smartshieldBlocking -s 192.168.1.100 -j DROP`
  - **Port-specific**: `iptables -I smartshieldBlocking -s 192.168.1.100 -p tcp --dport 443 -j DROP`
  - **Interface-specific**: `iptables -I smartshieldBlocking -s 192.168.1.100 -i eth0 -o eth0 -j DROP`
  - **Bidirectional**: Separate rules for `-s` (source) and `-d` (destination)

2.4.4.10.7 Unblocking System

- **Architecture**:
  - **Request queue**: Thread-safe dictionary of pending unblock requests
  - **Time tracking**: Unix timestamp-based expiration
  - **Background thread**: Continuous expiration checking
  - **Graceful cleanup**: Proper rule removal and state updates
- **Unblocking Logic**:

```text
Time Window Closure → Duration Calculation → Expiration Checking → Rule Removal
```

  - **Time-based expiration**: Blocking duration measured in time windows
  - **Extension support**: Blocking periods can be extended
  - **Clean removal**: Precise iptables rule deletion
  - **State cleanup**: Database and internal state updates
- **Unblocking Process**:
  - **Request registration**: Unblock time calculated and queued
  - **Background monitoring**: Continuous expiration checking
  - **Rule removal**: Precise iptables rule deletion
  - **State cleanup**: Database updates and logging
  - **Request cleanup**: Removal from internal queue

2.4.4.10.8 Integration Points

- **Internal SmartShield Integration**:
  - **Channel subscription**: `new_blocking` for blocking requests
  - **Database coordination**: Blocked IP tracking and state management
  - **Time window sync**: Coordination with profiling system
  - **Logging integration**: Unified logging system
- **System Integration**:
  - **iptables firewall**: Direct rule manipulation via system commands
  - **Network interfaces**: Multi-interface awareness and handling
  - **Sudo permissions**: Proper privilege escalation for firewall changes
  - **Platform services**: System-level integration for persistence
- **Detection Module Integration**:
  - **Alert triggering**: Automatic blocking upon malicious detection
  - **Evidence correlation**: Blocking tied to specific threat evidence

- **Risk scoring**: Integration with threat scoring systems
- **Context awareness**: Intelligent blocking based on threat context

2.4.4.10.9 Operational Characteristics

- **Performance**:
  - **Low latency**: Immediate blocking upon detection
  - **Minimal overhead**: Efficient iptables rule management
  - **Thread-safe**: Concurrent operation support
  - **Resource efficient**: Minimal memory and CPU usage
- **Reliability**:
  - **Error resilience**: Graceful handling of command failures
  - **State consistency**: Database and firewall state synchronization
  - **Recovery mechanisms**: Automatic cleanup on failures
  - **Validation checks**: Comprehensive input and state validation
- **Security**:
  - **Privilege management**: Proper sudo usage for firewall operations
  - **Input validation**: Comprehensive parameter sanitization
  - **Rule validation**: Safe iptables command construction
  - **Access control**: Proper permission handling
- **Maintainability**:
  - **Modular design**: Clean separation of concerns
  - **Comprehensive logging**: Detailed operation tracking
  - **Error reporting**: Clear failure notifications
  - **Configuration flexibility**: Adaptable to different environments

2.4.4.10.10 Usage & Configuration

- **Activation**:

```
# Enable blocking during SmartShield startup
./smartshield.py -i eth0 -p

# Alternative syntax
./smartshield.py --blocking
```

- **Configuration Options**:
  - **Platform restriction**: Linux with iptables only
  - **Interface requirement**: Network interface monitoring mode

- **Permission requirements**: Sudo/root privileges for iptables
- **Dependencies**: `iptables` command availability

- **Manual Interaction**:

```python
# Programmatic blocking request
blocking_data = {
    "ip": "10.0.0.5",
    "tw": 1,
    "block": True,
    "from": True,
    "to": True
}
self.db.publish('new_blocking', json.dumps(blocking_data))
```

- **Monitoring & Management**:
  - **Log file**: `blocking.log` in output directory
  - **Database state**: `blocked_ips` key in Redis database
  - **Firewall state**: `iptables -L smartshieldBlocking -v -n`
  - **System monitoring**: Standard Linux monitoring tools

2.4.4.10.11 OT-Safe Blocking

If SmartShield ever blocks a PLC's IP address with iptables, that PLC stops receiving commands from the SCADA system. The factory production line halts. This is the single most dangerous operational risk of running an IPS in an OT environment. The OT-safe blocking system prevents this. PLCs, HMIs, and SCADA servers listed in config/ot_protected_devices.conf are NEVER passed to `iptables`; no matter what evidence is detected against them.

2.4.4.10.11.1 How OT-Safe Blocking Works

The blocking pipeline has been modified at three levels:

- `modules/blocking/blocking.py` — `_is_ot_protected()` is called before every iptables rule. If the IP is protected, the rule is skipped and a [OT-SAFE] entry is written to blocking.log.
- `smartshield_files/core/database/redis_db/alert_handler.py` — New `mark_as_ot_protected()`, `is_ot_protected()`, `get_ot_protected_ips()` methods persist the protected set in Redis key `ot_protected_ips`.

- `modules/ot_detection/ot_detection.py` — When evidence is set against a protected OT IP, the description includes [OT-SAFE: blocking suppressed] so analysts know immediately.
- **What is NOT blocked**: The protected OT device's own IP address (e.g. the PLC at 192.168.10.10).
- **What IS still blocked**: External IT-side hosts attacking OT devices. If 10.5.1.88 is attacking PLC 192.168.10.10 with a Modbus flood, the attacker IP 10.5.1.88 is blocked normally; only the PLC itself is protected.
- **What always happens**: Evidence, alerts, and SIEM exports are generated regardless of blocking status.

2.4.4.10.11.2 Configuring OT Protected Devices

Edit `config/ot_protected_devices.conf`. Add one IP address or CIDR range per line. Lines starting with `#` are comments.

config

```
# The Purdue Model (ISA-95) typically places OT devices in:
#   Level 0-1: Field devices, PLCs, RTUs       → 192.168.10.0/24
#   Level 2:   HMIs, Supervisory workstations  → 192.168.20.0/24
#   Level 3:   SCADA, Historians               → 192.168.30.0/24
# ── LEVEL 0-1: PLCs, RTUs, Field Controllers
# 192.168.10.10   # PLC-01 Siemens S7-1200 (Assembly Line A)
# 192.168.10.11   # PLC-02 Siemens S7-1500 (Assembly Line B)
# 192.168.10.12   # PLC-03 Allen-Bradley (Conveyor)
# 192.168.10.20   # RTU-01 (Remote Terminal Unit, substation)
# 192.168.10.30   # VFD-01 Variable Frequency Drive
# ── LEVEL 2: HMIs, Engineering Workstations
# 192.168.20.10   # HMI-01 SCADA operator workstation
# 192.168.20.11   # HMI-02 Control room display
# 192.168.20.50   # EWS-01 Engineering workstation (Siemens TIA Portal)
# ── LEVEL 3: SCADA Servers, Historians
# 192.168.30.10   # SCADA-Server-01
# 192.168.30.20   # Historian-01 OSIsoft PI
# 192.168.30.30   # OPC-UA Server
# ── ENTIRE OT SUBNETS (use CIDR to protect a whole segment)
# 192.168.10.0/24   # All field devices
# 192.168.20.0/24   # All HMIs and engineering workstations
```

```
22  # 192.168.30.0/24   # All SCADA / historian servers
```

2.4.4.10.11.3 Configuring Authorized Modbus Masters

Edit `config/modbus_authorized_masters.conf`. List every IP that is legitimately allowed to send Modbus read/write commands. Any unlisted host contacting port 502 on a protected OT device triggers `OT_UNAUTHORIZED_MODBUS_ACCESS`.

```
config
1  # config/modbus_authorized_masters.conf
2  # One exact IP per line. CIDR not supported here.
3  192.168.20.10          # HMI-01 (authorized to read/write PLCs)
4  192.168.20.50          # EWS-01 (engineering workstation, TIA Portal)
5  192.168.30.10          # SCADA-Server-01 (authorized polling)
```

> **Leave both files empty initially**
>
> If you leave `ot_protected_devices.conf` empty, OT-safe blocking is still active but no IPs are protected. If you leave `modbus_authorized_masters.conf` empty, unauthorized-access detection is disabled (flood detection still works). Populate both files before enabling blocking in production.

2.4.4.10.12 Critical Considerations

- **Permanent Blocking**:
  - **Current limitation**: No automatic time-based unblocking
  - **Manual intervention**: Required for IP unblocking
  - **Persistence**: Blocking maintained across SmartShield restarts
  - **Database tracking**: All blocked IPs recorded in database
- **Platform Limitations**:
  - **Linux only**: iptables dependency restricts to Linux
  - **Interface mode**: Requires live interface monitoring
  - **Root privileges**: Sudo/root required for iptables operations
  - **Network impact**: Direct modification of system firewall
- **Operational Impact**:
  - **Network disruption**: Blocking affects all traffic to/from IP
  - **False positive risk**: Legitimate IP blocking potential
  - **Permanent effect**: Manual unblocking required
  - **System dependency**: Relies on iptables and system stability

- **Security Implications**:
  - **Privilege escalation**: Requires elevated permissions
  - **System modification**: Direct firewall rule manipulation
  - **Persistence**: Changes persist beyond SmartShield session
  - **Recovery**: Manual cleanup required for abnormal termination
- **Module Significance**: The Blocking module represents SmartShield's active defense capability, transforming passive detection into proactive protection. By automatically implementing firewall-level blocks against malicious IPs, it provides real-time threat mitigation that stops attacks in progress and prevents further compromise. While powerful, this capability requires careful consideration of operational impact, platform dependencies, and security implications.

2.4.4.11 OT Detection Module

The `modules/ot_detection/ot_detection.py` module detects 20 attack categories. Every detection generates an Evidence object that flows through the normal SmartShield pipeline; visible in the web interface, written to alerts.json, and exported to any configured SIEM.

2.4.4.11.1 Technical Details

2.4.4.11.1.1 Module Architecture

The OT Detection module (`modules/ot_detection/ot_detection.py`) follows the same `IModule` interface as all other SmartShield modules. It is automatically discovered by the process manager because the file name matches the directory name (`ot_detection/ot_detection.py`). It subscribes to five Redis pub/sub channels:

Channel	Source and Contents
`new_flow`	Every Zeek `conn.log` entry — used for flood detection on OT ports (502, 102, 20000, 34964, 44818, 123, 319).
`new_notice`	Zeek `notice.log` entries — catches `PLCModeSwitch`, `WatchdogManipulation`, `ValveCycling`, `MotorCycling` from `ot_protocols.zeek`.
`new_modbus`	Per-PDU Modbus events from Zeek OT parser — function code, exception flag, coil values.
`new_s7comm`	Per-PDU S7Comm events — ROSCTR, function code, read/write operations.

Channel	Source and Contents
`new_dnp3`	Per-PDU DNP3 events — function code, `is_unsolicited`, data objects.

2.4.4.11.1.2 Sliding Window Flood Detection

All flood counters use sliding time windows (not fixed intervals) to avoid boundary-crossing attacks. The implementation:

```python
# Example: Modbus flood detection sliding window
now = time.time()

# Append new connection timestamp
self._modbus_conns[saddr].append(now)

# Evict timestamps older than flood_window_seconds
self._modbus_conns[saddr] = [
    t for t in self._modbus_conns[saddr]
    if now - t ≤ self._flood_window  # default: 60 seconds
]

# Count connections in the current window
count = len(self._modbus_conns[saddr])

# Trigger evidence if threshold exceeded
if count ≥ self._modbus_flood_threshold:  # default: 20
    self._set_evidence(
        ev_type=EvidenceType.OT_MODBUS_CONNECTION_FLOOD,
        ...
    )
```

2.4.4.11.1.3 Alert Deduplication

The module maintains a `self._alerted` set of alert keys (format: `"modbus_flood:<srcip>:<twid>"`) to prevent the same attack from generating hundreds of duplicate evidence objects.

The set is per-instance and resets when the module restarts. Within a time window, each unique `(attack_type, source_ip, dest_ip)` combination generates exactly one `Evidence` object.

2.4.4.11.1.4 Zeek OT Protocol Script

The `zeek-scripts/ot_protocols.zeek` script is loaded via `zeek-scripts/__load__.zeek`. It:

- Hooks into Zeek's built-in Modbus policy (`modbus_message`, `modbus_write_single_coil_request`, `modbus_exception`, etc.).
- Hooks into Zeek's built-in DNP3 policy (`dnp3_application_request_header`).
- Tracks S7Comm connections at the TCP level (port 102) since Zeek has no full S7 parser.
- Detects PROFINET DCP floods on port 34964 UDP via `udp_request` events.
- Detects PTP (IEEE 1588) traffic on port 319 UDP.
- Defines custom Notice types:
  - `PLCModeSwitch`
  - `WatchdogManipulation`
  - `ValveCycling`
  - `MotorCycling`
  - `ModbusWriteCoil`
  - `ModbusIllegalFunction`
  - `S7CommPLCStop`
  - `DNP3UnauthorizedFunc`

If Zeek's OT policy bundle is not available (older Zeek installations), `ot_protocols.zeek` will partially fail. SmartShield falls back gracefully to `conn.log`-based detection, which still handles flood attacks on OT ports.

2.4.4.11.1.5 Evidence Object Structure

Every OT detection creates a fully structured `Evidence` object compatible with the SmartShield evidence pipeline:

```python
Evidence(
    evidence_type=EvidenceType.OT_MODBUS_WRITE_COILS,
    description="Unauthorized Modbus write from 10.5.1.88 to OT device",

```

```
        attacker=Attacker(
            direction=Direction.SRC,
            ioc_type=IoCType.IP,
            value="10.5.1.88",
            profile=ProfileID(ip="10.5.1.88"),
        ),

        victim=Victim(
            direction=Direction.DST,
            ioc_type=IoCType.IP,
            value="192.168.10.10",
        ),

        threat_level=ThreatLevel.CRITICAL,
        profile=ProfileID(ip="10.5.1.88"),
        timewindow=TimeWindow(number=3),
        uid=["CYbGfv3fLZzYMVqj12"],
        timestamp="2024/03/15 14:32:01.000000+0000",
        proto=Proto.TCP,
        dst_port=502,
        confidence=0.95,
    )
```

2.4.4.11.2 Modbus/TCP — Port 502

Evidence Type	Threat Level	Detection Logic
OT_MODBUS_CONNECTION_FLOOD	HIGH	Too many TCP connections/min to port 502 from one source (default threshold: 20/min)
OT_UNAUTHORIZED_MODBUS_ACCESS	HIGH	Source IP not in modbus_authorized_masters.conf contacts a protected OT device on port 502

OT_MODBUS_WRITE_COILS	CRITICAL	Write Single/Multiple Coils or Registers (func codes 5,6,15,16,22,23) from unauthorized host
OT_MODBUS_ILLEGAL_FUNCTION	MEDIUM	Malformed function code (>127) or Modbus exception response from device

2.4.4.11.3 Siemens S7Comm / ISO-TSAP — Port 102

Evidence Type	Threat Level	Detection Logic
OT_S7COMM_JOB_FLOOD	HIGH	High-rate S7 job requests (TCP connections to port 102) within sliding 60s window (threshold: 50)
OT_S7COMM_UNAUTHORIZED_READ	HIGH	S7Comm connection from a host not in the authorized masters list targeting a protected OT device
OT_S7COMM_UNAUTHORIZED_WRITE	CRITICAL	S7 WriteVar command (ROSCTR=1, function 0x05) from unauthorized source
OT_S7COMM_PLC_STOP	CRITICAL	S7 STOP/control-plane command (function 0x29 or 0x00, ROSCTR=1) — halts the PLC immediately

2.4.4.11.4 DNP3 — Port 20000

Evidence Type	Threat Level	Detection Logic
OT_DNP3_FLOOD	HIGH	High-rate DNP3 fragment/request flood (default threshold: 100 requests/60s)
OT_DNP3_UNAUTHORIZED_FUNCTION	HIGH	Function code > 33 which is reserved/dangerous in the DNP3 specification — potential exploitation attempt

2.4.4.11.5 Industrial Network Protocols

Evidence Type	Threat Level	Detection Logic

`OT_PROFINET_DCP_FLOOD`	MEDIUM	PROFINET DCP discovery flood (port 34964 UDP) — can overwhelm embedded PLCs with limited stack memory
`OT_CIP_MESSAGE_FLOOD`	HIGH	EtherNet/IP CIP explicit message flood (port 44818 TCP) — exhausts connection slots on Allen-Bradley controllers
`OT_NTP_TIME_SYNC_ATTACK`	HIGH	Possible NTP spoofing: > 5 NTP responses in 10s from same source — desynchronizes precision OT clocks
`OT_PTP_SPOOFING`	HIGH	IEEE 1588 PTP event message (port 319 UDP) from a non-OT-known source — grandmaster clock spoofing

2.4.4.11.6 PLC Command and Process Attacks

Evidence Type	Threat Level	Detection Logic
`OT_PLC_MODE_SWITCH`	CRITICAL	Rapid RUN/STOP mode-switch commands to a PLC (threshold: 5 switches/60s) — can crash the PLC into fault state
`OT_WATCHDOG_MANIPULATION`	CRITICAL	Watchdog timer disabled or manipulated (from Zeek OT::WatchdogManipulation notice) — safety systems bypassed
`OT_VALVE_CYCLING`	CRITICAL	Repeated open/close valve commands (OT::ValveCycling notice) — causes mechanical wear and process disruption
`OT_MOTOR_CYCLING`	CRITICAL	Rapid motor start/stop commands (OT::MotorCycling notice) — causes overheating and winding damage
`OT_PROTOCOL_ANOMALY`	MEDIUM	Generic OT protocol anomaly from Zeek OT notices — catch-all for patterns not covered by specific detectors

2.4.4.11.7 Confidence and Threat Levels

Every Evidence object has a confidence score (0.0–1.0) and a threat level. The `evidence_detection_threshold` in `smartshield.yaml` (set to `0.15` for factory deployments) controls how much accumulated evidence is required before an Alert is triggered. OT

attacks use higher confidence scores (0.85–0.95) because protocol behavior is well-defined.

Threat Level	**Evidence Types Assigned**
CRITICAL (0.95)	• `OT_S7COMM_PLC_STOP`, • `OT_MODBUS_WRITE_COILS`, • `OT_PLC_MODE_SWITCH`, • `OT_WATCHDOG_MANIPULATION`, • `OT_VALVE_CYCLING`, • `OT_MOTOR_CYCLING`
HIGH (0.80–0.90)	• `OT_MODBUS_CONNECTION_FLOOD`, • `OT_UNAUTHORIZED_MODBUS_ACCESS`, • `OT_S7COMM_JOB_FLOOD`, • `OT_DNP3_FLOOD`, • `OT_CIP_MESSAGE_FLOOD`, • `OT_NTP_TIME_SYNC_ATTACK`, • `OT_PTP_SPOOFING`
MEDIUM (0.70–0.80)	• `OT_MODBUS_ILLEGAL_FUNCTION`, • `OT_DNP3_UNAUTHORIZED_FUNCTION`, • `OT_PROTOCOL_ANOMALY`, • `OT_PROFINET_DCP_FLOOD`

2.4.5 SmartShield Evidence & Alert Processing Pipeline

The evidence and alert processing system in SmartShield is the central nervous system that transforms individual detections into actionable security alerts. This comprehensive pipeline ensures that raw detection data from various modules is properly collected, analyzed, aggregated, and presented for security analysis. The core components are:

1. Evidence Handler (`EvidenceHandler`): The primary coordinator that manages the entire evidence lifecycle from collection to alert generation.
2. Evidence Logger (`EvidenceLogger`): A dedicated background worker that handles asynchronous logging to prevent I/O operations from blocking the main evidence processing pipeline.
3. Alert Structure (`Alert` dataclass): The structured representation of aggregated security incidents that cross the detection threshold.

We will see how the complete processing pipeline

2.4.5.1 Phase 1: Evidence Generation & Collection

- Source: Individual detection modules (Threat Intelligence, Flow Alerts, ARP, HTTP Analyzer,etc.)
- Process:
  1. **Module Detection**: Each module analyzes network traffic and identifies suspicious patterns.
  2. **Evidence Creation**: Module creates an `Evidence` object with details:
     - Attacker/Victim information
     - Evidence type (from predefined `EvidenceType` enum)
     - Threat level (LOW, MEDIUM, HIGH, CRITICAL)
     - Confidence score (0.0 to 1.0)
     - Timestamps and flow UIDs
     - Description and context
  3. **Channel Publication**: Evidence is published to `evidence_added` channel as JSON.

2.4.5.2 Phase 2: Evidence Processing

Handler: `EvidenceHandler.main()` method.

Steps:

1. **Channel Listening**: Continuously monitors `evidence_added` channel.
2. **JSON Deserialization**: Converts JSON evidence back to `Evidence` object.

3. **Whitelist Validation**: Checks if evidence should be ignored (false positives).
4. **Timestamp Processing**: Converts to local timezone if running live.
5. **Format Enhancement**: Adds threat level context to evidence description.
6. **Asynchronous Logging**: Queues evidence for background file writing.
7. **Attack Counter Update**: Tracks evidence types per attacker.
8. **Threat Level Accumulation**: Updates running total for profile/timewindow.
9. **Real-time Export**: Immediately streams evidence to exporting modules.

2.4.5.3 Phase 3: Threshold Evaluation & Alert Generation

- **Logic**: Accumulated threat level vs. configured threshold.
- **Calculation**:
  - **Detection Threshold**: Configured in `evidence_detection_threshold` (default: 0.15 attacks/minute).
  - **Time Window Adjustment**: Threshold adjusted for current time window width (1-hour = 3600 seconds).

$$\text{threshold per width} = \text{evidence\_detection\_threshold} \times \frac{\text{time\_window\_width}}{60} \tag{2.1}$$

  - **Accumulation**: `accumulated_threat_level` = Sum of all threat level for all evidence in a single time window.
  - **Alert Trigger**: When

```
accumulated_threat_level ≥ detection_threshold_in_this_width (0.15)
```

- **Alert Creation Process**:
  - **Evidence Collection**: Gathers all relevant evidence for profile/timewindow.
  - **Filtering**: Excludes already-alerted and whitelisted evidence.
  - **Alert Object Creation**: Alert with:
    - Profile (attacker IP)
    - Timewindow details
    - Last triggering evidence
    - Accumulated threat level
    - List of evidence IDs causing alert
    - Calculated confidence (normalized 0-1)

Example Calculation: Threat Accumulation and Alert Triggering

Consider a 1-hour time window (3600 seconds) with the default detection threshold of 0.15 attacks per minute. The threshold for this window is calculated as:

$$\text{Threshold} = 0.15 \times \frac{3600}{60} = 9.0 \tag{2.2}$$

Threat levels are weighted as follows: LOW = 0.2, MEDIUM = 0.5, HIGH = 0.8, CRITICAL = 1.0. Each evidence item has a confidence score (0.0 to 1.0). The accumulated threat level is the sum of ("threat_level" × "confidence") for all evidence in the time window.

Scenario 1: No Alert

- Evidence: 3 MEDIUM threats with confidence 1.0 each → $3 \times 0.5 \times 1.0 = 1.5$
- Evidence: 5 LOW threats with confidence 1.0 each → $5 \times 0.2 \times 1.0 = 1.0$
- Total accumulated threat level = $1.5 + 1.0 = 2.5$
- Since $2.5 < 15.0$, no alert is generated.

Scenario 2: Alert Generated

- Evidence: 20 MEDIUM threats with confidence 1.0 each → $20 \times 0.5 \times 1.0 = 10.0$
- Evidence: 25 LOW threats with confidence 1.0 each → $25 \times 0.2 \times 1.0 = 5.0$
- Total accumulated threat level = $10.0 + 5.0 = 15.0$
- Since $15.0 \geq 15.0$, an alert is generated.

This demonstrates how multiple evidence items contribute to the accumulated threat level and how the threshold determines alert generation. In practice, confidence scores may vary, reducing or increasing the contribution of each evidence item.

2.4.5.4 Phase 4: Alert Processing & Response

Actions Taken:

1. **Database Storage**: Alert saved with associated evidence IDs.
2. **Blocking Decision**: Checks if IP should be blocked.
   - Validates IP not from our network.
   - Sends blocking request if enabled.
3. **Console Display**: Formatted alert shown in CLI.
4. **Popup Notification**: Desktop alerts if configured.
5. **File Logging**: Alert written to `alerts.log` and `alerts.json`.
6. **Export Streaming**: Alert sent to exporting modules.

2.4.5.5 Phase 5: Logging & Output Management

Dual Logging System:

1. **Human-Readable Logs (`alerts.log`)**:
   - Timestamped entries

- Evidence descriptions
- Alert summaries

2. **Structured Logs (`alerts.json`)**:
   - IDMEFv2 format (standardized security event format)
   - Machine-readable JSON
   - Complete evidence context
   - Accumulated threat levels

Background Logging Architecture:

- **Queue-based**: Evidence/Alert objects queued for processing.
- **Separate Thread**: `EvidenceLogger` handles file I/O independently.
- **Thread-safe**: Prevents main processing pipeline blocking.
- **Graceful Shutdown**: Ensures all queued items are logged before exit.

2.4.5.6 Key Configuration Parameters

Critical Settings in `smartshield.yaml`:

```yaml
detection:
  evidence_detection_threshold: 0.15  # Attacks per minute
  popup_alerts: false                 # Desktop notifications
parameters:
  time_window_width: 3600             # 1-hour time windows
  analysis_direction: all             # Monitor all traffic
modules:
  disable: [template]                 # Disabled modules
DisabledAlerts:
  disabled_detections: []             # Which evidence types to ignore
exporting_alerts:
  export_to: []                       # Export destinations (Slack, STIX, etc.)
```

Threshold Calculation:

```text
detection_threshold_in_this_width =
  evidence_detection_threshold * (time_window_width / 60)
```

Example: $0.15 \times \left(\frac{3600}{60}\right) = 9$ accumulated threat level required.

2.4.5.7 Evidence Lifecycle States

1. **Generated**
   - Created by detection module
   - Published to `evidence_added` channel
   - Stored in database
2. **Processed**
   - Retrieved from channel
   - Marked as "processed" in database
   - Added to threat level accumulation
3. **Whitelisted (Filtered)**
   - Matches whitelist criteria
   - Removed from database
   - Excluded from threat calculation
4. **Aggregated**
   - Combined with other evidence
   - Contributes to accumulated threat level
   - Part of timewindow analysis
5. **Alert-Triggering**
   - Crosses detection threshold
   - Included in alert correlation
   - Referenced in alert object
6. **Archived**
   - Logged to files
   - Available for historical analysis
   - Exportable to external systems

2.4.5.8 Alert vs Evidence Distinction

Evidence:

- Single detection instance.
- Generated by individual modules.
- Threat level: INFO = 0, LOW = 0.2, MEDIUM = 0.5, HIGH = 0.8, CRITICAL = 1.
- Confidence: 0.0-1.0 based on detection certainty.
- Example: "Suspicious user agent detected".

Alert:

- Aggregated security incident.
- Created when threshold crossed.
- Threat level: CRITICAL (always).
- Confidence: 0.0-1.0 based on evidence volume.
- Example: "Multiple evidence types indicate C&C communication".

2.4.5.9 Special Processing Cases

- **Whitelist Filtering**
  - Organization whitelists (Microsoft, Google, etc.)
  - Domain whitelists (Tranco top sites)
  - IP whitelists (trusted networks)
  - MAC whitelists (known devices)
- **Duplicate Prevention**
  - Evidence ID tracking prevents double-counting
  - Timewindow-based aggregation
  - Already-alerted evidence exclusion
- **Blocking Integration**
  - Automatic IP blocking on alert
  - Interface-specific blocking rules
  - Permanent blocking (until manual removal)

2.4.5.10 Output Formats

1. **Console Output**:

```text
[Alert] Timewindow 1 - IP 192.168.1.100:
Accumulated Threat Level: 15.8 (Threshold: 15.0)
Evidence:
• Suspicious User Agent (Threat: HIGH, Confidence: 1.0)
• Connection Without DNS (Threat: MEDIUM, Confidence: 0.8)
• Data Upload >100MB (Threat: HIGH, Confidence: 0.9)
```

2.4.5.11 Operational Workflow

For Security Analysts:

1. **Real-time Monitoring**: Watch console for red alerts.
2. **Evidence Review**: Check `alerts.log` for detailed evidence.
3. **Context Analysis**: Use `alerts.json` for machine processing.

4. **Response Actions**: Block IPs, investigate further, update whitelists.

For System Integrators:

1. **API Integration**: Consume `alerts.json` in SIEM systems.
2. **Automation**: Trigger workflows based on evidence types.
3. **Reporting**: Generate security reports from structured logs.
4. **Archiving**: Store historical evidence for compliance.

This comprehensive pipeline ensures that SmartShield transforms raw network data into actionable security intelligence with proper context, prioritization, and response capabilities.

2.4.6 Output

2.4.6.1 Web Interface

The Web-Interface provides a browser-based GUI for SmartShield. It offers a more accessible visual experience compared to the terminal, allowing users to point-and-click through data.

- Language: Python
- Backend Framework: Flask
- Frontend Libraries: Bootstrap 5, jQuery, DataTables, and FontAwesome.
- Entry Point: `webinterface.sh` or the `-w` flag in `smartshield.py`
- Access: served at `http://localhost:55000/`
- Username: Set in `SMARTSHIELD_WEB_USER` environment variable (default: `admin`)
- Password: Set in `SMARTSHIELD_WEB_PASSWORD` (default shows warning `must be changed`)
- Session: 8-hour sessions. Expires automatically. All routes require login.
- Port change: Edit `web_interface.port` in `smartshield.yaml`, then restart.

How it works?

When a user accesses the interface in a browser, the backend queries the Redis database and renders HTML templates populated with the traffic data. Failed login attempts add a 0.5-second delay to slow automated attacks.

Data Acquisition

- Flow Visualization: It reads the `timeline` keys to render flows.
- Alerts: It queries the `evidence` keys to show red flags on compromised profiles.
- Traffic Stats: It calculates stats (bytes sent/received) by aggregating flow data stored in profile hashes.

Restrict access to port 55000

The web interface should only be accessible from management VLAN. Add a firewall rule to restrict access:

```
sudo ufw allow from 192.168.20.0/24 to any port 55000
sudo ufw deny 55000
```

Never expose port 55000 to the internet. Use a VPN if remote access is needed.

2.4.6.2 Terminal User Interface (TUI)

It is designed to provide security analysts with a fast, keyboard-driven way to navigate through network traffic profiles, time windows, and evidence without leaving the command line.

- Language: JS JavaScript (Node.js)
- Libraries: `blessed` and `blessed-contrib` (for rendering TUI widgets like boxes, trees, and tables), `redis` (for database connectivity).
- Entry Point: `TUI.js` (executes the main screen initialization).

How it works?

TUI initializes a main `Screen` object which acts as a canvas. It renders a grid of `Widgets`, each responsible for a specific view of the data. It operates on an event loop that listens for keyboard inputs to switch focus or views, and it periodically polls the database for updates.

Data Acquisition

TUI connects directly to the SmartShield Redis Database It does not process PCAP files or sniff interfaces directly. Instead, it queries specific Redis Keys populated by the core engine

2.5 Usage

SmartShield can read the packets directly from the network interface of the host machine, and packets and flows from different types of files, including

- Pcap files
- Packets directly from a network interface
- Suricata flows (from JSON files created by Suricata, such as eve.json)
- Argus flows (CSV file separated by commas or TABs)
- Zeek flows from a Zeek folder with log files
- Nfdump (NetFlow dump) flows from a binary `nfdump` file
- Text flows from stdin in zeek, argus or suricata form

All the input flows are converted to a normalized internal format. So once read, SmartShield works the same with all of them. See Appendix (A): SmartShield Network Input Formats for comparison between the different input types.

After SmartShield runs on the given traffic, the output can be analyzed with Terminal User Interface (TUI) or Web Interface.

2.5.1 SmartShield Server Placement

The SmartShield server MUST be connected to the network via two separate interfaces:

1. `eth0` (management): SSH access, web UI on port 55000, syslog output. Connected to IT/management VLAN.
2. `eth1` (monitoring): Connected to a SPAN port or TAP at the IT/OT boundary. Receives a mirror of all traffic.
   - This interface does NOT need an IP address.

> ⚠ **Warning**
>
> **NEVER** use your management interface for traffic capture; it will flood the interface and make SSH unusable.

2.5.2 Pre-Installation: Switch SPAN Port Setup

Before installing SmartShield, configure your managed switch to mirror the IT/OT boundary port to the server's monitoring interface. This is done in the switch management interface — the exact steps depend on the switch vendor.

Switch Vendor	Source Command	Destination / Notes
Cisco IOS / Catalyst	`monitor session 1 source interface Gi1/0/1`	`monitor session 1 destination interface Gi1/0/24`
Juniper EX	`set ethernet-switching-options analyzer SPAN input ingress interface ge-0/0/1.0`	`set analyzer SPAN output interface ge-0/0/24.0`
HP / Aruba ProCurve	`mirror-session 1 port A1 mirror-port A24`	`Mirror port A24 configured as destination`
Siemens SCALANCE	Port Mirroring under Diagnostics menu	`via web UI: Diagnostics → Mirror`
Generic / Other	Look for: Port Mirroring, SPAN, Port Monitor, Traffic Mirror	`Source = IT/OT boundary uplink, Destination = SmartShield eth1`

Verify SPAN is working before installing SmartShield

After configuring the SPAN port, SSH into the future SmartShield server and run:

```bash
sudo ip link set eth1 up
sudo tcpdump -i eth1 -n -c 50
```

You should see traffic from your OT devices. If the interface is silent, the SPAN port is not configured correctly.

2.5.3 Option A: Bare-Metal Install (Recommended for Production)

The included install script handles everything on a fresh Ubuntu server. Run it as root:

```bash
# Step 1 - Copy the project to the factory server
scp -r SmartShield-Factory/ admin@<server-ip>:/opt/
# Step 2 - SSH into the server
ssh admin@<server-ip>
# Step 3 - Run the installer as root
cd /opt/SmartShield-Factory
sudo chmod +x install/install_factory.sh
sudo ./install/install_factory.sh
# The script will:
```

```
#- Install Zeek 8, Python 3, Redis, iptables-persistent
#- Ask for the monitoring interface name (e.g. eth1)
#- Ask for web UI username and password (min 12 chars)
#- Copy config to /etc/smartshield/
#- Create /var/lib/smartshield/ for persistent Redis data
#- Install and enable the smartshield systemd service
#- Optionally start SmartShield immediately
After installation — edit OT config files:
# Add your factory's PLC/HMI/SCADA IPs (CRITICAL — do before starting)
sudo nano /opt/SmartShield-Factory/config/ot_protected_devices.conf
# Add authorized Modbus master IPs
sudo nano /opt/SmartShield-Factory/config/
modbus_authorized_masters.conf
# Start the service
sudo systemctl start smartshield
sudo systemctl status smartshield
```

After installation — edit OT config files:

```bash
# Add your factory's PLC/HMI/SCADA IPs (CRITICAL — do before starting)
sudo nano /opt/SmartShield-Factory/config/ot_protected_devices.conf
# Add authorized Modbus master IPs
sudo nano /opt/SmartShield-Factory/config/
modbus_authorized_masters.conf
# Start the service
sudo systemctl start smartshield
sudo systemctl status smartshield
```

2.5.4 Option B: Docker (Lab / Pre-Production Testing)

Use Docker if you want to test before deploying bare-metal, or if you are running in a virtualized environment.

```bash
# Step 1 — Create the .env file
```

```
cat > docker/.env << 'ENVEOF'
SMARTSHIELD_INTERFACE=eth1
SMARTSHIELD_WEB_USER=admin
SMARTSHIELD_WEB_PASSWORD=YourStrongFactoryPassword!
SMARTSHIELD_SECRET_KEY=$(python3 -c "import secrets;
print(secrets.token_hex(32))")
ENVEOF
# Step 2 - Create persistent storage directories on the host
sudo mkdir -p /var/lib/smartshield/{redis,output}
# Step 3 - Build the Docker image
docker build -f docker/Dockerfile -t smartshield_gp:latest .
# Step 4 - Start all services
docker compose -f docker/docker-compose.factory.yml up -d
# Step 5 - Check status
docker compose -f docker/docker-compose.factory.yml ps
docker logs smartshield -f
```

> **Docker vs Bare-Metal**
>
> Docker is easier to set up but has limitations in a factory:
>
> - `iptables` blocking requires `network_mode`: host which reduces container isolation
> - Live interface capture with `NET_RAW` needs the host interface to be UP before starting
> - Redis volumes must be bind-mounted to a host path (`/var/lib/smartshield/redis`) for true persistence
>
> Bare-metal is recommended for production deployments where stability is critical.

2.5.5 Starting for the First Time — Recommended Sequence

1. **Populate OT config files**: Add all PLC, HMI, and SCADA IPs to `ot_protected_devices.conf` and all authorized Modbus master IPs to `modbus_authorized_masters.conf`. This is mandatory before enabling blocking.

2. **Start in monitor-only mode first**: Do not enable `--blocking` on the first run. Let SmartShield observe for 1–2 weeks to establish a normal baseline. Watch for false positives.

3. **Verify SPAN capture**: Run: `sudo tcpdump -i eth1 -n port 502 or port 102 -c 20` to confirm OT traffic is arriving on the monitoring interface.

4. **Check the web interface**: Open `http://<server-ip>:55000` and log in. You should see profiles appearing within a few minutes of capture starting.

5. **Tune thresholds**: Review alerts after 1 week. Adjust `modbus_flood_threshold` and `s7_job_flood_threshold` in `smartshield.yaml` based on your factory's normal polling rates.

6. **Enable blocking**: Add `--blocking` to the start command in the systemd service or Docker Compose configuration, then restart. Verify no OT device IPs appear in the iptables rules:

```bash
sudo iptables -L smartshieldBlocking -n
```

7. **Configure SIEM export**: Add `syslog` to `export_to` and set `syslog_server`, then restart. Verify alerts arrive in your SIEM.

2.5.6 Modes

Smart Shield has 2 modes, interactive and daemonized.

- **Interactive (Default)** : For viewing output, logs and alerts in your terminal.
- **Daemonized** : Smart Shield runs as a daemon in the background. which means all output, logs and alerts are written in files and nothing is displayed in the terminal. Only one instance of the daemon can run at a time unless ran with `-m` (see (2.5.7): Multiple Instances)

Daemonized Mode

```
./smartshield.py  -D | --daemon
```

2.5.7 Multiple Instances

If multiple instances of SmartShield need to be run (e.g, in case of multiple network interfaces,) you can specify `-m` flag:

Multiple SmartShield Instances

```
./smartshield.py  -m
```

2.5.8 Reading Output

The output process collects output from the modules and handles the display of information on screen. Currently, Smartshield' analysis and detected malicious behaviour can be analyzed as following:

- Web Interface: GUI for vewing Smartshield detections, Incoming and ongoing traffic and an organized timeline of flows.
- Terminal User Interface (TUI): It displays SmartShield detection and analysis in colorful table and graphs, highlighting important detections
- `alerts.json` and `alerts.log` in the output folder: collects all evidences and detections generated by SmartShield.

2.5.9 Web Interface

Run using

```
./webinterface.sh
```

or

```
./smartshield.py -w
```

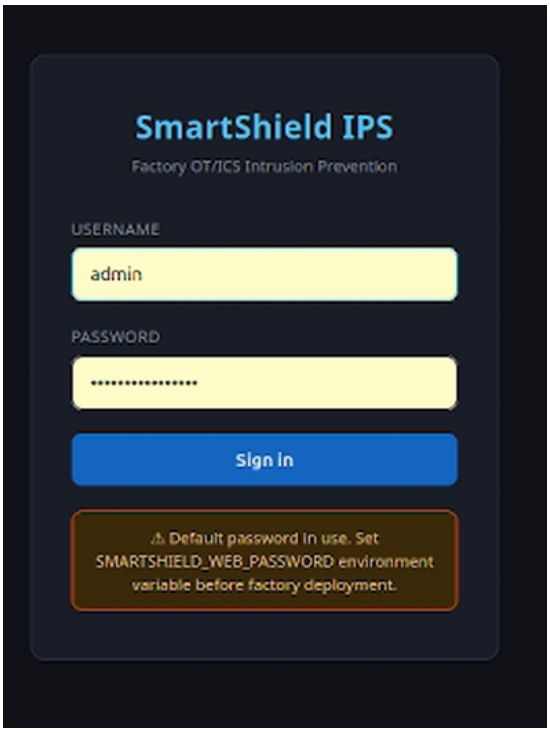

Figure 2.9: Login Page

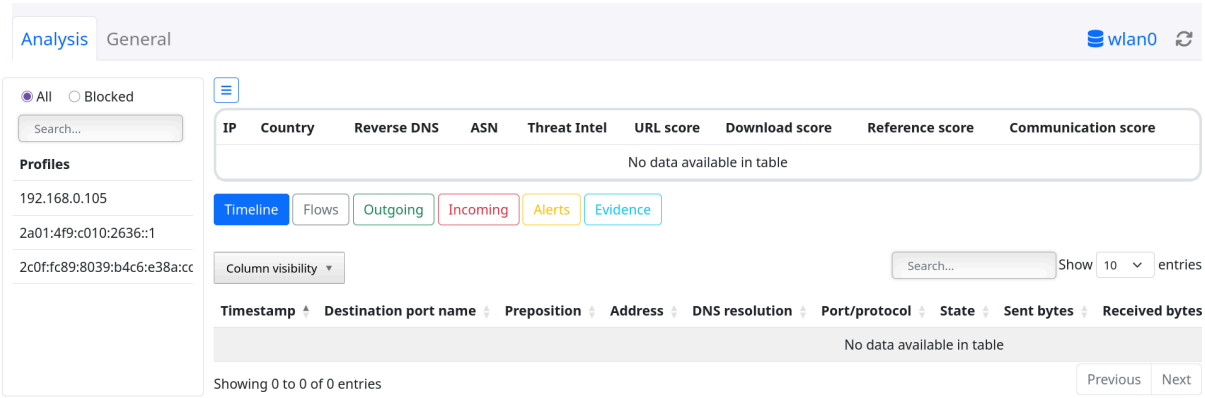

Figure 2.10: Web Interface

On the right column, you can see a list of all the IPs seen in your traffic. The traffic of IP is splitted into time windows. each time window is 1h long of traffic by default (configurable). IPs and timewindows that are marked in red are considered malicious in SmartShield. You can view the traffic of each time window by clicking on it

- The timeline button shows the zeek logs formatted into a human readable format
- The flows button shows the raw flows as seen in the input file, or by zeek in case of running smartshield on a PCAP or on your interface
- The outgoing button shows flow sent from this IP/profile to other IPs only. it doesn't show traffic sent to the profile
- The incoming button shows flow sent to this IP/profile to other IPs only. It doesn't show traffic sent from the profile.
- The Alerts button shows the alerts smartshield saw for this IP, each alert is a bunch of evidence that the given profile is malicious. smartshield decides to block the IP if an alert is generated for it (if the blocking module is enabled). Clicking on each alert expands the evidence that resulted in the alert.
- The Evidence button shows all the evidence of the timewindow whether they were part of an alert or not.

2.5.10 Terminal User Interface (TUI)

In docker, you can open a new terminal inside the smartshield container:

```
docker ps                       #find the container id of smartshield
docker exec -it <container_id> bash
```

and execute:

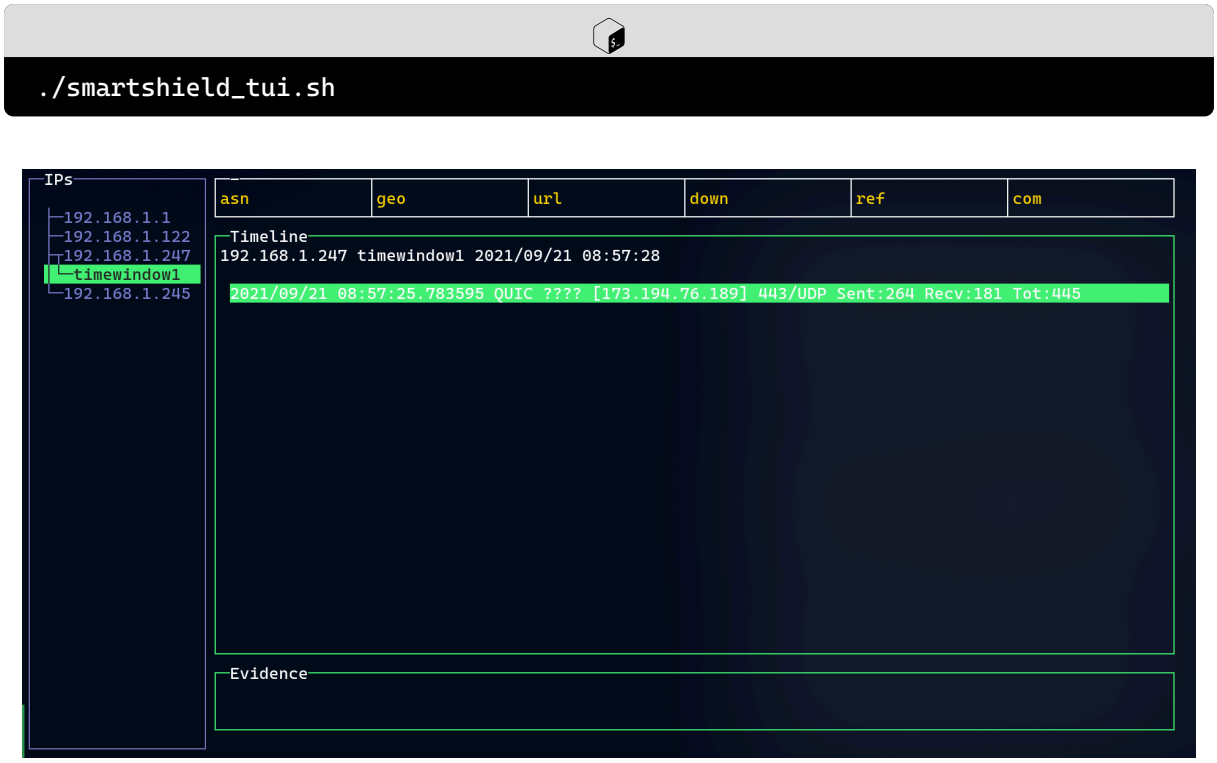

Figure 2.11: Terminal User Interface

3

Security Testing Framework

3.1 Threat Intelligence Module Tests

```bash
# 1. Feed poisoning attack
curl -X POST http://ti-update-server/malicious-feed.csv
# Upload malicious TI feed with whitelisted attacker IPs

# 2. Local TI file injection
echo "8.8.8.8,low,['google_dns']" >> config/local_data_files/own_malicious_iocs.csv
# Inject Google DNS as "malicious" to cause self-DoS

# 3. Cache poisoning via Redis
redis-cli -h localhost -p 6379 SET "TI:domain:google.com" "malicious"
```

3.2 PortScan Module Evasion

```python
# Slow/low-rate scanning to avoid thresholds
import time
import socket
import random

def stealth_scan(target_ip, ports):
    for port in ports:
        # Random delays between scans
        time.sleep(random.uniform(5, 30))

        # Use different source ports
        src_port = random.randint(1024, 65535)

        # Mix successful/unsuccessful connections
        if random.random() > 0.7:
            # Establish real connection to look legitimate
            establish_legitimate_connection(target_ip, port)
        else:
```

```
            # Quick SYN scan
            send_syn(target_ip, port, src_port)
```

3.3 RNN C&C Detection Bypass

```python
# Manipulate letter generation
def generate_evasive_letters():
    """
    SmartShield expects patterns like: H*y*H*y*H*y
    Where H=high traffic, y=yes (established), etc.

    Evasion techniques:
    1. Insert random '0' (no data) letters
    2. Mix capital/lowercase variations
    3. Vary timing beyond training thresholds
    4. Use legitimate protocol patterns
    """
    legitimate_patterns = [
        "H*y*H*y*H*y",      # Normal browsing
        "y*H*y*0*y*H",      # Intermittent traffic
        "h*Y*h*Y*h*Y",      # Case variation
    ]
    return random.choice(legitimate_patterns)
```

3.4 Container/Docker Security Tests

```bash
# Docker escape attempts
# 1. Privilege escalation in container
docker run --privileged smartshield:latest
# 2. Host mount escape
docker run -v /:/host smartshield:latest
# 3. Socket escape
docker run -v /var/run/docker.sock:/var/run/docker.sock
# 4. Capabilities testing
```

```
docker run --cap-add=NET_ADMIN --cap-add=SYS_ADMIN
```

3.5 Web Interface Security Tests

python

```python
import requests

class WebInterfaceTester:
    def __init__(self, base_url="http://localhost:55000"):
        self.base_url = base_url

    def test_auth_bypass(self):
        """Test for unauthenticated endpoints"""
        endpoints = [
            "/api/alerts",
            "/api/evidence",
            "/api/block",
            "/api/whitelist"
        ]

        for endpoint in endpoints:
            response = requests.get(f"{self.base_url}{endpoint}")
            if response.status_code == 200:
                print(f"UNAUTHENTICATED ACCESS: {endpoint}")

    def test_xss_injection(self):
        """Test alert description fields for XSS"""
        payloads = [
            "<script>alert('XSS')</script>",
            "javascript:alert(1)",
            "onerror=alert`1`"
        ]

        # Try injecting via whitelist
```

```
        for payload in payloads:
            data = {
                "ip": f"192.168.1.100{payload}",
                "action": "whitelist"
            }
            requests.post(f"{self.base_url}/api/whitelist", json=data)
```

3.6 Evidence Processing Attack Vectors

python

```python
# Race condition in evidence processing
import threading
import time
import json
import random

def evidence_flood_attack(db_connection):
    """Flood evidence channel to cause:
    1. Memory exhaustion
    2. Race conditions in alert aggregation
    3. Threshold bypass via rapid evidence
    """

    def flood_evidence():
        for i in range(10000):
            evidence = {
                "attacker": f"10.0.0.{random.randint(1,255)}",
                "victim": "192.168.1.100",
                "evidence_type": "PORT_SCAN",
                "threat_level": "HIGH",
                "confidence": 0.9,
                "timestamp": time.time()
            }
```

```
                db_connection.publish('evidence_added',
                json.dumps(evidence))

    # Start multiple flooding threads
    threads = []
    for _ in range(10):
        t = threading.Thread(target=flood_evidence)
        threads.append(t)
        t.start()
```

3.7 OT Denial of Service (DoS) Testing

The following section defines and evaluates a test matrix of Operational Technology (OT) Denial-of-Service attacks. In Semester 1 this matrix was a *plan* expressed as pseudo-code (3.7.1): Test Plan. In Semester 2, with the OT Detection module implemented, the flood-class attacks were validated empirically against real packet captures, and the remaining categories were assessed against the implemented detection logic. This section reports those results honestly: what was tested and detected, what is covered by the detection design but not yet empirically validated, and what is outside the current detection scope.

3.7.1 Test Plan

The following pseudo-code outlines the planned tests for OT environments, focusing on protocol-specific, application-layer, resource-exhaustion, state-manipulation, physical-process, network-infrastructure, and time-based DoS attacks.

```c
// 1. PROTOCOL-SPECIFIC DoS ATTACKS

FUNCTION test_protocol_dos():
    PRINT "[*] Testing Protocol-Specific DoS Attacks"

    // Attack 1: ModBUS Flood Attack
    PRINT "\n[1] ModBUS Connection Flood"
    OPEN 1000_TCP_connections_to(TARGET_PLC, port=502)
    ON_EACH connection:
        SEND modbus_request = {
            function: "READ_HOLDING_REGISTERS",
            address: RANDOM(),
            count: 125  // Max allowed
```

```
        }
        KEEP connection_open()  // Don't close
    // PLC runs out of connection slots
    CHECK IF PLC_becomes_unresponsive
    CHECK SmartShield_detects("MODBUS_FLOOD")

    // Attack 2: ModBUS Illegal PDU Flood
    PRINT "\n[2] ModBUS Illegal PDU Flood"
    FOR i FROM 1 TO 10000:
        SEND modbus_packet = {
            transaction_id: i,
            protocol_id: 0,
            length: 2,  // Too short
            unit_id: 1,
            pdu: [0xFF]  // Invalid
        }
        NO_WAIT  // Send as fast as possible
    // PLC spends CPU parsing malformed packets
    CHECK IF PLC_CPU_spikes
    CHECK SmartShield_detects("PROTOCOL_FLOOD")

    // Attack 3: S7COMM Job Flood
    PRINT "\n[3] Siemens S7 Job Flood"
    FOR i FROM 1 TO 500:
        SEND s7comm_packet = {
            rosctr: 1,  // Job request
            function: "READ_VARIABLE",
            items: 100  // Request 100 variables each
        }
    // PLC overwhelmed with job processing
    CHECK IF legitimate_commands_fail
    CHECK SmartShield_detects("S7_JOB_FLOOD")

    // Attack 4: DNP3 Fragment Flood
    PRINT "\n[4] DNP3 Fragment Flood Attack"
    // Break legitimate packets into fragments
    legitimate_packet = GET_legitimate_dnp3_packet()
    fragments = SPLIT_INTO_100_fragments(legitimate_packet)

    FOR EACH fragment IN random_order(fragments):
        SEND fragment_with_wrong_sequence()
        WAIT random(10, 100) ms
    // PLC spends resources reassembling
    CHECK IF communication_disrupted
```

```
58      CHECK SmartShield_detects("FRAGMENT_FLOOD")
59
60  // 2. APPLICATION LAYER DoS ATTACKS
61
62  FUNCTION test_application_dos():
63      PRINT "\n[*] Testing Application Layer DoS"
64
65      // Attack 1: OPC UA Memory Exhaustion
66      PRINT "\n[1] OPC UA Memory Exhaustion"
67      // OPC UA allows large data reads
68      SEND opcua_request = {
69          request_type: "BROWSE",
70          max_results: 999999,  // Extremely large
71          nodes_to_browse: ["/", "/Objects", "/Types"]
72      }
73      // Server allocates huge result buffers
74      REPEAT 100_times  // Multiple connections
75      CHECK IF server_memory_exhausted
76      CHECK SmartShield_detects("OPCUA_MEMORY_DOS")
77
78      // Attack 2: Historian Query Flood
79      PRINT "\n[2] Historian Database Query Flood"
80      // Most OT systems have data historians
81      FOR i FROM 1 TO 1000:
82          SEND sql_query = "SELECT * FROM process_data "
                             + "WHERE timestamp > DATEADD(day, -30,
83                           GETDATE())"
84          // Expensive query - last 30 days of data
85      // Database CPU hits 100%
86      CHECK IF historian_becomes_slow
87      CHECK SmartShield_detects("HISTORIAN_QUERY_FLOOD")
88
89      // Attack 3: HMI Screen Update Flood
90      PRINT "\n[3] HMI Screen Update Attack"
91      // HMIs update screens based on tag changes
92      CREATE 1000_fake_tags()
93      FOR EACH tag:
94          CHANGE tag_value_rapidly(1000_times_per_second)
95      // HMI tries to update all screens constantly
96      CHECK IF HMI_becomes_unresponsive
97      CHECK SmartShield_detects("HMI_UPDATE_FLOOD")
98
99      // Attack 4: Alarm Flood Attack
```

```
    PRINT "\n[4] Alarm/Event Flood"
    // Generate thousands of fake alarms
    FOR i FROM 1 TO 10000:
        SEND alarm_event = {
            severity: "HIGH",
            message: "CRITICAL_FAILURE_" + RANDOM_STRING(),
            timestamp: CURRENT_TIME
        }
    // Operators can't see real alarms in noise
    CHECK IF alarm_system_overwhelmed
    CHECK SmartShield_detects("ALARM_FLOOD")

// 3. RESOURCE EXHAUSTION ATTACKS

FUNCTION test_resource_exhaustion():
    PRINT "\n[*] Testing Resource Exhaustion Attacks"

    // Attack 1: PLC Memory Exhaustion
    PRINT "\n[1] PLC Program Memory Exhaustion"
    // Try to upload huge programs
    huge_program = GENERATE_large_program(10_MB)
    ATTEMPT upload_to_plc(huge_program)
    // PLC has limited memory (often 1-16MB)
    CHECK IF plc_rejects_legitimate_programs
    CHECK SmartShield_detects("MEMORY_EXHAUSTION")

    // Attack 2: Connection Table Exhaustion
    PRINT "\n[2] Connection Table Flood"
    // PLCs have limited connection slots
    FOR i FROM 1 TO 256:  // Typical PLC limit
        SPOOF source_ip = "192.168.1." + (150 + i)
        OPEN connection_to_plc()
        SEND keepalive_packets()
    // No more legitimate connections possible
    CHECK IF legitimate_hmi_cannot_connect
    CHECK SmartShield_detects("CONNECTION_FLOOD")

    // Attack 3: Bandwidth Consumption
    PRINT "\n[3] OT Network Bandwidth Flood"
    // OT networks often have low bandwidth (10-100Mbps)
    GENERATE high_bandwidth_traffic(90_Mbps)
    // Legitimate commands get dropped
    CHECK IF control_packets_lost
    CHECK SmartShield_detects("BANDWIDTH_FLOOD")
```

```
144
145     // Attack 4: CPU Exhaustion via Complex Calculations
146     PRINT "\n[4] PLC CPU Exhaustion"
147     // Upload program with infinite loops
148     malicious_program = "
149         WHILE TRUE:
150             PERFORM complex_math_calculations()
151             WRITE results_to_memory()
152     "
153     IF successful_upload:
154         // PLC CPU hits 100%
155         CHECK IF scan_time_exceeds_limit
156         CHECK SmartShield_detects("CPU_EXHAUSTION")
157
158 // 4. STATE-MANIPULATION DoS ATTACKS
159
160 FUNCTION test_state_manipulation_dos():
161     PRINT "\n[*] Testing State Manipulation DoS"
162
163     // Attack 1: PLC Mode Switching DoS
164     PRINT "\n[1] Rapid PLC Mode Switching"
165     // Many PLCs can't handle rapid mode changes
166     FOR i FROM 1 TO 100:
167         SEND "SWITCH_TO_PROGRAM_MODE"
168         WAIT 50 ms
169         SEND "SWITCH_TO_RUN_MODE"
170         WAIT 50 ms
171     // PLC gets confused, may crash
172     CHECK IF plc_enters_fault_state
173     CHECK SmartShield_detects("MODE_SWITCHING_DOS")
174
175     // Attack 2: Watchdog Timer Attack
176     PRINT "\n[2] Watchdog Timer Manipulation"
177     // PLCs have watchdog timers for safety
178     UPLOAD program_that_disables_watchdog()
179     OR: CONSTANTLY reset_watchdog_preventing_timeout()
180     // Safety features disabled
181     CHECK IF safety_system_bypassed
182     CHECK SmartShield_detects("WATCHDOG_MANIPULATION")
183
184     // Attack 3: I/O Module Disable
185     PRINT "\n[3] I/O Module Communication Disrupt"
186     // Target communication to I/O modules
187     SEND malformed_profinet_packets_to_io_module()
```

```
188     // I/O module stops responding
189     CHECK IF sensors/actuators_disconnected
190     CHECK SmartShield_detects("IO_COMMUNICATION_DOS")
191
192     // Attack 4: Configuration Corruption
193     PRINT "\n[4] Configuration Corruption Attack"
194     // Upload corrupt configuration
195     corrupt_config = TAKE_valid_config() + ADD_corrupt_data()
196     UPLOAD_TO_device(corrupt_config)
197     // Device may crash on reboot
198     CHECK IF device_fails_to_start
199     CHECK SmartShield_detects("CONFIG_CORRUPTION")
200
201 // 5. PHYSICAL PROCESS DoS ATTACKS
202
203 FUNCTION test_physical_process_dos():
204     PRINT "\n[*] Testing Physical Process DoS Attacks"
205
206     // Attack 1: Valve Cycling Attack
207     PRINT "\n[1] Rapid Valve Cycling"
208     // Rapid open/close commands
209     FOR i FROM 1 TO 1000:
210         SEND "OPEN_VALVE_101"
211         WAIT 100 ms
212         SEND "CLOSE_VALVE_101"
213         WAIT 100 ms
214     // Valve mechanical wear or motor burnout
215     CHECK IF valve_fails_physically
216     CHECK SmartShield_detects("VALVE_CYCLING_ATTACK")
217
218     // Attack 2: Motor Start/Stop Cycling
219     PRINT "\n[2] Motor Start-Stop Cycling"
220     // Electric motors can't handle rapid starts
221     FOR i FROM 1 TO 50:
222         SEND "START_MOTOR_5"
223         WAIT 1 second
224         SEND "STOP_MOTOR_5"
225         WAIT 1 second
226     // Motor overheats, breaker trips
227     CHECK IF motor_fails_or_trips
228     CHECK SmartShield_detects("MOTOR_CYCLING_ATTACK")
229
230     // Attack 3: Heater Overdrive
231     PRINT "\n[3] Heater Overdrive Attack"
```

```
232     // Turn heater to 100%, disable overtemp protection
233     SEND "SET_HEATER_POWER = 100%"
234     SEND "DISABLE_OVERTEMP_ALARM"
235     WAIT until temperature_exceeds_limits
236     // Thermal cutoff or fire risk
237     CHECK SmartShield_detects("HEATER_OVERRIDE_ATTACK")
238
239     // Attack 4: Tank Overflow/Underflow
240     PRINT "\n[4] Tank Level Manipulation"
241     // Fake level sensor readings
242     WHILE tank_is_filling:
243         SEND fake_sensor_reading = "LEVEL_LOW"
244         // Controller keeps filling
245     WAIT until tank_overflows_physically
246     CHECK SmartShield_detects("LEVEL_SENSOR_DOS")
247
248 // 6. NETWORK INFRASTRUCTURE DoS ATTACKS
249
250 FUNCTION test_network_infrastructure_dos():
251     PRINT "\n[*] Testing Network Infrastructure DoS"
252
253     // Attack 1: OT Switch Flood
254     PRINT "\n[1] Industrial Switch MAC Flood"
255     // Flood switch with fake MACs
256     FOR i FROM 1 TO 10000:
257         SEND packet_with_random_source_mac()
258         DESTINATION = BROADCAST
259     // Switch CAM table full, floods all ports
260     CHECK IF switch_becomes_hub
261     CHECK SmartShield_detects("SWITCH_MAC_FLOOD")
262
263     // Attack 2: Spanning Tree Attack
264     PRINT "\n[2] Spanning Tree Protocol Attack"
265     SEND spoofed_bpdu = {
266         root_bridge: ATTACKER_MAC,
267         priority: 0  // Best priority
268     }
269     // Network reconvergence causes outages
270     CHECK IF network_goes_down_temporarily
271     CHECK SmartShield_detects("STP_ATTACK")
272
273     // Attack 3: PROFINET DCP Flood
274     PRINT "\n[3] PROFINET DCP Discovery Flood"
275     PROFINET_devices_use_DCP_for_discovery
```

```
    SEND 1000_DCP_identify_requests_per_second
    // Devices overwhelmed with discovery requests
    CHECK IF profinet_devices_unresponsive
    CHECK SmartShield_detects("PROFINET_DCP_FLOOD")

    // Attack 4: CIP Explicit Messaging Flood
    PRINT "\n[4] CIP Explicit Message Flood"
    // Rockwell Ethernet/IP uses CIP
    SEND cip_packet = {
        service: "GET_ATTRIBUTE_ALL",
        class: 0x01,  // Identity Object
        instance: 1
    }
    REPEAT 1000_times_with_different_instances
    CHECK IF device_stops_responding
    CHECK SmartShield_detects("CIP_MESSAGE_FLOOD")

// 7. TIME-BASED DoS ATTACKS

FUNCTION test_time_based_dos():
    PRINT "\n[*] Testing Time-Based DoS Attacks"

    // Attack 1: NTP/Time Sync Attack
    PRINT "\n[1] Time Synchronization Attack"
    // Many OT systems sync time via NTP
    SPOOF ntp_server = REAL_NTP_SERVER_IP
    SEND ntp_response = {
        current_time: "2099-01-01 00:00:00"
    }
    // Systems confused by time jump
    CHECK IF historians_logs_corrupted
    CHECK SmartShield_detects("TIME_SYNC_ATTACK")

    // Attack 2: PTP (Precision Time Protocol) Attack
    PRINT "\n[2] PTP Grandmaster Spoofing"
    // IEEE 1588 used in power grids, manufacturing
    SPOOF ptp_grandmaster_clock()
    SEND wrong_timing_messages()
    // Synchronization lost in precise systems
    CHECK IF synchronized_processes_fail
    CHECK SmartShield_detects("PTP_ATTACK")

    // Attack 3: Scan Cycle Disruption
    PRINT "\n[3] PLC Scan Cycle Disruption"
```

```
320     // PLCs have fixed scan cycles (e.g., 10ms)
321     INTERRUPT scan_cycle_with_high_priority_packets()
322     // Scan overruns occur
323     CHECK IF plc_enters_fault_mode
324     CHECK SmartShield_detects("SCAN_CYCLE_ATTACK")
```

3.7.2 Test Methodology and Environment

Each test capture was replayed through SmartShield offline:

```bash
python3 smartshield.py -f <capture.pcap> -o output/<test>
```

Zeek parses the capture into `conn.log`, the profiler publishes each flow on the `new_flow` channel, and the OT Detection module evaluates every flow whose destination port is an OT service port (502, 102, 20000, 34964, 44818, 123, 319). The detection thresholds used (from `config/smartshield.yaml`, factory profile) are:

Parameter	Value
`modbus_flood_threshold`	20 connections / 60 s window
`dnp3_flood_threshold`	100 requests / 60 s window
`s7_job_flood_threshold`	50 connections / 60 s window
Single-connection volume floor	≥ 200 packets in one flow
Single-connection rate floor	≥ 50 packets / second
`evidence_detection_threshold`	0.15

3.7.3 Empirical Results

Three captures were used. Their measured traffic characteristics and the resulting detections are summarised below.

#	Attack (capture)	Measured traffic	Evidence produced	Detection mechanism
1	Modbus connection / request flood (`Zero-SWARM Flooding`)	`192.168.88.201` → `192.168.88.100:502`, **one** TCP connection, **61,062**	`OT_MODBUS_CONNECTION_FLOOD` (HIGH, 0.9)	Single-connection volume/rate detector

#	Attack (capture)	Measured traffic	Evidence produced	Detection mechanism
		packets, ~595 s, sustained **~102 pkts/s**		
2	Modbus scan + flood (`ZeroSWARM Nmap`)	`192.168.88.201` → `192.168.88.100:502`, single connection, **~37,000 packets**	`OT_MODBUS_CONNECTION_FLOOD` (HIGH)	Single-connection volume/rate detector
3	DNP3 request flood (`dnp3dataset_capture`)	`192.168.0.198` → `:20000`, **~539 TCP connections**	`OT_DNP3_FLOOD` (HIGH)	Connection-count sliding-window counter

In all three cases the resulting evidence carried a HIGH threat level and a confidence of 0.85–0.90, which exceeds the factory `evidence_detection_threshold` of 0.15 and therefore raises an Alert in the SmartShield pipeline (visible in the web interface and written to `alerts.json`).

3.7.4 Key Finding — Connection-Count vs Single-Connection Floods

The most significant testing result concerns the *shape* of a real flood.

The original Semester 1 design counted **distinct TCP connections** per source to an OT port (default 20 connections / 60 s). Testing revealed that the captured Modbus flood does **not** match this shape: the entire attack rode a **single** long-lived TCP connection carrying 61,062 packets (35,618 outbound + 25,444 in reply) at a sustained ~102 packets/second. Zeek logs such a connection as **one** `conn.log` entry, so the connection-count detector observed a count of 1 — far below the threshold of 20 — and produced no evidence.

This is a true positive that the connection-count detector structurally cannot catch. To address it, a **single-connection volume/rate detector** was added: any flow to

an OT service port that carries at least 200 packets *and* sustains at least 50 packets/second is flagged as a flood. This complements, rather than replaces, the connection-count detector:

- many short connections (e.g. the DNP3 capture, 539 connections) → caught by the connection-count counter;
- one connection with very high packet volume/rate (the Modbus capture) → caught by the volume/rate detector.

The thresholds were chosen to separate an attack from legitimate sustained polling: a normal SCADA poll runs at a few packets/second, whereas the captured flood ran at ~102 packets/second; a clear, two-order-of-magnitude separation.

3.7.5 Detection Coverage and Limitations

For transparency, the full Semester 1 OT DoS test matrix is mapped here against the current detection capability.

Detected (validated against captures):

- Modbus connection / request flood (port 502): `OT_MODBUS_CONNECTION_FLOOD`
- DNP3 request flood (port 20000): `OT_DNP3_FLOOD`
- Modbus scan / flood hybrid (port 502): `OT_MODBUS_CONNECTION_FLOOD`

Covered by the detection design, not yet empirically captured:

Flood-class attacks on the remaining OT ports; S7Comm job flood (102), PROFINET DCP flood (34964), CIP explicit-message flood (44818), and the NTP/PTP time-sync floods (123/319); are handled by the same connection-count and volume/rate logic, but no representative capture was available to validate them during this phase. They are expected to behave identically to the Modbus and DNP3 cases.

Requires the Zeek OT protocol parser (current limitation):

The application-semantic and physical-process detections: `OT_MODBUS_WRITE_COILS`, `OT_MODBUS_ILLEGAL_FUNCTION`, `OT_S7COMM_PLC_STOP`, `OT_PLC_MODE_SWITCH`, `OT_WATCHDOG_MANIPULATION`, `OT_VALVE_CYCLING`, `OT_MOTOR_CYCLING`: depend on per-PDU events published by `zeek-scripts/ot_protocols.zeek` (Modbus function codes, S7 ROSCTR, DNP3 function codes, and custom OT notices). At the time of testing this parser was disabled because the `@load policy/protocols/modbus` / `dnp3` directives abort on Zeek builds that lack those policy scripts. Wiring the Zeek OT

parser to the OT module's `new_modbus` / `new_s7comm` / `new_dnp3` channels: so that these per-message detections fire: is the principal remaining task for these attack classes.

Outside the current detection scope:

Several attacks in the Semester 1 plan target hosts and protocols that SmartShield does not model at the application layer: OPC-UA memory exhaustion, historian query floods, HMI screen-update and alarm floods, PLC program-memory / CPU exhaustion, raw bandwidth floods, switch MAC-flooding and Spanning-Tree attacks, and configuration-corruption uploads. These are documented as out of scope for the OT Detection module; some (e.g. raw bandwidth or generic connection floods) may surface indirectly through existing IT-side modules, but they are not claimed as OT detections.

3.7.6 Summary

Testing confirmed that SmartShield reliably detects high-volume flood attacks against OT services from real captures, and **importantly** that detection had to be designed around the *single-connection* flood shape that real attack traffic exhibits, not only the connection-count shape assumed in the original plan. The per-message semantic detections are implemented in the OT module but await the Zeek OT parser integration to fire end-to-end, and a defined set of application- and infrastructure-layer DoS attacks remain outside the module's scope.

4

Why SmartShield? (Market Distinction)

While many Intrusion Detection Systems (IDS) exist in the market (such as Snort or Suricata), SmartShield distinguishes itself through a unique combination of flexibility, behavioral analysis, and analyst-centric design.

Capability	Snort/ Suricata	Zeek	Commercial IPS	SmartShield
OT Protocol Awareness	✗	⚠ Limited	✗	✓ Detects 20 attack categories
Behavioral AI (C&C)	✗	✗	⚠ Limited	✓ RNN-based
Multi-Format Input	✗	Zeek only	✗	✓ 7 formats
Layer 2 (ARP) Detection	✗	✗	✗	✓ Built-in
Data Leak Detection	✗	✗	✗	✓ YARA-based
Time-Windowed Profiling	✗	✗	⚠ Limited	✓ Core feature
Analyst UI (TUI + Web)	Logs only	Logs only	Vendor UI	✓ Dual interfaces
Automatic Blocking	Manual rules	✗	✓	✓ Integrated

4.1 Beyond Signatures: AI-Driven Behavioral Analysis

Traditional market solutions rely heavily on "signatures"; looking for known bad strings of code. SmartShield moves beyond this by using Machine Learning and Behavioral Analysis.

- **RNN C&C Detection:** It employs Recurrent Neural Networks (RNN) to identify Command & Control channels based on behavioral patterns (timing, size, duration) rather than just payload content, detecting "unknown" stealthy threats that signature-based tools miss.

4.2 Unmatched Input Versatility

Most security tools are designed to read only one or two formats (e.g., only live traffic or only PCAP). SmartShield is uniquely adaptable, capable of ingesting and normalizing data from virtually any source:

- **Live Traffic:** For real-time NIDS.
- **PCAP:** For forensic investigations.
- **Flow Data:** Supports Zeek logs, Suricata JSON, Argus flows, and Nfdump. This allows SmartShield to be deployed in diverse scenarios, from heavy enterprise monitoring to specialized wireless security analysis.

4.3 Analyst-First Visualization

Market solutions often output dense, hard-to-read logs. SmartShield prioritizes the human analyst by providing two distinct visualization layers:

- **TUI:** A high-performance, keyboard-driven interface for rapid triage in the terminal.
- **Web-Interface:** A browser-based GUI for deep-dive investigations and visual timelines. Both interfaces visualize the "Life Cycle" of an attack, grouping flows into time windows to help analysts understand the context of an attack, not just see a single alert.

A

SmartShield Network Input Formats

Feature	PCAP	Interface	Zeek Logs	Suricata	Argus	Nfdump	stdin	AP Mode
Data Completeness	100%	100%	85%	70%	40%	40%	Varies	100%
Privacy Risk	High	High	Medium	Low	Low	Low	Varies	High
Storage Efficiency	1x	N/A	0.1x	0.05x	0.01x	0.005x	N/A	1x
Processing Speed	Slow	Real-time	Fast	Fast	Very Fast	Very Fast	Fast	Real-time
Protocol Awareness	Full	Full	High	Medium	Low	Low	Varies	Full
Payload Analysis	Yes	Yes	Limited	Limited	No	No	No	Yes
Enterprise Scalability	Low	Medium	High	High	Very High	Very High	Medium	Low
Deployment Complexity	Low	Medium	High	Medium	Medium	High	Low	High

Table 1.1: Comparative Analysis Matrix

A.A Best Practices & Recommendations

A.A.A Selection Guidelines

1. **Forensic Investigations**: PCAP or Zeek logs
2. **Real-time NIDS**: Live interface or Suricata integration
3. **Enterprise Monitoring**: Nfdump for NetFlow, Argus for custom flows
4. **Wireless Security**: AP mode with specialized 802.11 analysis
5. **Research & Testing**: PCAP with full protocol visibility
6. **Long-term Analysis**: Flow data (Argus/Nfdump) for trend detection

SmartShield's multi-format ingestion capability makes it uniquely adaptable to diverse network security scenarios. From packet-level forensics to flow-based enterprise monitoring and specialized wireless analysis, SmartShield provides a consistent behavioral analysis framework across all data sources. The choice of input format depends on specific requirements for data fidelity, processing resources, deployment constraints, and detection objectives.

Key Takeaway: While PCAP provides the richest data for maximum detection efficacy, flow-based inputs enable scalable monitoring of large networks. The internal normalization ensures that regardless of input source, SmartShield applies the same sophisticated machine learning models for threat detection.

B

Configuration Reference

B.A Main Configuration: `config/smartshield.yaml`

The key parameters changed for factory deployment (from the original defaults):

```yaml
parameters:
  # CHANGED: detect inbound attacks FROM IT side TO OT devices
  analysis_direction: all # was: out
  # CHANGED: keep evidence history across server reboots
  deletePrevdb: false # was: true
  # CHANGED: rotate and keep Zeek logs for 1 week
  rotation: true
  rotation_period: 1day
  keep_rotated_files_for: 7 day
detection:
  # CHANGED: lower threshold so OT attacks alert faster
  evidence_detection_threshold: 0.15 # was: 0.25
ot_detection:
  ot_devices_file: config/ot_protected_devices.conf
  modbus_masters_file: config/modbus_authorized_masters.conf
  modbus_flood_threshold: 20
  # connections/minute before alerting
  s7_job_flood_threshold: 50
  # S7 connections/minute
  dnp3_flood_threshold: 100
  # DNP3 requests/minute
  profinet_dcp_threshold: 50
  # PROFINET DCP requests/minute
  cip_flood_threshold: 100
  # CIP messages/minute
  plc_mode_switch_threshold: 5 # RUN/STOP switches in 60s
  flood_window_seconds: 60
  # sliding window duration
web_interface:
```

```
30    port: 55000
```

B.B SIEM / Syslog Export Configuration

Add `syslog` and/or `webhook` to the `export_to` list in the `exporting_alerts` section:

```yaml
exporting_alerts:
  export_to: [syslog]
  # Options: slack, stix, syslog, webhook
  # Syslog target — Splunk, QRadar, Wazuh, Graylog, Elastic SIEM
  syslog_server: 192.168.1.100
  # Your SIEM server IP
  syslog_port: 514
  # 514 for UDP, 601 for TCP syslog
  syslog_protocol: udp
  # "udp" or "tcp"
  syslog_facility: local0
  # local0 through local7, or daemon, user
  syslog_app_name: smartshield
  # HTTP Webhook — Splunk HEC, Grafana, PagerDuty, custom endpoint
  webhook_url: http://siem-server:8088/services/collector
  webhook_auth_token: your_splunk_hec_token
  webhook_timeout: 5
  # seconds before giving up
```

The `syslog` message format is structured for easy SIEM parsing:

```syslog
smartshield: SmartShield ALERT | type=OT_MODBUS_WRITE_COILS |
threat=CRITICAL | confidence=0.95 | attacker=10.5.1.88 |
ts=2024/03/15 14:32:01.000000+0000 |
desc=Unauthorized Modbus write from 10.5.1.88 to OT device
192.168.10.10...
```

B.C Web Interface Credentials

Set credentials via environment variables; never hardcode them in the config file:

```config
# For bare-metal: edit /etc/smartshield/environment
SMARTSHIELD_WEB_USER=factoryadmin
SMARTSHIELD_WEB_PASSWORD=MyFactory_Secure_Pass2024!
```

```bash
# Generate a cryptographic secret key (run this once):
python3 -c "import secrets; print(secrets.token_hex(32))"
SMARTSHIELD_SECRET_KEY=<paste output here>

# For Docker: set in docker/.env file

# Restart after changes:
sudo systemctl restart smartshield
```

B.D Redis Persistence Configuration

The new config/redis.conf enables both AOF (append-only file) and RDB snapshots:

```config
# AOF persistence - writes every command to disk

appendonly yes
appendfilename "smartshield.aof"
appendfsync everysec
# flush to disk every second (best balance)

# RDB snapshots - periodic full dump as backup

save 900 1
# save after 900s if at least 1 key changed

save 300 10
# save after 300s if at least 10 keys changed

save 60 10000
```

```
# save after 60s if at least 10000 keys changed

# Memory limit - adjust based on available RAM

maxmemory 512mb
maxmemory-policy allkeys-lru

# Dangerous commands disabled in production

rename-command FLUSHALL ""
rename-command FLUSHDB ""
```

Data is stored in `/var/lib/smartshield/redis/` on the host. This directory persists across reboots and Docker container restarts.

C

SmartShield Commands

C.A Systemd Commands (Bare-Metal)

Command	Description
`sudo systemctl status smartshield`	Check if running. Shows last 20 log lines.
`sudo systemctl start smartshield`	Start the service.
`sudo systemctl stop smartshield`	Stop the service. Redis data is preserved.
`sudo systemctl restart smartshield`	Restart after config changes.
`sudo systemctl enable smartshield`	Enable auto-start on boot (already done by install script).
`sudo journalctl -u smartshield -f`	Live log stream (Ctrl+C to stop).
`sudo journalctl -u smartshield --since "1h ago"`	Logs from last hour.
`sudo journalctl -u smartshield --since "2024-03-15"`	Logs from a specific date.
`sudo journalctl -u smartshield -p err`	Show only errors.

C.B Docker Commands

Command	Description
`docker compose -f docker/docker-compose.factory.yml up -d`	Start all services in background.
`docker compose -f docker/docker-compose.factory.yml down`	Stop all services. Volumes (Redis data) are preserved.
`docker compose -f docker/docker-compose.factory.yml restart`	Restart all services.
`docker compose -f docker/docker-compose.factory.yml logs -f`	Follow all container logs.
`docker logs smartshield -f --tail 100`	Follow SmartShield container logs only.
`docker logs smartshield-redis -f`	Follow Redis container logs.
`docker stats`	Live CPU and memory usage for all containers.
`docker exec -it smartshield-redis redis-cli ping`	Test Redis connectivity.

Command	Description
`docker exec -it smartshield-redis redis-cli info persistence`	Check AOF/RDB persistence status.

C.C Useful Diagnostic Commands

```
# ── Network verification
# Check that OT traffic is arriving on the monitoring interface
sudo tcpdump -i eth1 -n port 502 or port 102 or port 20000 -c 30
# Count Modbus packets in last 10 seconds
sudo timeout 10 tcpdump -i eth1 -n port 502 | wc -l
# ── iptables blocking rules
sudo iptables -L smartshieldBlocking -v -n
# Verify no OT device IPs are in the blocked list
sudo iptables -L smartshieldBlocking -n | grep 192.168.10
# ── Redis status
# Check persistence status
redis-cli INFO persistence
# Check OT-protected IPs loaded into Redis
redis-cli SMEMBERS ot_protected_ips
# Count total evidence stored
redis-cli KEYS "evidence_*" | wc -l
# Count alerts triggered
redis-cli KEYS "alert_*" | wc -l
# ── SmartShield output
ls -lht /var/lib/smartshield/output/
tail -f /var/lib/smartshield/output/alerts.json
cat /var/lib/smartshield/output/blocking.log
# ── Zeek status
Factory Internal
 Page
SmartShield IPS — Factory Production Deployment Report
zeek --version
ls /opt/zeek/logs/current/
```

D

Tuning and Optimization

D.A Baseline Period

OT networks have highly predictable, cyclic traffic patterns — PLCs poll sensors at fixed intervals, HMIs refresh at fixed rates. This predictability is a strength for detection but requires careful tuning to avoid false positives.

Recommended tuning approach:

- Week 1–2: Monitor only (no blocking). Collect baseline data. Note the normal Modbus polling rate from your SCADA server.
- Week 3: Lower flood thresholds toward the observed normal + 30% headroom. Watch for false positives.
- Week 4: Enable blocking with OT-safe list fully populated. Start with high thresholds and tighten over time.
- Ongoing: Review blocking.log weekly. Any [OT-SAFE] entries indicate attacks on OT devices that need manual investigation.

D.B Flood Threshold Tuning Guide

D.B.A Parameter Mapping

Parameter	How to Tune	
`modbus_flood_threshold`	Your SCADA polling rate * 2	If SCADA polls 5 PLCs every 5s → normal rate = 60 connections/min → set threshold to 120.
`s7_job_flood_threshold`	TIA Portal polling rate * 3	Engineering workstations poll slowly (1 req/sec). Attacks are 50+/sec → keep at 50.
`dnp3_flood_threshold`	RTU polling rate * 3	RTUs typically send 1–5 frames/second. Attacks send 100+/sec → start at 100.
`profinet_dcp_threshold`	Near 0 in normal operation	DCP discovery is only sent during device startup. Any sustained rate > 10 is suspicious.

Parameter	How to Tune	
`plc_mode_switch_threshold`	Set to 2 if one SCADA can switch PLC	A legitimate mode switch is 1 per maintenance window. 5 in 60s is always an attack.

D.C Evidence Detection Threshold

The `evidence_detection_threshold` (in `detection:` section of `smartshield.yaml`) controls how much accumulated threat level is needed before an Alert is generated.

Threshold	Effect
`0.08`	Very sensitive. Initial testing mode; expect false positives.
`0.15`	Balanced factory setting for OT environments with sudden high-confidence attacks.
`0.25`	IT-oriented default; too slow for OT response.
`0.43`	Insensitive; requires many corroborating evidence objects, may miss fast attacks.

D.D Performance Considerations

- Zeek CPU: Heaviest component. At 1 Gbps sustained, uses 2–3 cores.
- Redis memory: Default 512 MB limit. Each IP profile 50 KB. Supports 10,000 concurrent profiles.
- OT detection overhead: Minimal. Uses in-memory lists with bounded O(n) cleanup.
- Log rotation: `rotation: true`, `rotation_period: 1day`. Zeek logs kept for 7 days max.
- Web interface: Slowdowns usually indicate Redis query latency. Check `redis-cli INFO stats`.

E

Troubleshooting

E.A Common Problems and Solutions

Problem	First Check	Solution/Cause
OT module not loading	`journalctl -u smartshield | grep OT`	Missing `__init__.py`. Fix: create file in `modules/ot_detection/` and restart service.
No Modbus alerts despite flood traffic	`tcpdump -i eth1 port 502`	SPAN not configured OR thresholds too high OR device not in OT protected list.
Web UI shows no alerts after 1 hour	`redis-cli KEYS "alert_*"`	Evidence threshold too high or long time window. Lower to 0.08 for testing.
Redis data lost after restart	`redis-cli INFO persistence`	AOF disabled. Ensure `appendonly yes` and writable path.
Cannot log into web interface	`curl -I http://localhost:55000/login`	Missing `SMARTSHIELD_WEB_USER` or password misconfigured.
Syslog not reaching SIEM	`nc -u <siem-ip> 514`	Firewall or misconfigured syslog settings.
Factory line halted – PLC was blocked	`iptables -D smartshieldBlocking ...`	Immediately whitelist PLC in `ot_protected_devices.conf`.
Zeek OT script load error	`zeek -i eth1 zeek-scripts/ | head`	Missing OT bundle; install ICS package or rely on fallback mode.
High CPU usage	`top -p $(pgrep -f smartshield.py)`	Enable profiling or reduce traffic load.
`OT_UNAUTHORIZED_MODBUS_ACCESS` false positives	Check `modbus_authorized_masters.conf`	Add the source IP if it is a legitimate Modbus master. Or leave the file

Problem	First Check	Solution/Cause
		empty to disable this specific check.

E.B Emergency Procedures

If SmartShield blocks an OT device:

1. Step 1:

```bash
sudo iptables -D smartshieldBlocking -s <plc-ip> -j DROP
sudo iptables -D smartshieldBlocking -d <plc-ip> -j DROP
```

2. Step 2:

```bash
echo "192.168.10.10 # PLC-01 EMERGENCY ADDED" >> config/ot_protected_devices.conf
```

3. Step 3:

```bash
sudo systemctl restart smartshield
```

4. Step 4:

```bash
sudo iptables -L smartshieldBlocking -n | grep 192.168.10.10
```

5. Step 5: Investigate original alertto determine if there was a real attack.

E.C Log File Locations

Log File	Location and Contents
alerts.json	/var/lib/smartshield/output/alerts.json: All generated alerts in JSON format
blocking.log	/var/lib/smartshield/output/blocking.log: All blocking/unblocking events with [OT-SAFE] tags
smartshield.log	/var/lib/smartshield/output/smartshield.log: Main application log
errors.log	/var/lib/smartshield/output/errors.log: Error and exception log

Zeek conn.log	/var/lib/smartshield/output/zeek/conn.log: All network connections parsed by Zeek
Redis AOF	/var/lib/smartshield/redis/smartshield.aof: Persistent Redis command log
systemd journal	journalctl -u smartshield: Full service log including startup/shutdown

F

Security Hardening Checklist

Before going live in a factory environment, verify each item:

F.A Network Hardening

- SPAN port: Monitoring interface (`eth1`) has no IP address assigned. Run: `ip addr show eth1` — should show no inet line.
- Firewall: Port 55000 is only accessible from management VLAN. Verify: `sudo ufw status numbered`
- Management interface: `eth0` has only necessary ports open (22/SSH, 55000/Web). No services listening on `eth1`.
- Redis binding: Redis only listens on `127.0.0.1` (confirmed in redis.conf: bind 127.0.0.1)

F.B Authentication Hardening

- Default password changed: `SMARTSHIELD_WEB_PASSWORD` is not SmartShield2024!. Verify no warning banner in web UI.
- Secret key set: `SMARTSHIELD_SECRET_KEY` is a 32-byte random hex string. If not set, sessions invalidate on restart.
- Environment file permissions: `chmod 600 /etc/smartshield/environment` — only root can read credentials.
- No credentials in git: Verify .env and environment files are in `.gitignore`

F.C OT Safety Verification

- OT device list populated: `config/ot_protected_devices.conf`has all PLCs, HMIs, and SCADA servers.
- Protected IPs in Redis: `redis-cli SMEMBERS ot_protected_ips` — should list all configured OT IPs.
- No OT IPs blocked: `sudo iptables -L smartshieldBlocking -n`: must not show any PLC/HMI/SCADA IPs.
- `blocking.log` reviewed: No unexpected [OT-SAFE] entries — each one means an attack hit a protected OT device.

F.D Data Persistence Verification

- AOF enabled: redis-cli INFO persistence — shows aof_enabled:1 and `aof_rewrite_in_progress:0`.
- AOF file exists: `ls -lh /var/lib/smartshield/redis/smartshield.aof` — should be non-empty after 5 minutes.

- Reboot test: `sudo reboot`, then verify SmartShield auto-starts and evidence history is preserved.

F.E Monitoring and Alerting

- Syslog tested: Trigger a test alert and verify it arrives in SIEM within 30 seconds.

- Systemd watchdog: Restart policy is on-failure. Test: `sudo kill -9 $(pgrep -f smartshield.py)` — service should restart within 10 seconds.
- Disk space monitor: Set up a cron job or monitoring alert if /var/lib/smartshield exceeds 80% full.
- Health check script: Consider a cron job that pings http://localhost:55000 and sends alert if web UI is down.

G

Quick Reference

G.A All OT Evidence Types

Evidence Type	Threat Level	Description
OT_MODBUS_CONNECTION_FLOOD	HIGH	Port 502 TCP flood — too many connections/min
OT_UNAUTHORIZED_MODBUS_ACCES	HIGH	Modbus from non-authorized master IP
OT_MODBUS_WRITE_COILS	CRITICAL	Unauthorized Modbus write coil/register command
OT_MODBUS_ILLEGAL_FUNCTION	MEDIUM	Malformed/exception Modbus function code
OT_S7COMM_JOB_FLOOD	HIGH	S7 port 102 connection flood
OT_S7COMM_UNAUTHORIZED_READ	HIGH	S7 connection from non-authorized source
OT_S7COMM_UNAUTHORIZED_WRITE	CRITICAL	S7 WriteVar from unauthorized host
OT_S7COMM_PLC_STOP	CRITICAL	S7 STOP command sent to PLC
OT_DNP3_FLOOD	HIGH	DNP3 request flood (port 20000)
OT_DNP3_UNAUTHORIZED_FUNCTION	HIGH	DNP3 function code > 33
OT_PROFINET_DCP_FLOOD	MEDIUM	PROFINET DCP discovery flood (port 34964)
OT_CIP_MESSAGE_FLOOD	HIGH	EtherNet/IP CIP explicit message flood (port 44818)
OT_NTP_TIME_SYNC_ATTACK	HIGH	NTP spoofing → 5 NTP responses/10s
OT_PTP_SPOOFING	HIGH	PTP grandmaster spoofing (port 319 UDP)
OT_PLC_MODE_SWITCH	CRITICAL	Rapid PLC RUN/STOP mode switching
OT_WATCHDOG_MANIPULATION	CRITICAL	Watchdog timer disabled or manipulated
OT_VALVE_CYCLING	CRITICAL	Rapid valve open/close commands
OT_MOTOR_CYCLING	CRITICAL	Rapid motor start/stop commands
OT_MODBUS_SCAN	MEDIUM	Reconnaissance scan across Modbus unit IDs / function codes on port 502 — reserved for future per-

Evidence Type	Threat Level	Description
		PDU detection (not yet active in this build)
OT_PROTOCOL_ANOMALY	MEDIUM	Generic OT protocol anomaly (Zeek notice)

G.B Key File Paths

Item	Path
Main config	/opt/SmartShield-Factory/config/smartshield.yaml
OT protected devices	/opt/SmartShield-Factory/config/ot_protected_devices.conf
Authorized Modbus masters	/opt/SmartShield-Factory/config/modbus_authorized_masters.conf
Redis config	/opt/SmartShield-Factory/config/redis.conf
Whitelist	/opt/SmartShield-Factory/config/whitelist.conf
TI feeds list	/opt/SmartShield-Factory/config/TI_feeds.csv
Service environment file	/etc/smartshield/environment
Systemd service	/etc/systemd/system/smartshield.service
Redis data	/var/lib/smartshield/redis/
Alerts output	/var/lib/smartshield/output/alerts.json
Blocking log	/var/lib/smartshield/output/blocking.log
Zeek logs	/var/lib/smartshield/output/zeek/
System logs	journalctl -u smartshield

G.C Port Reference

Port	Protocol	OT Devices
502/TCP	Modbus/TCP	PLCs, remote I/O modules, Modbus gateways
102/TCP	S7Comm	Siemens S7-300, S7-400, S7-1200, S7-1500 PLCs (ISO-TSAP)
20000/TCP+UDP	DNP3	RTUs, substations, water/power utility systems
4840/TCP	OPC-UA	SCADA servers, historians, modern PLC gateways
34964/UDP	PROFINET DCP	Siemens PROFINET devices, distributed I/O
44818/TCP	EtherNet/IP CIP	Allen-Bradley (Rockwell) PLCs, ControlLogix, CompactLogix
123/UDP	NTP	Time synchronization for PLCs and SCADA servers

Port	Protocol	OT Devices
319/UDP	PTP (IEEE 1588)	Precision time for motion control and substations
55000/TCP	SmartShield Web UI	Management interface — restrict to management VLAN only
6379/TCP	Redis	Internal only — localhost binding, not exposed externally

References

[1] "(PDF) Stuxnet." [Online]. Available: https://www.researchgate.net/publication/323199431_Stuxnet

[2] M. Pollard, "A Case Study of Russian Cyber-Attacks on the Ukrainian Power Grid: Implications and Best Practices for the United States," *Pepperdine Policy Review*, vol. 16, no. 1, Jan. 2024, [Online]. Available: https://digitalcommons.pepperdine.edu/ppr/vol16/iss1/1

[3] "(PDF) Analysis of the Colonial Pipeline Cybersecurity Incident." [Online]. Available: https://www.researchgate.net/publication/387104186_Analysis_of_the_Colonial_Pipeline_Cybersecurity_Incident

[4] "MITRE ATT&CK®." [Online]. Available: https://attack.mitre.org/

[5] "Techniques - ICS | MITRE ATT&CK®." [Online]. Available: https://attack.mitre.org/techniques/ics/

[6] Z. Xu *et al.*, "Deep Learning-based Intrusion Detection Systems: A Survey." [Online]. Available: http://arxiv.org/abs/2504.07839

[7] A. Hozouri, A. Mirzaei, and M. Effatparvar, "A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges," *Discover Artificial Intelligence*, vol. 5, no. 1, p. 314, Nov. 2025, doi: 10.1007/s44163-025-00578-1.

[8] M. Abrams and J. Weiss, "Malicious Control System Cyber Security Attack Case Study: Maroochy Water Services, Australia," Aug. 2008, [Online]. Available: https://www.mitre.org/news-insights/publication/malicious-control-system-cyber-security-attack-case-study-maroochy-0

[9] V. Sharma, D. Shah, S. Sharma, and S. Gautam, "Artificial Intelligence based Intrusion Detection System – A Detailed Survey," *ITM Web of Conferences*, vol. 65, p. 4002, 2024, doi: 10.1051/itmconf/20246504002.

[10] H. Nguyen and D. Choi, "Network Anomaly Detection: Flow-based or Packet-based Approach?." [Online]. Available: http://arxiv.org/abs/1007.1266

[11] B. Andreas, J. Dilruksha, and E. McCandless, "Flow-Based and Packet-Based Intrusion Detection Using BLSTM," vol. 3, no. 3, 2020.

[12] Y. Reddy, "Artificial Intelligence in Intrusion Detection Systems: Trends, Frameworks, and Future Directions for Cybersecurity," *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 17, pp. 949–959, Feb. 2024, [Online]. Available: https://ijisae.org/index.php/IJISAE/article/view/7689

[13] "(PDF) A Survey on Advanced Persistent Threats: Techniques, Solutions, Challenges, and Research Opportunities," *ResearchGate*, doi: 10.1109/COMST.2019.2891891.